Universidad Tecnológica Nacional — Facultad Regional Mendoza
Tecnicatura Universitaria en Programación

Integración de un honeypot de media interacción con n8n para la captura, el procesamiento y el reporte automatizados de eventos de ataque: prototipo y validación funcional

Trabajo Final — Versión final

Grupo Nocturne Society
Integrantes: Emmanuel Crespo, Lucas Norton, Julián Santos
Profesores: Alberto Cortez y Ariel Enferrel

Repositorio: https://github.com/lucasnorton01/Honeypot_Final_Nocturne_Society
Tag: Honeypot_Final_2026-10-08g (commit
930acf617ca8c2cb20860aec812deccb074afd5e)

Mendoza, 2026

Índice general

Resumen 8
Abstract 9
Glosario de códigos y abreviaturas 10
Capítulo I: Introducción 11
1.1 Introducción y contextualización del problema 11
1.2 Planteamiento del problema 11
1.3 Justificación de la investigación 12
1.4 Objetivos de la investigación 12
1.5 Alcances y delimitaciones 13
1.6 Hipótesis general e hipótesis operativas 14
1.7 Variables de análisis 15
1.8 Estructura del trabajo 15
Capítulo II: Estado del Arte 16
2.1 Introducción 16
2.2 Evolución reciente del panorama de amenazas 16
2.3 Tendencias actuales en defensa cibernética 16
2.4 Estado actual del uso de honeypots 16
2.5 Evolución de honeypots: de Spitzner a Cowrie y Dionaea 17
2.6 Proyectos relacionados 18
2.7 Posicionamiento de la arquitectura propuesta 19
2.8 Descripción de la búsqueda bibliográfica 21
Capítulo III: Marco Teórico 23
3.1 Introducción 23
3.2 Ciberseguridad y cambio de paradigma defensivo 23
3.3 Seguridad de la información y ciberseguridad 23
3.4 Teoría del engaño en ciberseguridad 24
3.5 Honeypots como mecanismos de observación 24
3.6 Inteligencia de amenazas (Threat Intelligence) 27
3.7 Ciclo de vida de la inteligencia de amenazas (modelo PCPAD) 29
3.8 Análisis forense digital 30
3.9 Automatización en seguridad 30
3.10 Modelos de madurez de gestión de incidentes (SIM3) 32
3.11 Indicadores de compromiso (IoCs) 32
3.12 Integración conceptual del modelo propuesto 33
Capítulo IV: Metodología 35
4.1 Introducción 35
4.2 Enfoque metodológico 35
4.3 Tipo de investigación 36
4.4 Diseño de investigación 36
4.5 Problema de investigación 36

4.6 Hipótesis general e hipótesis operativas 37
4.7 Variables de investigación 38
4.8 Operacionalización con métricas 39
4.9 Criterios de validación del sistema 39
4.10 Población, unidad de análisis y muestra 40
4.11 Escenario de validación 41
4.12 Procedimiento 41
4.13 Métodos de análisis 41
4.14 Validez y confiabilidad 42
4.15 Consideraciones éticas 43
4.16 Limitaciones metodológicas 43
Capítulo V: Diseño e Implementación del Prototipo 44
5.1 Introducción 44
5.2 Arquitectura del prototipo 44
5.3 Herramientas utilizadas 46
5.4 Componentes desplegados 46
5.5 Red e instrumentación 47
5.6 Implementación de workflows n8n 48
5.7 Modelo de datos 51
Capítulo VI: Resultados de la Validación Funcional 52
6.1 Introducción 52
6.2 Diseño del entorno de validación 52
6.3 Análisis cuantitativo 55
6.4 Observaciones generales del entorno 57
6.5 Caracterización de la actividad observada 58
6.6 Métricas de evaluación del sistema 58
6.7 Resultados del procesamiento automatizado 59
6.8 Generación de inteligencia de amenazas 59
6.9 Validación de las hipótesis operativas 61
6.10 Cumplimiento de objetivos 63
6.11 Limitaciones de los resultados y amenazas a la validez 64
6.12 Comparación con el procesamiento manual 65
6.13 Validación ampliada: R1 y réplica C 69
Capítulo VII: Discusión 72
7.1 Interpretación de la comparación con el procesamiento manual 72
7.2 Interpretación de resultados 72
7.3 Comparación con literatura existente 73
7.4 Limitaciones del estudio 74
7.5 Implicaciones prácticas 75
7.6 Evaluación crítica de n8n como plataforma de automatización CTI 75
7.7 Calidad y utilidad de los IoCs generados 77
Capítulo VIII: Conclusiones 79
8.1 Síntesis del problema abordado 79
8.2 Evaluación de hipótesis y contribuciones 79

8.3 Síntesis global 83
8.4 Trabajo futuro 83
8.5 Trabajo futuro priorizado 84
8.6 Cierre 85
Capítulo IX: Recomendaciones 86
9.1 Introducción 86
9.2 Recomendaciones para instituciones educativas 86
9.3 Recomendaciones para PyMEs 86
9.4 Recomendaciones para investigadores 87
9.5 Matriz resumen de recomendaciones 87
Anexos 88
Anexo I: Diagramas de arquitectura 88
Anexo II: Workflows n8n exportados 91
Anexo III: Datos crudos, tablas completas y tablas de apoyo conceptual 91
Anexo IV: Herramienta de demostración para la defensa 97
Anexo V: Ventanas anteriores al 25/09/2026 y limitaciones técnicas identificadas en un segundo entorno 98
Anexo VI: Bitácora de las ventanas de validación 99
Anexo VII: Detalle de los contrastes de P1 a P4 y consultas SQL 105
Anexo VIII: Medición de la línea de base manual 114
Anexo IX: Modelo de madurez SIM3 y estándares de representación de IoCs 117
Anexo X: Trazabilidad del repositorio 120
Referencias 121

Índice de figuras

Figura 5.1: Arquitectura general de la solución por capas, en la configuración final. Dionaea figura con línea punteada como componente previsto en el diseño objetivo, no desplegado (§5.4). En esta configuración log-reader y postgres no publican puertos, Cowrie publica los suyos mediante cowrie-proxy, n8n ejecuta cuatro workflows (incluido health-monitor) y las redes Docker son captura, proceso y datos (internal) más entrada y salida (Tabla 5.3, §5.5). El simulador atacó por Telnet (puerto 2323) en el lote L0 y por SSH (puerto 2222) en las validaciones W1, W2, C1, C2 y C3. Fuente: Elaboración propia. 45

Figura 5.2: Topología de red con segmentación VLAN y reglas de firewall — diseño objetivo. El prototipo no implementa VLAN ni firewall: las validaciones L0 a C1 usaron una red Docker única; desde el 05/10/2026 (validación C2) sus tres capas se implementan con redes Docker (10.0.1.0/24, 10.0.2.0/24 y 10.0.3.0/24, rangos privados; Rekhter et al., 1996), con la red de datos como internal, y desde el 06/10/2026 las tres son internal, más una red de entrada y otra de salida (§5.5). Fuente: Elaboración propia. 45

Figura 5.3: Diagrama de contenedores Docker con puertos y red, con las direcciones asignadas por Docker en el lote L0 (subred 172.18.0.0/16). La subred fue 172.19.0.0/16 en W1 y W2 y 172.18.0.0/16 en C1. En las tres el simulador se ejecutó desde attack-runner (§5.5). Fuente: Elaboración propia. 47

Figura I-1: Diagrama de secuencia del procesamiento de un evento de ataque, en tres partes: (A) ingesta, (B) extracción de 89

Figura I-2: Flujo de datos desde la captura del evento hasta la generación del reporte, con el extractor vigente en la validación del 30/09/2026 (filtro de ítems vacíos, registro en ioc_sessions y rama de error hacia error_log). Fuente: 90

Figura III-1: Modelo de datos del prototipo (seis tablas de PostgreSQL definidas en db/schema.sql). Fuente: Elaboración propia. 93

Índice de tablas

Tabla 2.1: Comparación de la propuesta con los antecedentes recientes más cercanos. 18
Tabla 3.1: Modelos tradicionales de detección. 23
Tabla 3.2: Taxonomía multidimensional de honeypots. 25
Tabla 3.3: Niveles de inteligencia de amenazas. 27
Tabla 3.4: Ciclo PCPAD: propósito, actividades y mapeo a los componentes implementados. 29
Tabla 3.5: Correspondencia entre principios teóricos y decisiones de arquitectura. 33
Tabla 4.1: Validaciones realizadas y papel de cada una en el análisis. 37
Tabla 4.2: Decisiones tomadas antes y después de observar los datos. 38
Tabla 4.3: Operacionalización de variables mediante métricas. 39
Tabla 5.1: Herramientas utilizadas y previstas, y su rol en la arquitectura. 46
Tabla 5.2: Servicios del prototipo, dirección en la red de laboratorio y puertos publicados. 47
Tabla 5.3: Configuración validada y configuración final del prototipo 48
Tabla 6.1: Sesiones por lote, persistencia y cobertura (denominador: sesiones persistidas). 55
Tabla 6.2: Resultados de las ventanas W1 y W2. 55
Tabla 6.3: Sesiones persistidas del lote 2 55
Tabla 6.4: Credenciales intentadas en el lote 2. 57
Tabla 6.5: Indicadores de validación del lote 2. 58
Tabla 6.6: Modo de ejecución de los workflows en la validación L0. 59
Tabla 6.7: Contenido de la tabla iocs tras la ejecución de ioc-extractor. 59
Tabla 6.8: Cumplimiento de los objetivos específicos. 63
Tabla 6.9: Limitaciones de los resultados y amenazas a la validez. 64
Tabla 6.10: Comparación de métricas entre procesamiento manual estimado y sistema automatizado. 67
Tabla 6.11: R1: criterios registrados de antemano y resultados. 69
Tabla 7.1: Limitaciones del estudio: impacto y mitigación 74
Tabla 7.2: Comparación de IoCs locales frente a feeds públicos de inteligencia colectiva. 78
Tabla 8.1: Verificación de las hipótesis operativas frente a los resultados (P1 y P2: lote L0, N = 5 sesiones, 65 eventos; P3 y P4: validación W2, N = 41 sesiones; antecedente: W1, N = 10; contrastes con reglas escritas de antemano: C1, N = 15; C2, N = 66, y C3, N = 49). 80
Tabla 8.2: Priorización del trabajo futuro 84
Tabla 9.1: Costos estimados de despliegue. 86
Tabla 9.2: Matriz resumen de recomendaciones. 87
Tabla III-1: Tiempos de procesamiento por evento (sesión de referencia 10–11/08/2026 — referencia complementaria) 93
Tabla III-1b: Latencia de persistencia por evento — lote del 25/09/2026 (medición principal de P1). 94
Tabla III-6: Comparación de precisión de parseo manual vs automatizado. 94
Tabla III-7: Comparación de plataformas de automatización para CTI 95
Tabla III-8: Línea temporal de hitos clave en la evolución de honeypots. 96
Tabla III-9: Protocolo de búsqueda bibliográfica. 96
Tabla III-11: Comparación ampliada entre SOAR empresarial, n8n y Node-RED. 97

Tabla III-13: Alternativas abiertas de orquestación y de plataformas de honeypots no evaluadas en esta tesis. 97
Tabla VI-1: Cronología de la validación del 25/09/2026. 104
Tabla VI-2: Resultados de la validación C3 (08–09/10/2026). 104
Tabla VII-1: IoCs atribuidos por sesión mediante event_id. 107
Tabla VII-2: IoCs atribuidos por sesión mediante event_id (validación del 28/09/2026). 109
Tabla VII-3: P3 en la validación del 30/09/2026 (N = 41 sesiones con autenticación exitosa). 110
Tabla VII-4: Cobertura por reportes en modo trigger de las sesiones con autenticación exitosa (validación del 28/09/2026). 112
Tabla VII-5: Cobertura por reportes en modo trigger de las sesiones con autenticación exitosa (validación del 30/09/2026). 113
Tabla VIII-1: Descomposición del tiempo de procesamiento manual por fase. 116
Tabla IX-1: Autoevaluación integrada de madurez de la arquitectura. 118
Tabla IX-2: Estándares de representación, transporte y enriquecimiento de IoCs. 119
Tabla X-1: Tags del repositorio citados en esta tesis. 120

Resumen

Las PyMEs y las instituciones educativas suelen carecer de equipos para convertir eventos de ataque en inteligencia de amenazas. Este trabajo construye y valida, en un laboratorio sin exposición entrante desde Internet, un prototipo que integra un honeypot de media interacción (Cowrie) con n8n y PostgreSQL: captura, normaliza y persiste eventos, extrae indicadores de compromiso (IoC) y emite reportes. Se despliega con Docker Compose; las VLAN con firewall y Dionaea quedan como diseño objetivo. Los objetivos general, 3 y 5 se reformularon para ajustarlos a ese alcance (§1.4).

La validación usa ataques simulados por el propio equipo desde una única dirección interna y, en R1, 200 sesiones reales de un dataset público reproducidas por los autores. Los umbrales de las cuatro hipótesis operativas se fijaron de antemano; la ventana de veredicto, la definición de «ataque de interés» (sesión con autenticación exitosa) y la categoría «parcialmente sostenida», al analizar los resultados. Se confirman P1 (ingesta de 0,3 a 1,7 s por evento frente a 10 a 18 s de triaje manual, tareas no equivalentes) y P2 (eventos persistidos coincidentes con el log de Cowrie), ambas con escaso poder discriminante. No se confirman P3 (IoC por sesión: 14,6 %, por un defecto de diseño del catálogo) ni P4 (cobertura de reportes: 85,4 % frente al 90 % exigido): la hipótesis general queda parcialmente sostenida. El reporte obtuvo una utilidad percibida de 3,0 sobre 5 (umbral: 4,0). R1 no modifica ningún veredicto.

Se concluye que la integración es técnicamente viable en laboratorio para capturar, procesar y persistir eventos y para extraer indicadores y emitir reportes, pero no alcanza los umbrales de atribución por sesión ni de cobertura de reportes. No se demuestra inteligencia de amenazas con valor operativo, porque los indicadores provienen de ataques simulados o de sesiones reales reproducidas por los propios autores.

Palabras clave: honeypots; inteligencia de amenazas; n8n; automatización; ciberseguridad; indicadores de compromiso (IoC).

Abstract

Small and medium-sized enterprises and educational institutions often lack teams to turn attack events into threat intelligence. This work builds and validates, in a laboratory with no inbound exposure from the Internet, a prototype that integrates a medium-interaction honeypot (Cowrie) with n8n and PostgreSQL: it captures, normalizes and persists events, extracts indicators of compromise (IoC) and issues reports. It is deployed with Docker Compose; VLANs with firewalling and Dionaea remain a target design. The general objective and objectives 3 and 5 were reworded to fit that scope (§1.4).

Validation uses attacks simulated by the team itself from a single internal address and, in R1, 200 real sessions from a public dataset replayed by the authors. The thresholds of the four operational hypotheses were fixed beforehand; the verdict window, the definition of «attack of interest» (a session with successful authentication) and the category «partially supported», when analysing the results. P1 (ingestion of 0.3 to 1.7 s per event against 10 to 18 s of manual triage, not equivalent tasks) and P2 (persisted events matching the Cowrie log) are confirmed, both with little discriminating power. P3 (IoC per session: 14.6%, because of a design flaw in the catalog) and P4 (report coverage: 85.4% against the required 90%) are not confirmed: the general hypothesis is partially supported. The report obtained a perceived usefulness of 3.0 out of 5 (threshold: 4.0). R1 does not change any verdict.

We conclude that the integration is technically viable in a laboratory to capture, process and persist events and to extract indicators and issue reports, but it does not reach the thresholds for per-session attribution or report coverage. Threat intelligence with operational value is not demonstrated, because the indicators come from simulated attacks or from real sessions replayed by the authors themselves.

Keywords: honeypots; threat intelligence; n8n; automation; cybersecurity; indicators of compromise (IoC).

Glosario de códigos y abreviaturas

Las validaciones, las hipótesis y los modos de ejecución se citan en el documento por los códigos siguientes; la Tabla 4.1 describe cada validación. Los códigos se agrupan en familias: lote y ventanas (L0, W1, W2), contrastes con registro previo (C1 a C3), validación ampliada (R1 y Réplica C), línea de base manual (B1, B1b, B1c) e hipótesis (P1 a P4 y P3′). Todas las fechas son de 2026.

L0 — Lote del 25/09/2026: lote 1 (sin eventos, con la ingesta inactiva) y lote 2 (5 sesiones, 65 eventos). Sustenta P1 y P2 y es el primer contraste de P3 y P4.

W1 — Ventana automática del 28/09/2026 (≈ 7 h 32 min; 10 sesiones con autenticación exitosa). Es antecedente: no determina veredictos.

W2 — Ventana automática del 30/09/2026 (8 h 07 min; 41 sesiones con autenticación exitosa). Es la ventana de veredicto de P3 y P4, designada después de observar los resultados.

C1 — Contraste del 02/10/2026 (67 min; 15 sesiones), con reglas escritas antes de la ventana pero publicadas después.

C2 — Contraste del 05–06/10/2026 (14 h; 66 sesiones), con registro previo publicado antes del inicio.

C3 — Contraste del 08–09/10/2026 (12 h; 49 sesiones), con registro previo publicado antes del inicio, sobre la configuración final del 06/10.

R1 — Validación ampliada del 10–11/10/2026: 200 sesiones reales de un dataset público (1.020 comandos), reproducidas por los autores desde una dirección interna, con registro previo publicado antes de la primera sesión. No modifica los veredictos.

Réplica C — Réplica independiente: una persona ajena al equipo levanta el sistema siguiendo solo el README. No es un contraste (C1 a C3); se ejecutó una vez el 11/10/2026 y resultó exitosa, con asistencia de IA para ejecutar los comandos y sin intervenciones del equipo (§6.13.4).

B1 — Medición de la línea de base manual con cinco participantes ajenos al equipo (05/10/2026).

B1b — Intento de ampliar la línea de base con más participantes ajenos al equipo; no se completó y no se usa en ninguna cifra.

B1c — Ampliación de la línea de base con registro previo, con diez participantes ajenos al equipo (08/10/2026).

P1 a P4 — Hipótesis operativas: reducción de tiempo (P1), estructuración de datos (P2), generación de IoCs (P3) y reportes automatizados (P4).

P3′ — Completitud por sesión, reformulación de P3: la sesión cuenta si ioc_sessions contiene todos los indicadores que las reglas del extractor derivan del log de Cowrie de esa sesión, con el mismo umbral del 70 %. Se definió después de observar W1 y no modifica el veredicto de P3.

Modo trigger — Ejecución disparada por la programación (cron) del propio workflow, sin intervención humana; es la única que se computa como automática.

Modo manual — Ejecución lanzada a mano desde el editor o desde la línea de comandos de n8n (modo manual o cli); no se computa como automática.

Capítulo I: Introducción

1.1 Introducción y contextualización del problema

Durante las últimas décadas, la transformación digital ha modificado la manera en que organizaciones públicas, empresas e individuos gestionan información y prestan servicios. La adopción masiva de infraestructuras conectadas, servicios en la nube, dispositivos móviles, automatización industrial e IoT ha incrementado la superficie de exposición frente a amenazas cada vez más sofisticadas. Los actores maliciosos han evolucionado hacia estructuras profesionalizadas con altos niveles de automatización —fuerza bruta, explotación de vulnerabilidades, ransomware, botnets, robo de credenciales—, y gran parte de las organizaciones sostiene modelos defensivos reactivos (firewalls, antivirus, IDS/IPS y EDR responden a patrones conocidos). Los honeypots son un enfoque complementario de observación activa: sistemas diseñados para simular servicios legítimos, atraer interacciones maliciosas y obtener inteligencia accionable sin comprometer activos productivos. Esta investigación propone el diseño e implementación de una arquitectura de honeypots integrada con la plataforma de automatización n8n, orientada a transformar eventos de ataque en inteligencia de amenazas procesable de forma automática. La arquitectura se organiza en cuatro capas funcionales —captura, procesamiento, persistencia y análisis—; la Figura 5.1 (§5.2) las desglosa en cinco capas de despliegue (simulación, exposición, reenvío, automatización y persistencia) y la Figura 5.2 las agrupa en tres segmentos de red (captura, proceso y datos).

1.2 Planteamiento del problema

Las organizaciones modernas enfrentan una asimetría operativa significativa en materia de ciberseguridad: el atacante necesita encontrar una única debilidad explotable, mientras que el defensor debe proteger toda su infraestructura, y la desigualdad se profundiza cuando los actores hostiles emplean automatización y herramientas de código abierto. Uno de los principales problemas en entornos reales es la falta de visibilidad sobre el comportamiento del adversario: los registros tradicionales indican que ocurrió un escaneo o un intento de acceso, pero no permiten reconstruir qué credenciales se intentan, qué comandos se ejecutarían tras un acceso exitoso, qué malware se descarga o qué IoCs podrían derivarse del evento. Además, muchos equipos enfrentan sobrecarga de alertas, escasez de personal especializado y limitaciones presupuestarias, y en el ámbito académico numerosos despliegues de honeypots se realizan sin metodología documentada ni criterios de validación definidos. En consecuencia, el problema central puede formularse así: ¿En qué medida una arquitectura de honeypots integrada con herramientas de automatización de acceso libre mejora la generación de

inteligencia de amenazas local, la estructuración de los eventos técnicos y el tiempo de procesamiento, en comparación con un enfoque manual no automatizado? El fortalecimiento de la capacidad de respuesta defensiva queda fuera de lo medido (§1.5).

1.3 Justificación de la investigación

La relevancia de esta investigación se sustenta en dimensiones académicas, técnicas, operativas, económicas y estratégicas. Académicamente, aporta una aplicación concreta de conocimientos de programación, redes, seguridad informática, bases de datos y automatización, mediante una arquitectura reproducible orientada a una problemática actual (Morić et al., 2025). Técnicamente, los honeypots dan evidencia empírica sobre ataques (Nawrocki et al., 2016) de alto valor analítico, porque no existe tráfico legítimo esperado. Operativamente, la automatización con n8n reduce tareas manuales repetitivas y genera alertas y reportes de forma automática, algo valioso para organizaciones con recursos humanos limitados.

Económicamente, el uso de herramientas de código abierto o gratuito (n8n se distribuye bajo licencia fair-code) reduce barreras de entrada para instituciones educativas, pequeñas empresas y organismos públicos sin grandes presupuestos. Como indicio histórico, y con la salvedad de su antigüedad, en la encuesta de SANS de 2015 el 45 % de los encuestados que respondieron sobre la integración de inteligencia de amenazas en sus sistemas señaló el aprovechamiento de sistemas e inteligencia internos como práctica que contribuyó al éxito de sus programas de inteligencia de amenazas (Shackleford, 2015); la inteligencia generada localmente permite adaptar los controles defensivos a evidencia propia y no solo a reportes globales.

1.4 Objetivos de la investigación

1.4.1 Objetivo general

Diseñar e implementar una arquitectura de honeypots integrada con la plataforma n8n para automatizar la recolección, procesamiento y generación de indicadores y reportes de amenazas, verificados mediante prueba funcional controlada.

1.4.2 Objetivos específicos

1. Analizar el marco conceptual de honeypots, deception security e inteligencia de amenazas.
2. Diseñar una arquitectura de red segmentada para el despliegue de sensores honeypot e implementar el prototipo en un entorno de laboratorio contenerizado sin exposición a Internet.
3. Implementar un honeypot de media interacción orientado a capturar intentos de acceso, credenciales y comandos ejecutados tras la autenticación.
4. Integrar los eventos recolectados con flujos automatizados en n8n.

5. Incorporar al pipeline un proceso de enriquecimiento contextual de los eventos mediante geolocalización de la IP de origen y verificar su funcionamiento.

6. Evaluar la utilidad operativa del sistema mediante métricas de procesamiento, trazabilidad y capacidad analítica.

7. Elaborar recomendaciones técnicas para replicabilidad institucional futura.

Nota. Tres objetivos se reformularon para ajustarlos al alcance del laboratorio. Objetivos 3 y 5: durante el desarrollo, con conformidad de la dirección del trabajo (09/10/2026). Redacción original: «3. Implementar honeypots orientados a capturar intentos de acceso, escaneos y actividad maliciosa.» y «5. Incorporar procesos de enriquecimiento mediante geolocalización, reputación IP y análisis de artefactos.» Motivos: se priorizó una única cadena de punta a punta, validada en un lote (L0) y cinco ventanas (W1, W2, C1, C2 y C3), en lugar de varios sensores sin validar. La reputación de IP no produce datos verificables con tráfico de direcciones privadas. El análisis de artefactos exige manipular malware, fuera de un laboratorio académico (§1.5). Objetivo general: el 12/10/2026, para ajustarlo a lo que valida la prueba funcional, en coherencia con el título. Redacción original: «Diseñar e implementar una arquitectura de honeypots integrada con la plataforma n8n para automatizar la recolección, procesamiento y generación de inteligencia de amenazas validada mediante prueba funcional controlada.» Con las redacciones reformuladas, los objetivos 1, 2, 3, 5 y 7 se cumplieron y los objetivos 4 y 6 se cumplieron solo en parte; el objetivo general se cumple solo en parte con ambas redacciones (Tabla 6.8, §6.10).

Alcance efectivo del trabajo. Con los objetivos 3 y 5 reformulados, lo implementado es un único honeypot de media interacción (Cowrie), con segmentación en redes Docker internal y con enriquecimiento limitado a la geolocalización del país de origen de la IP, verificada en prueba con direcciones públicas. Dionaea, la segmentación con VLAN y firewall, la reputación de IP, el análisis de artefactos y STIX quedan como diseño objetivo, no implementado, o como trabajo futuro (§1.5); no se computan como incumplimiento de un objetivo. El título describe ese alcance efectivo: un prototipo con un honeypot y una validación funcional en laboratorio.

1.5 Alcances y delimitaciones

La investigación se circunscribe al análisis técnico de eventos registrados por honeypots desplegados en un laboratorio controlado, sin exposición entrante desde Internet (red Docker de laboratorio). La salida a Internet no estuvo restringida hasta C2; en la configuración final, validada en C3, solo n8n y el forwarder tienen salida (§5.6.6). El trabajo se orienta exclusivamente a la observación defensiva y la generación de inteligencia: no persigue identificar a los atacantes, realizar contraataques, interferir con terceros ni acceder sin autorización a sistemas externos. Queda excluida de lo medido la capacidad de respuesta

defensiva de una organización, que requeriría tráfico real y analistas (§4.6). Los resultados dependen del período de observación y de los servicios expuestos; otras soluciones comerciales o arquitecturas alternativas quedan fuera del alcance.

Quedan fuera del alcance cuatro componentes que el diseño objetivo contemplaba, por las razones siguientes. Dionaea: se priorizó una única cadena de punta a punta, validada en un lote (L0) y cinco ventanas (W1, W2, C1, C2 y C3), en lugar de varios sensores sin validar. VLAN con firewall: en un único host con Docker, la segmentación con redes Docker internal da el aislamiento necesario para el laboratorio; la VLAN requiere infraestructura de red fuera del alcance y queda como diseño objetivo (Figura 5.2). Reputación de IP: todo el tráfico del laboratorio proviene de direcciones privadas, sobre las que una consulta de reputación no produce datos verificables. Análisis de artefactos: el simulador no descarga malware, y manipular malware real exige resguardos que exceden un laboratorio académico.

1.6 Hipótesis general e hipótesis operativas

Hipótesis general. Una arquitectura que integre un honeypot de media interacción con n8n y PostgreSQL permite, en un entorno de laboratorio controlado, capturar eventos de ataque, estructurarlos y convertirlos en indicadores de compromiso y en reportes de forma automatizada, y procesar cada evento en menos tiempo que el enfoque manual. Se la contrasta mediante las cuatro hipótesis operativas siguientes (P1 a P4), cuyo criterio de aceptación conjunto se fija en el §4.6.

La comparación de tiempos se operacionaliza en P1 solo para la etapa de ingesta; la extracción y el reporte se cronometraron por ejecución y no por evento (§6.12).

Dado el diseño del estudio —una prueba funcional controlada, con estímulo simulado por el propio equipo, transversal y sin grupo de comparación ni asignación aleatoria (§4.3)—, se formulan hipótesis operativas y no estadísticas. Cada una define un umbral cuantificable derivado de los criterios de validación (§4.9): 1. P1 — Reducción de tiempo: El tiempo de procesamiento por evento mediante la arquitectura automatizada es inferior al 50 % del tiempo estimado para procesamiento manual (criterio §4.9.1). 2. P2 — Estructuración de datos: Al menos el 80 % de los eventos capturados son estructurados en formato JSON válido con campos consistentes (operacionalizado en el §4.9.2 como eventos persistidos con eventid, session y timestamp no nulos). 3. P3 — Generación de IoCs: El pipeline genera indicadores de compromiso (IoCs) automáticamente para al menos el 70 % de las sesiones con actividad maliciosa confirmada (criterio §4.9.3). 4. P4 — Reportes automatizados: El sistema produce reportes de inteligencia sin intervención manual directa para al menos el 90 % de los ataques categorizados como de interés (criterio §4.9.4). A los fines de esta hipótesis, se adopta desde aquí el mismo criterio operacional definido y aplicado en §6.9: un «ataque de interés» es una

sesión en la que el atacante logra autenticarse exitosamente (compromiso confirmado del honeypot), a diferencia de un sondeo que agota sus intentos sin éxito.

1.7 Variables de análisis

Variable independiente: Implementación de arquitectura de honeypots integrada con automatización mediante n8n. Variables dependientes (definidas en el §4.7 y operacionalizadas en la Tabla 4.3): tiempo de procesamiento por evento; porcentaje de eventos correctamente estructurados; cantidad de IoCs generados; tiempo de generación de reportes, y nivel de utilidad operativa de la información.

1.8 Estructura del trabajo

El documento se organiza en nueve capítulos: introducción (I), estado del arte (II), marco teórico (III), metodología (IV), diseño e implementación del prototipo (V), resultados de la validación funcional (VI), discusión (VII), conclusiones (VIII) y recomendaciones (IX). Los diez anexos reúnen los diagramas de secuencia y de flujo (I), los workflows (II), los datos y las tablas de detalle (III), la herramienta de demostración (IV), los antecedentes y las limitaciones técnicas (V), la bitácora de las ventanas (VI), los contrastes de P1 a P4 con sus consultas (VII), la línea de base manual (VIII), el modelo SIM3 con los estándares de IoC (IX) y la trazabilidad de las versiones del repositorio (X).

Capítulo II: Estado del Arte

2.1 Introducción

Este capítulo examina qué se ha hecho y qué herramientas existen en honeypots, tecnologías de engaño y su integración con automatización para inteligencia de amenazas, con el objetivo de posicionar la arquitectura propuesta; el marco teórico (Capítulo III) desarrolla los fundamentos conceptuales.

2.2 Evolución reciente del panorama de amenazas

Durante los últimos años los ataques informáticos ganaron en complejidad (Tounsi y Rais, 2018) y el cibercrimen se profesionalizó en torno a la alianza entre operadores de ransomware y mercados de acceso inicial (Group-IB, 2024). Los reportes sectoriales de 2024 coinciden en la persistencia de ataques automatizados de fuerza bruta, explotación masiva de vulnerabilidades, ransomware, phishing, robo de credenciales y escaneos permanentes de servicios expuestos, con crecimiento de los ataques a aplicaciones web y APIs (Akamai Technologies, 2024; Cisco Talos, 2025; SonicWall, 2024; Shodan, s.f.). Por eso la defensa basada solo en el perímetro resulta insuficiente frente a amenazas distribuidas, veloces y adaptativas.

2.3 Tendencias actuales en defensa cibernética

Frente a este escenario, la ciberseguridad evolucionó hacia modelos integrales y proactivos: Zero Trust (no confiar por defecto en ninguna conexión y verificar cada acceso; Kindervag, 2010), plataformas XDR que integran telemetría multi-fuente, soluciones SOAR —término popularizado por Gartner en 2017 (Neiva et al., 2017)—, la Deception Technology con señuelos y honeypots de alta precisión (Spitzner, 2002; Javadpour et al., 2024) y la inteligencia de amenazas contextualizada con la realidad local de cada organización (Bromiley, 2016). Este trabajo se sitúa en la convergencia entre deception security, threat intelligence y automatización.

2.4 Estado actual del uso de honeypots

Los honeypots evolucionaron desde sus primeras implementaciones académicas (Spitzner, 2002) hasta emplearse en laboratorios universitarios, CSIRT/CERT, SOC, investigación de malware y estudios estadísticos sobre amenazas globales (detección de escaneos, captura de malware, credenciales de botnets, IoCs, entrenamiento defensivo). Los estudios empíricos con Cowrie localizados en Crossref (Baser et al., 2021; Lingenfelter et al., 2020; Năstase et al.,

2024) analizan, agrupan o visualizan lo capturado, según sus títulos y resúmenes, sin un flujo automatizado de indicadores y reportes. Una búsqueda acotada en Crossref (11/10/2026), con los términos «honeypot», «Cowrie», «SSH», «análisis», «Argentina», «Latinoamérica», «Colombia» y «Brasil», no devolvió estudios regionales sobre Cowrie; no se la registró en la Tabla III-9. Gran parte de los despliegues presenta, además, limitaciones comunes —escasa documentación metodológica, falta de automatización analítica, baja integración con procesos de respuesta y dependencia de la revisión manual de logs—, brechas que esta tesis busca superar.

2.5 Evolución de honeypots: de Spitzner a Cowrie y Dionaea

El concepto de honeypot fue sistematizado y popularizado por Spitzner (2002) (definición formal en §3.5.1), y la tecnología evolucionó desde entonces a través de distintas generaciones.

2.5.1 Línea temporal de hitos clave

La Tabla III-8 (Anexo III) resume los hitos principales de esa evolución: el concepto sistematizado por Spitzner (2002) y Honeyd (2002), Nepenthes (2005), Kippo, Dionaea y Glastopf (2009), y Cowrie y Snare (2015). De ellos siguen activos Dionaea, Cowrie y Snare; Honeyd, Nepenthes, Kippo y Glastopf figuran como inactivos, y Cowrie reemplazó a Kippo con soporte de Telnet, registro JSON y captura de descargas.

2.5.2 Generaciones de honeypots

La evolución puede agruparse en tres generaciones: la primera (2000-2005) consolidó los fundamentos académicos con Honeyd (Provos, 2004; Provos y Holz, 2007) y Nepenthes, que sentó las bases de la captura automatizada de malware. La segunda (2005-2015) especializó la emulación por protocolo con Kippo (SSH interactivo), Dionaea (multi-protocolo) y Glastopf (web) (Dionaea Project, s.f.; Cowrie Project, s.f.). La tercera (2015-presente) prioriza la calidad del registro estructurado en JSON y la integración con plataformas externas, con Cowrie como uno de los honeypots SSH/Telnet de media interacción más utilizados, incluido en plataformas como T-Pot (Cowrie Project, s.f.; Deutsche Telekom Security GmbH, 2023).

2.5.3 Clasificación del ecosistema de honeypots

Herramientas especializadas complementan el ecosistema: Conpot (protocolos ICS/SCADA; Nawrocki et al., 2016) y T-Pot (múltiples honeypots en Docker; Deutsche Telekom Security GmbH, 2023). La evolución marca el paso de los honeypots de investigación, orientados a estudiar al atacante, a los honeypots de producción, orientados a proteger a una organización

(Spitzner, 2002), y más recientemente a su uso como fuente de inteligencia de amenazas (Al-Mohannadi et al., 2018; Chin et al., 2025).

2.6 Proyectos relacionados

Protocolo de búsqueda. La selección de proyectos relacionados partió de un protocolo orientador, sintetizado en la Tabla III-9 (Anexo III). Es una revisión narrativa con protocolo, no sistemática: no hay registro cuantitativo contemporáneo ni diagrama PRISMA (§2.8).

Tres proyectos históricos enmarcan el problema. The Honeynet Project es una red de investigación cuyas herramientas de código abierto (Dionaea, Glastopf) no tienen una plataforma de integración unificada. hpfeeds es un protocolo de publicación y suscripción que lleva los eventos de los sensores a un broker central, sin procesarlos. MHN centraliza la gestión de sensores (Cowrie, Dionaea y Glastopf) desde una interfaz web y una API HTTP, sin generar IoCs de forma automática (hpfeeds Project, 2011; Honeynet Project, s.f.; ThreatStream, s.f.). La Tabla 2.1 compara con la propuesta los cuatro antecedentes recientes más cercanos, en cuatro dimensiones: automatización, enriquecimiento, IoCs y replicabilidad.

Tabla 2.1: Comparación de la propuesta con los antecedentes recientes más cercanos.

Trabajo	Automatización	Enriquecimiento	IoCs	Replicabilidad
Chin et al. (2025): Cowrie en una VM de Azure, expuesto a Internet	Azure Monitor, Sentinel y Logic Apps; un flujo cerrado que actualiza el firewall de Azure (tiempo medio de bloqueo de 0,86 s)	Etiquetado con MITRE ATT&CK y un paso «Parse & Enrich IP» en la Logic App; no detalla las fuentes de enriquecimiento	Extrae las IP de origen para bloquearlas; no informa un catálogo de IoCs ni un formato (STIX, MISP)	No informada; depende de servicios de Azure. Datos de 7 días (12.224 intentos SSH)
Parmaksız y Turhan (2026): Wazuh, n8n, OpenVAS y GLPI; sin honeypot	n8n recibe alertas de Wazuh por webhook, dispara OpenVAS y crea tickets en GLPI; latencia media de orquestación de 502 ms	Añade a cada ticket las vulnerabilidades detectadas por OpenVAS	No informados (el producto son tickets de incidente)	Arquitectura de 21 microservicios en Docker descrita; código y datos: no informado
Casaclang et al. (2026): n8n autoalojado; sin honeypot	Workflows de n8n que generan escaneos Nmap, los analizan con modelos de lenguaje (ChatGPT, Gemini, Ollama) y redactan informes	Consulta datos públicos de vulnerabilidades (CVE/CVSS) a partir del resultado del escaneo	No informados	Describe la instalación de n8n y los workflows; publicación del código propio: no informado
Bartwal et al. (2022): motor SOAR que despliega honeypots dockerizados (entre ellos Cowrie)	Motor SOAR propio con clasificadores de aprendizaje automático; despliega honeypots según el comportamiento del atacante (40 veces en 4 días)	No informado	Sin formato de IoCs informado; recolectó 3 muestras de malware	No informada; prueba de 4 días en una red interna

Trabajo	Automatización	Enriquecimiento	IoCs	Replicabilidad
Esta propuesta: Cowrie, n8n y PostgreSQL	Ingesta por webhook (event-ingest), extracción cada 15 minutos (ioc-extractor) y reportes programados (report-generator), observados en modo trigger; el cron diario de producción (08:00) se observó dos veces: el 06/10/2026 (C2) y el 09/10/2026 (C3)	Solo el país de origen, verificado en prueba y no ejercido con tráfico real; reputación de IP y análisis de artefactos no implementados	21 IoCs (1 IP y 20 credenciales) en W1 y en W2, 26 en C1, 32 en C2 y 32 en C3, en formato propio (tabla iocs de PostgreSQL); STIX no implementado	Docker Compose con Cowrie como único honeypot; configuración versionada en Git. Diseñada, no demostrada: en un segundo entorno se perdieron sin aviso unas 29 de 39 sesiones (Anexo V, §V.2); réplica por un tercero: una ejecución exitosa, con asistencia de IA para los comandos (Tabla 4.1)

Nota. Elaboración propia a partir del texto de cada trabajo (Chin et al., 2025; Parmaksız y Turhan, 2026; Casaclang et al., 2026; Bartwal et al., 2022), revisado en octubre de 2026. «No informado» indica que el texto no lo menciona, no que el trabajo carezca de ello. Fila de la propuesta: prototipo validado (Tablas 6.6 y 6.7, §6.8.3).

En síntesis, los tres proyectos cubren facetas complementarias —investigación (Honeynet Project), transporte (HPFeeds) y gestión centralizada (MHN)—, pero ninguno integra un pipeline que transforme automáticamente eventos crudos en inteligencia estructurada. Entre los antecedentes recientes de la Tabla 2.1, ninguno combina un honeypot de media interacción con una plataforma de bajo código para producir IoCs estructurados y reportes: Chin et al. usan Cowrie con servicios de Azure para bloquear IP, Bartwal et al. despliegan honeypots con un motor SOAR propio, y Parmaksız y Turhan y Casaclang et al. usan n8n sin honeypot. Al-Mohannadi et al. (2018) combinan datos de honeypot con Elasticsearch, sin automatización low-code ni generación estructurada de IoCs.

Alternativas abiertas. El competidor natural de n8n en este nicho no es solo el SOAR empresarial, sino también las plataformas de código abierto: Shuffle (SOAR con editor visual de flujos; Shuffle, s.f.), TheHive y Cortex (gestión de casos y análisis de observables; TheHive Project, s.f.), StackStorm (automatización dirigida por eventos; StackStorm, s.f.) y T-Pot (más de veinte honeypots, entre ellos Cowrie y Dionaea, con visualización en la pila Elastic; Deutsche Telekom Security GmbH, 2023). La Tabla III-13 (Anexo III) las resume. Esta tesis no las desplegó ni midió: la elección de n8n responde a las razones del §3.9.6 y no a una superioridad demostrada, y la comparación se propone como trabajo futuro. La brecha que el trabajo aborda no es la ausencia de herramientas de código abierto, sino la falta de una integración documentada y reproducible entre un honeypot de media interacción y una plataforma de automatización de bajo código.

2.7 Posicionamiento de la arquitectura propuesta

2.7.1 Brecha identificada en el estado del arte

El análisis de los proyectos clásicos (§2.6) revela una brecha transversal: la desconexión entre la captura de eventos honeypot y la generación efectiva de inteligencia de amenazas. Las

plataformas comparadas se enfocan en recolección y visualización (MHN), transporte de eventos (HPFeeds) o investigación académica (Honeynet Project), pero ninguna ofrece un pipeline completo que transforme eventos crudos en IoCs estructurados, enriquecidos y listos para consumo defensivo. Para verificar si esa brecha persiste en la producción académica reciente, se realizó en septiembre de 2026 una búsqueda acotada, no sistemática, sobre Google Scholar y arXiv ("honeypot automation SOAR threat intelligence pipeline" y variantes), restringida a trabajos de 2023 en adelante. Esta búsqueda complementa la del §2.6 y es la única en la que Google Scholar se usó como fuente de búsqueda. Los trabajos identificados persiguen el mismo objetivo general —cerrar el ciclo entre captura de honeypot y respuesta automatizada— con arquitecturas distintas a la de esta tesis. Chin et al. (2025), el más directamente comparable, figura en la Tabla 2.1; Amal y Venkadesh (2023) proponen H-DOCTOR, un ajuste dinámico de firewall basado en telemetría de honeypot.

Esta búsqueda matiza la afirmación de originalidad: la integración de honeypots con automatización para generar inteligencia de amenazas no es un problema inexplorado, y existe al menos un antecedente directo (Chin et al., 2025) y otro relacionado (d'Ambrosio et al., 2025, una defensa por movimiento de objetivo en redes SDN con honeypots, sin generar inteligencia), ambos de 2025. Un tercer antecedente es el de Bartwal et al. (2022), que proponen un motor SOAR que despliega honeypots personalizados de forma dinámica y reportan, en cuatro días de prueba, 7.823 ataques identificados y una interacción media de 3.148 s frente a 102 s con honeypots estáticos; su objetivo es el despliegue adaptativo de señuelos, no la generación de indicadores estructurados. Sanjeev et al. (2020) presentan un enfoque automatizado de generación de inteligencia a partir de datos de honeypot, integrado con formatos estándar de intercambio. En 2026, Parmaksız y Turhan proponen un marco SOAR de código abierto y low-code que combina Wazuh, n8n, OpenVAS y GLPI sobre Docker, y Casaclang et al. evalúan n8n autoalojado como alternativa de bajo costo para tareas de seguridad en organizaciones pequeñas y en docencia. Según sus textos, ninguno combina un honeypot de media interacción con n8n ni mide el resultado con umbrales fijados de antemano; por eso el aporte de esta tesis es la integración documentada y reproducible de ambos. Su diferencia específica es el uso de una plataforma low-code autoalojada (n8n), en lugar de servicios cloud propietarios, orientada a generar indicadores estructurados que el prototipo almacena en un formato propio (tabla iocs de PostgreSQL) y no al bloqueo activo inmediato; la exportación a STIX 2.1 está diseñada, pero no implementada. Esa brecha más acotada es la que aborda la tesis.

2.7.2 Comparación con proyectos existentes

La comparación con los antecedentes recientes se sintetiza en la Tabla 2.1 (§2.6). Frente a ellos, a los proyectos históricos y a las soluciones comerciales ligeras, la propuesta se diferencia en cinco aspectos:

1. Automatización de extremo a extremo (diseño): n8n transforma eventos en indicadores y reportes estructurados con la intervención manual limitada a la puesta en marcha; en el prototipo, la extracción y el reporte programados operaron en modo trigger, pero no alcanzaron los umbrales de P3 y P4 (§6.9). MHN, en cambio, solo centraliza gestión. 2. Enriquecimiento contextual: el diseño prevé geolocalización, reputación IP y análisis de artefactos en el flujo automatizado; en el prototipo validado solo se implementó la geolocalización por país (§6.8.3). 3. Arquitectura replicable: basada en Docker, despliega en infraestructura mínima, con la documentación y los scripts publicados en el repositorio. 4. Bajo costo: herramientas de código abierto o de uso gratuito, adecuadas para PyMEs e instituciones educativas. 5. Pipeline completo: abarca desde la captura hasta la generación de reportes; la diseminación externa no está implementada.

2.7.3 Limitaciones que persisten

A pesar de las ventajas, la arquitectura presenta limitaciones explícitas: no alcanza la integración de una plataforma SOAR empresarial; no se comparó con alternativas abiertas como Shuffle, StackStorm o T-Pot (Tabla III-13); la automatización complementa y no reemplaza el análisis humano; la cobertura se limita a los protocolos emulados por Cowrie (Dionaea es diseño objetivo); y la escalabilidad a grandes volúmenes requeriría optimizaciones de infraestructura. Esta revisión atiende al objetivo específico 1 y da la base contextual para los objetivos 2 y 3.

2.8 Descripción de la búsqueda bibliográfica

La búsqueda de §2.6 se hizo en IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink y arXiv, y se completó con informes de la industria, documentación técnica de software libre, legislación y estándares, según las cadenas y los criterios de la Tabla III-9; la cadena base se adaptó a cada base y tema. No se llevó un registro contemporáneo por base de datos, por lo que no se informan conteos por etapa ni diagrama PRISMA; la lista de Referencias es la única constancia verificable del resultado.

La lista final contiene 72 referencias: 31 artículos, capítulos y preprints académicos; 6 libros; 11 informes de industria y de inteligencia de amenazas; 12 fuentes de documentación técnica y software; 10 estándares y guías, y 2 normas legales. Constituyen excepciones declaradas a los criterios del §2.6, incorporadas por conocimiento del equipo o por rastreo de citas: (a) por

período, los trabajos académicos, libros y fuentes técnicas anteriores a 2018 que fundamentan técnicas o conceptos citados (por ejemplo, Provos, 2004; Spitzner, 2002; Kindervag, 2010; Cichonski et al., 2012; Wagner et al., 2016); (b) por ausencia de revisión de pares, los preprints de Nawrocki et al. (2016) y Chin et al. (2025), este último por ser el antecedente más cercano (§2.7.1); y (c) por tipo de fuente, los blogs de Bianco (2013) y del Honeynet Project (2009). Otras referencias (Wilson, 1927; los RFC; la Ley 25.326 y su decreto; Stikvoort, 2019) provienen del rastreo de citas; Scopus y Google Scholar sirvieron para localizar y verificar trabajos.

Capítulo III: Marco Teórico

3.1 Introducción

El capítulo desarrolla los fundamentos teóricos de la investigación en cuatro ejes: el paradigma de la ciberseguridad defensiva, los honeypots como mecanismos de observación, la inteligencia de amenazas como objetivo del sistema y la automatización aplicada a operaciones de seguridad. Responde al objetivo específico 1 y es base de los objetivos 4 y 5.

3.2 Ciberseguridad y cambio de paradigma defensivo

La ciberseguridad contemporánea se caracteriza por sistemas conectados a redes públicas, expuestos permanentemente a intentos de acceso no autorizado: los honeypots accesibles desde Internet reciben ataques a los pocos minutos de activarse (Nawrocki et al., 2016). El enfoque basado solo en prevención resulta limitado ante ataques automatizados y veloces, y surge un cambio de paradigma hacia el monitoreo continuo y la detección temprana (Bejtlich, 2004) y, más recientemente, la generación de inteligencia de amenazas (Tounsi y Rais, 2018): la defensa ya no se basa solo en bloquear accesos, sino también en observar, aprender y adaptarse.

3.3 Seguridad de la información y ciberseguridad

La seguridad de la información comprende las políticas, controles y prácticas orientadas a proteger activos informacionales frente a amenazas internas y externas; con la expansión del entorno digital evolucionó hacia la ciberseguridad, que incorpora la protección de redes, servicios en línea, identidades digitales, dispositivos conectados e infraestructuras críticas. La Tabla 3.1 sintetiza los modelos tradicionales de detección.

Tabla 3.1: Modelos tradicionales de detección.

Eje	Concepto	Definición o función
Detección tradicional	Firewalls	Controlan tráfico entrante y saliente según reglas definidas.
Detección tradicional	IDS/IPS	Detectan o bloquean patrones maliciosos conocidos.
Detección tradicional	Antivirus / EDR	Analizan procesos, archivos y comportamiento de endpoints.
Detección tradicional	SIEM	Centralizan logs y correlacionan eventos.

Nota. Elaboración propia.

Aunque indispensables, los modelos tradicionales de detección enfrentan limitaciones: dependencia de firmas conocidas, alto volumen de alertas, falsos positivos, baja visibilidad sobre la intención del atacante y escasa capacidad frente a amenazas nuevas. Por ello se vuelve relevante complementar la defensa con mecanismos de observación activa: estos modelos

registran que ocurrió un evento, pero no reconstruyen las credenciales, los comandos ni el comportamiento del atacante, que es la inteligencia local que el problema (§1.2) requiere.

3.4 Teoría del engaño en ciberseguridad

El engaño constituye una práctica histórica en ámbitos militares, estratégicos y de inteligencia; en el plano digital, consiste en inducir al atacante a interactuar con activos controlados por el defensor, y puede reforzar la seguridad de los sistemas actuales si se lo planifica e integra en las defensas (Almeshekah y Spafford, 2016). Sus principios fundamentales (síntesis propia del equipo) son: 1. Desviar atención de activos reales. 2. Aumentar costo operativo del atacante. 3. Obtener información del adversario. 4. Detectar intrusiones tempranas. 5. Reducir incertidumbre defensiva. Los honeypots son una de las formas más consolidadas de deception technology, sistematizada como estrategia defensiva (Javadpour et al., 2024; Morić et al., 2025).

3.5 Honeypots como mecanismos de observación

3.5.1 Definición conceptual

Un honeypot es un sistema diseñado deliberadamente para ser explorado o atacado con el fin de registrar actividad maliciosa (Spitzner, 2002). A diferencia de los sistemas productivos, su propósito no es prestar servicios legítimos, sino actuar como: sensor de actividad hostil, herramienta de análisis, y fuente de evidencia técnica.

3.5.2 Ventajas y limitaciones

Entre las ventajas de los honeypots se destaca su alta relación señal/ruido, ya que toda interacción registrada puede considerarse potencialmente maliciosa (Spitzner, 2002), junto con la captura directa de actividad maliciosa y la facilidad de análisis; sus limitaciones son la cobertura limitada, la posibilidad de detección por atacantes y la dependencia del entorno. Las revisiones de la literatura (Ikuomenisan y Morgan, 2022; Ilg et al., 2023; Franco et al., 2021) las consideran inherentes a la tecnología; por ejemplo, un honeypot de baja interacción emula uno o pocos servicios y no un sistema operativo completo (Sokol et al., 2015).

3.5.3 Taxonomía de honeypots y criterios de selección

La clasificación de honeypots es un área de estudio en sí misma (Spitzner, 2002); los criterios más relevantes son el nivel de interacción, el propósito, el protocolo emulado y el tipo de despliegue.

3.5.3.1 Taxonomía multidimensional

La taxonomía multidimensional de la Tabla 3.2 clasifica los honeypots por nivel de interacción (baja, media y alta, con riesgo bajo, medio y alto; Cowrie es de media interacción), por propósito (investigación, producción y entrenamiento), por protocolo emulado (SSH/Telnet, multiprotocolo, web, bases de datos, ICS/SCADA e IoT) y por tipo de despliegue, e indica para cada categoría el riesgo y ejemplos.

Tabla 3.2: Taxonomía multidimensional de honeypots.

Dimensión	Categoría	Descripción	Ejemplos	Riesgo
Nivel de interacción	Baja	Emulan servicios básicos con comportamiento predefinido, sin ejecución real de comandos. Bajo riesgo, fácil despliegue y mínimo mantenimiento; fácil detección por atacantes experimentados e información limitada. Ideales para censado de escaneos, IoCs básicos y monitoreo de tendencias.	Honeyd, Heralding	Bajo
Nivel de interacción	Media	Entornos simulados más realistas (sistemas de archivos falsos, respuestas dinámicas) que permiten observar comandos y comportamiento post-autenticación sin exponer un sistema real. Mayor riqueza de datos; riesgo moderado (bugs de emulación) y configuración más compleja. Ideales para TTPs básicos, captura de credenciales y campañas automatizadas.	Kippo, Cowrie, Glastopf	Medio
Nivel de interacción	Alta	Sistemas operativos reales o completos, sin restricciones de interacción: captura total del	Honeynet real	Alto
		comportamiento atacante, con alto riesgo, alta carga operativa y monitoreo constante. Ideales para forense avanzada, APTs y malware complejo.		

Dimensión	Categoría	Descripción	Ejemplos	Riesgo
Propósito	Investigación	Captura de datos sobre amenazas y comportamiento atacante	Cowrie (modo investigación)	Variable
Propósito	Producción	Detección temprana en entornos productivos reales	Honeyd, honeypots personalizados	Bajo
Propósito	Entrenamiento	Formación de equipos de seguridad en entornos controlados	Honeynet, laboratorios	Controlado
Protocolo	SSH/Telnet	Emulación de servicios de acceso remoto	Cowrie, Kippo, Heralding	Medio
Protocolo	Multi-protocolo	Emulación de múltiples servicios simultáneos	Honeyd	Bajo-Medio
Protocolo	Web	Emulación de servidores HTTP y aplicaciones web	Glastopf, Snare, Wordpot	Medio-Alto
Protocolo	Base de datos	Emulación de servicios de bases de datos	MongoDB Honeypot, MySQLPot	Bajo
Protocolo	ICS/SCADA	Emulación de protocolos industriales	Conpot, SCADA Honeynet	Bajo
Protocolo	IoT	Emulación de dispositivos inteligentes	IoT Honeypot, HoneyThing	Bajo
Despliegue	Físico	Hardware dedicado con sistema operativo real	Servidor físico con Honeynet	Alto
Despliegue	Virtual	Máquina virtual sobre hipervisor	VM con Cowrie	Medio
Despliegue	Contenedor	Aislamiento a nivel de sistema operativo	Docker con T-Pot, contenedores individuales	Bajo
Despliegue	Híbrido	Combinación de los anteriores	T-Pot, MHN	Medio

Nota. Elaboración propia. La columna «Riesgo» y la asignación de ejemplos a cada categoría son valoración del equipo, a partir de la documentación de cada herramienta y de Spitzner (2002); no provienen de una medición.

3.5.3.2 Criterios de selección de honeypots según el escenario

La selección de un honeypot depende del objetivo del despliegue, los recursos disponibles y el nivel de riesgo aceptable: el riesgo operativo y la riqueza de datos de cada nivel de interacción (Nawrocki et al., 2016), la integración con herramientas existentes, los recursos de mantenimiento y el formato de salida. La arquitectura propuesta se posiciona en la investigación académica con proyección operativa para PyMEs, con un honeypot de media interacción, que da más información que uno de baja sin el riesgo operativo de uno de alta.

3.5.3.3 Criterios de selección aplicados al proyecto

En el prototipo validado se desplegó únicamente Cowrie, honeypot de media interacción (SSH y Telnet) con registro estructurado en JSON, mecanismos de integración externa (webhooks,

HPFeeds y syslog) y una comunidad activa; el reenvío a n8n lo hace un servicio propio (forwarder, §5.4). Dionaea (baja interacción, multi-protocolo y captura de malware) forma parte del diseño objetivo y no se desplegó (§5.4).

3.6 Inteligencia de amenazas (Threat Intelligence)

3.6.1 Definición y fundamentos

La inteligencia de amenazas (Threat Intelligence, TI) puede definirse como el proceso de obtener, de varias fuentes, conocimiento sobre las amenazas que sirva para decidir cómo responder a ellas (Bromiley, 2016); aquí se la describe, como síntesis propia, en cuatro actividades: recolección, análisis, contextualización y aplicación práctica. A diferencia de los datos crudos (logs, alertas sin procesar) o la información (datos organizados pero no interpretados), la inteligencia agrega contexto, relevancia y aplicabilidad operativa (Tounsi y Rais, 2018), y permite responder no solo a qué ocurrió, sino a por qué, quién lo hizo y cómo prevenirlo.

3.6.2 Los cuatro niveles de inteligencia de amenazas

Una taxonomía difundida (Chismon y Ruks, 2015, citado en Tounsi, 2019) clasifica la inteligencia de amenazas en cuatro niveles —estratégico, operacional, táctico y técnico— (Tabla 3.3), con destinatarios, contenidos y horizontes temporales distintos (Tounsi, 2019; Tounsi y Rais, 2018); los ejemplos de la tabla son didácticos y no resultados del proyecto.

Tabla 3.3: Niveles de inteligencia de amenazas.

Nivel	Destinatario	Contenido	Horizonte	Ejemplo ilustrativo del nivel
Estratégico	Dirección, tomadores de decisión	Información de alto nivel sobre riesgos y tendencias	Largo plazo	«El 78 % de los ataques registrados provienen de infraestructura automatizada; se recomienda priorizar controles de fuerza bruta en SSH.»
Operacional	Responsables de seguridad, jefes de respuesta a incidentes	Detalles de ataques específicos inminentes contra la organización	Corto plazo	«Se anticipa una campaña de fuerza bruta SSH dirigida al sector durante la próxima semana.»

Nivel	Destinatario	Contenido	Horizonte	Ejemplo ilustrativo del nivel
Táctico	Equipos de respuesta a incidentes y SOC	Tácticas, técnicas y procedimientos (TTP) de los atacantes	Largo plazo	«Tras autenticarse, los atacantes ejecutan reconocimiento (uname, cat /etc/passwd) antes de descargar cargas con wget.»
Técnico	Recursos técnicos (firewalls, filtros de correo, SIEM)	Indicadores de compromiso	Inmediato	«Feed STIX con 89 indicadores (IPs, hashes SHA-256 de malware, user agents) listos para consumo por firewall y sistemas de detección.»

Nota. Elaboración propia a partir de Tounsi (2019, Tabla 1.2). Los ejemplos son ilustrativos y no resultados del proyecto, que observó una sola IP por validación y generó cuatro IoCs en el lote del 25/09/2026, 21 en las ventanas automáticas del 28/09 y del 30/09/2026, 26 en la del 02/10/2026, 32 en la del 05–06/10/2026 y 32 en la del 08–09/10/2026.

Posicionamiento del proyecto: La arquitectura propuesta se sitúa principalmente en los niveles táctico y técnico, generando IoCs estructurados y tipados. De forma complementaria, los reportes agregados aportan una vista operacional para analistas; no se implementaron dashboards. El nivel estratégico se aborda de forma indirecta a través de las recomendaciones y conclusiones del presente trabajo.

3.6.3 Fuentes de inteligencia

En la arquitectura propuesta, los honeypots constituyen la fuente interna principal de inteligencia. En el diseño, esta fuente se complementa con enriquecimiento externo durante el procesamiento automatizado en n8n; el prototipo implementa solo la geolocalización por país, y la reputación IP queda prevista y no implementada (§6.8.3).

3.6.4 Calidad y valor de la inteligencia de amenazas

Para valorar los indicadores que produce el prototipo se usan dos criterios. El primero son los cuatro factores de utilidad operativa de un IoC que los autores sintetizan a partir de Bromiley (2016): actualidad (los IoCs pierden valor rápidamente), contexto (un IoC sin tipo de amenaza, momento de captura o fuente tiene utilidad limitada), precisión (los falsos positivos reducen la confianza y generan fatiga en los operadores) e integración (solo es útil si lo consumen rápidamente firewalls, IDS o listas de bloqueo). El segundo es la Pirámide del Dolor (Bianco, 2013), que ordena los indicadores según el costo que implica para el adversario cambiarlos: los valores hash, las direcciones IP y los nombres de dominio están en la base y son baratos de cambiar, mientras que los artefactos, las herramientas y, en la cima, las tácticas, técnicas y procedimientos son los más costosos. Los factores sirven para evaluar cada indicador y la pirámide, para ordenar los tipos de indicador; el §7.7 los aplica a los resultados.

3.7 Ciclo de vida de la inteligencia de amenazas (modelo PCPAD)

El ciclo de vida de la inteligencia de amenazas se organiza en cinco fases —Plan, Collect, Process, Analyze y Disseminate (PCPAD; adaptación propia del ciclo de inteligencia, que reduce a cinco las seis etapas habituales: planificación, recolección, procesamiento, análisis, diseminación y retroalimentación)—. La retroalimentación no se incluye porque el prototipo no tiene un consumidor de la inteligencia que devuelva requerimientos al ciclo; se la prevé como trabajo futuro, ligado a la evaluación del reporte con analistas (Tabla 8.2). La Tabla 3.4 sintetiza, para cada fase, su propósito, las actividades principales, los componentes responsables en la arquitectura y el formato y la frecuencia de sus datos.

Tabla 3.4: Ciclo PCPAD: propósito, actividades y mapeo a los componentes implementados.

Fase	Propósito (pregunta guía)	Actividades principales	Componentes responsables	Formato / frecuencia
Plan	Establecer los requisitos de inteligencia, los objetivos de captura y el alcance del monitoreo (¿qué información se necesita y para qué?).	Identificación de activos a proteger y vectores de amenaza relevantes; definición de IoCs de interés; establecimiento de fuentes internas y externas; frecuencia y formato de los entregables.	Configuración de Cowrie (SSH/Telnet); consulta de país en el nodo Code de event-ingest (implementada); workflow de reportes.	Documentación; al inicio + revisiones periódicas.
Collect	Obtener datos de las fuentes definidas (¿qué datos se están observando en el entorno?).	Captura de eventos de red y autenticación; recolección de muestras de malware; obtención de logs estructurados; almacenamiento temporal de datos crudos.	Cowrie (autenticación SSH/ Telnet); volumen Docker para el log crudo.	JSON; tiempo real (continuo).
Process	Transformar los datos crudos en información estructurada, normalizada y enriquecida (¿qué significan los datos recolectados?).	Parseo a JSON; normalización de campos entre fuentes; enriquecimiento contextual (geolocalización por país, implementada; reputación IP y WHOIS, previstos); validación y deduplicación; almacenamiento relacional.	n8n: nodo Webhook, nodo Code (normalización y consulta a ip-api.com) y nodo PostgreSQL.	JSON estructurado + enriquecido; por evento (webhook).
Analyze	Interpretar los datos procesados para identificar patrones, TTPs y campañas activas y generar IoCs (¿qué implican los eventos para la seguridad del entorno?).	Identificación de patrones recurrentes; correlación entre fuentes; clasificación por severidad y tipo; generación de IoCs; conclusiones analíticas.	n8n: nodo Code de ioc-extractor (extracción de IoCs) y consultas SQL en PostgreSQL.	Consultas SQL + reportes; por evento + batch diario.
Disseminate	Distribuir la inteligencia a sus consumidores en el formato y la frecuencia adecuados (¿quién necesita saber esto y cómo debe recibirlo?).	Reportes periódicos (diarios, semanales); alertas en tiempo real sobre eventos críticos; feeds de IoCs estandarizados; dashboards.	n8n: disparador programado de report-generator (reporte JSON guardado en PostgreSQL); alertas por Telegram desde el forwarder.	JSON; diario + alertas en tiempo real.

Nota. Elaboración propia. Solo se listan los componentes implementados en el prototipo (§5.6); Dionaea, la reputación de IP, la generación de PDF y los feeds STIX no se implementaron.

Las actividades de cada fase son las del modelo conceptual; el prototipo cubre la recolección, el procesamiento y el reporte, con un enriquecimiento limitado a la geolocalización por país.

El Capítulo VI presenta los resultados de las etapas implementadas (recolección, procesamiento y reporte).

3.8 Análisis forense digital

El análisis forense digital consiste en identificar, preservar, analizar y presentar evidencia contenida en sistemas informáticos, con integridad, trazabilidad y reproducibilidad (Bejtlich, 2004). En el prototipo, la base es el log de Cowrie, la tabla events y el historial de ejecuciones de n8n, con los que se reconstruye cada sesión y se concilian las cifras (§4.14); varios conjuntos de evidencia publicados (docs/evidencia/b1c/, b2/ y b3/, entre otros) llevan su SHA-256 en un archivo SHA256SUMS.txt. No se aplicó una cadena de custodia formal ni se analizaron discos o memoria: el uso es de trazabilidad de los resultados, no pericial.

3.9 Automatización en seguridad

3.9.1 Necesidad de automatización

El volumen de eventos generados en entornos de seguridad dificulta un análisis completamente manual; en la encuesta de SANS de 2015, la mayoría de los comentarios de los encuestados se centró en la automatización, en una mejor inteligencia en tiempo real y en una mejor verificación y precisión (Shackleford, 2015). Una revisión sistemática de los centros de operaciones de seguridad (SOC) recopila el estado del arte y sus desafíos abiertos (Vielberth et al., 2020). La automatización procesa grandes volúmenes de datos con menor intervención manual, y se espera que reduzca los tiempos de respuesta y estandarice los procedimientos; esa expectativa es la que contrasta esta tesis con la hipótesis P1. En la encuesta SANS 2023 a equipos SOC, la falta de contexto fue la barrera más citada (16 %), seguida de la falta de personal calificado, de visibilidad en toda la organización y de automatización y orquestación (Crowley et al., 2023).

3.9.2 SOAR (Security Orchestration, Automation and Response)

Las plataformas SOAR constituyen la aproximación tradicional a la automatización en ciberseguridad empresarial. El término fue popularizado por Gartner en 2017 (Neiva et al., 2017) como evolución de los playbooks de respuesta a incidentes, e integra orquestación (coordinación entre firewalls, SIEM, EDR y ticketing), automatización (tareas repetitivas según reglas predefinidas) y respuesta (contramedidas según la severidad y el tipo de incidente). Plataformas como Splunk SOAR, IBM Resilient, Palo Alto Cortex XSOAR y Siemplify ofrecen integraciones profundas con ecosistemas empresariales, pero suelen implicar alto costo de licenciamiento, complejidad de implementación y personal especializado. En la literatura, la orquestación de seguridad se define por tres funciones —

unificación, orquestación y automatización— (Islam et al., 2019); la combinación de SOAR con honeypots se ha explorado para desplegar señuelos de forma dinámica (Bartwal et al., 2022; §2.7.1), y Kinyua y Awuah (2021) revisan la investigación y las prácticas de la industria sobre SOAR.

3.9.3 Automatización low-code como alternativa

Las plataformas low-code, concebidas para flujos de trabajo empresarial, muestran utilidad creciente en seguridad por su flexibilidad, bajo costo y accesibilidad (n8n GmbH, s.f.). n8n es una plataforma de código accesible (fair-code, Sustainable Use License) basada en flujos visuales con más de 400 nodos de integración: recibe eventos por webhooks, transforma datos con nodos de código, consulta APIs externas, envía alertas, genera reportes y se integra con bases de datos, ejecutando cada operación en un nodo independiente, lo que facilita la depuración y el mantenimiento (n8n GmbH, s.f.). La Tabla III-11 (Anexo III) compara ambas plataformas con el SOAR empresarial.

3.9.4 Tabla comparativa ampliada

La comparación ampliada de la Tabla III-11 (Anexo III) contrasta el SOAR empresarial, n8n y Node-RED en costo, complejidad, flexibilidad, escalabilidad, curva de aprendizaje, enfoque, seguridad nativa, comunidad y casos de uso. El SOAR empresarial es el de mayor costo y complejidad (licencias anuales, infraestructura dedicada); n8n y Node-RED son gratuitos y de curva de aprendizaje baja, y n8n se orienta a la automatización de procesos (ingesta, parseo, enriquecimiento y reportes) y Node-RED, a los flujos de datos y eventos. Las valoraciones cualitativas son del equipo, a partir de la documentación general de cada plataforma, y no se verificaron una por una.

3.9.5 Automatización y limitaciones

Si bien la automatización mejora la eficiencia, no reemplaza el análisis humano, sino que lo complementa: la interpretación de patrones complejos, la validación de hallazgos y las decisiones estratégicas siguen requiriendo juicio especializado.

3.9.6 Justificación de la selección de n8n

Para la arquitectura propuesta, n8n se seleccionó sobre las alternativas analizadas por las siguientes razones (n8n GmbH, s.f.): 1. Arquitectura de nodos: cada nodo ejecuta una operación específica, lo que facilita el diseño modular de flujos para cada fase del ciclo de vida (ingesta, parseo, enriquecimiento, almacenamiento, diseminación). 2. Soporte nativo para webhooks: permite recibir eventos de honeypots en tiempo real sin infraestructura adicional de

mensajería. 3. Cobertura de integraciones: los nodos de PostgreSQL, Code y HTTP Request cubren las necesidades del proyecto. 4. Fair-code, de código accesible y autoalojable: se alinea con el bajo costo y la replicabilidad. 5. Comunidad activa y documentación.

3.10 Modelos de madurez de gestión de incidentes (SIM3)

SIM3 (Open CSIRT Foundation, s.f.) mide la madurez de un equipo de respuesta a incidentes, no la de un prototipo de software, y aquí se usa solo como marco de autoevaluación orientativa. Su detalle y el resultado (nivel 2, explícito e interno, en captura, procesamiento y diseminación, y sin integración con el ecosistema externo de compartición) están en el Anexo IX.

3.11 Indicadores de compromiso (IoCs)

3.11.1 Definición y utilidad operativa

Los IoCs (Indicators of Compromise) son evidencias observables asociadas a actividad maliciosa que permiten identificar intrusiones, bloquear amenazas y generar inteligencia accionable (Tounsi y Rais, 2018); su vinculación con técnicas adversarias puede expresarse mediante el marco MITRE ATT&CK (MITRE Corporation, 2024). El prototipo genera cuatro tipos de IoC: direcciones IP de origen (solo de eventos de conexión), credenciales usadas, comandos ejecutados y hashes SHA-256 presentes en los mensajes de Cowrie (§6.8.2); los puertos destino, los user-agents y los hashes de malware y protocolos como SMB (Dionaea, no desplegada) no están implementados en el extractor. Los factores de utilidad operativa de un IoC (actualidad, contexto, precisión e integración) se tratan en el §3.6.4. El prototipo implementa el procesamiento en tiempo real y la deduplicación; el enriquecimiento se limita al país de la IP de origen, el filtrado por confianza no está implementado (todos los IoCs quedan en nivel BAJO) y la exportación en formatos estandarizados como STIX 2.1 queda como propuesta (apartado 3.11.3).

3.11.2 Estándares de representación, transporte y enriquecimiento

La interoperabilidad entre sistemas de inteligencia de amenazas (CTI) requiere formatos estandarizados que permitan representar, transportar y consumir indicadores de forma consistente (Kampanakis, 2014). STIX 2.1 y TAXII 2.1, de OASIS, definen la representación de la amenaza como objetos JSON y su transporte sobre HTTPS; OpenIOC es un formato XML de Mandiant para indicadores, usado sobre todo en el ámbito forense; y las taxonomías MISP aportan etiquetas de máquina para clasificar eventos e indicadores. Su comparación por criterios está en la Tabla IX-2 (Anexo IX), y el prototipo no implementa ninguno de ellos.

3.11.3 Discusión y recomendación

Los estándares analizados operan en niveles distintos del ecosistema de inteligencia de amenazas (CTI): STIX define la representación semántica, TAXII el transporte, OpenIOC un formato heredado para indicadores simples y las taxonomías MISP el vocabulario de enriquecimiento (Anexo IX, Tabla IX-2). Para la arquitectura se recomienda STIX 2.1 como estándar primario de representación, por su completitud semántica, su interoperabilidad con el ecosistema de consumo (MISP, SIEM, TIP y MITRE ATT&CK), su evolución hacia TAXII para la diseminación externa y el soporte activo de la comunidad OASIS, con las taxonomías MISP como vocabulario complementario de enriquecimiento. Esta combinación es un diseño objetivo: el prototipo validado no implementa STIX, los IoCs se almacenan en un formato propio, en la tabla iocs de PostgreSQL, y la exportación a STIX/TAXII queda como trabajo futuro (§9.4).

Lo que el prototipo implementa es un modelo de datos propio: la tabla iocs, con tipo, valor y un único event_id por indicador, y la tabla ioc_sessions, con las observaciones por sesión (Figura III-1, Anexo III). Ese diseño, y no un estándar, es lo que contrastan P3 y P4, y de él proviene el conflicto entre catálogo deduplicado y cobertura por sesión (§7.2.3).

3.12 Integración conceptual del modelo propuesto

El modelo propuesto no resulta de una selección arbitraria de herramientas, sino de la aplicación consistente de los principios desarrollados en este capítulo a cada decisión arquitectónica. La Tabla 3.5 explicita esa correspondencia.

Tabla 3.5: Correspondencia entre principios teóricos y decisiones de arquitectura.

Principio teórico	Decisión de arquitectura	Fundamento de la correspondencia
Teoría del engaño (§3.4): un mecanismo de deception debe desviar la atención del atacante sin comprometer activos productivos reales (Spitzner, 2002).	Honeypot desplegado en un entorno Docker sin exposición entrante desde Internet (§4.11).	El aislamiento no es una medida de seguridad genérica, sino la materialización técnica del principio de “atraer sin arriesgar” que define al engaño como estrategia defensiva.
Taxonomía y criterios de selección (§3.5.3): el tipo de honeypot debe responder al escenario de uso antes que a la disponibilidad de la herramienta.	Honeypot de media interacción con comandos post-autenticación (Cowrie); el prototipo implementa solo Cowrie (§5.4).	La media interacción da más datos que la baja interacción sin el riesgo operativo de la alta (§3.5.3.2), lo que resulta adecuado para el escenario académico definido.

Principio teórico	Decisión de arquitectura	Fundamento de la correspondencia
Ciclo de vida PCPAD (§3.7) como esqueleto funcional, no como referencia teórica aislada.	Collect en el sensor Cowrie; Process y Analyze en los flujos n8n (§5.6); Disseminate en reportes automatizados y alertas en tiempo real.	La plataforma de automatización no es un agregado posterior al diseño, sino el componente que instrumenta el ciclo de inteligencia con la menor intervención manual posible (objetivo de diseño; en el prototipo, la extracción y el reporte programados operaron en modo trigger, aunque P3 y P4 no alcanzaron sus umbrales, §6.9), objetivo que exige la justificación técnica y operativa.
Estándares de representación de IoCs (§3.11.2; Anexo IX).	Diseño objetivo, no implementado: priorizar STIX 2.1 (§3.11.3) como estándar de representación interno antes que la integración con MISP o TAXII.	La falta de integración con el ecosistema externo de compartición (§3.10) es consecuencia coherente de la priorización, apropiada para el alcance académico, y se identifica como trabajo futuro.

Nota. Elaboración propia.

Capítulo IV: Metodología

4.1 Introducción

Códigos de las validaciones. Las validaciones se citan por un código: L0 (lote del 25/09/2026), W1 (ventana del 28/09, antecedente), W2 (ventana del 30/09, la de veredicto), C1 (contraste del 02/10, con reglas escritas antes de la ventana pero publicadas después), C2 (contraste del 05–06/10, con registro previo verificable) y C3 (contraste del 08–09/10, con registro previo verificable, sobre la configuración final). R1 es la validación ampliada del 10–11/10, con sesiones reales de un dataset público (§6.13). B1 y B1c son las mediciones de la línea de base manual con participantes ajenos al equipo (05/10 y 08/10). Todas las fechas son de 2026; la Tabla 4.1 describe cada validación, el Glosario de códigos y abreviaturas (antes del Capítulo I) los reúne con los de las hipótesis y los modos de ejecución, y el Anexo VI da su cronología.

Este capítulo establece el diseño metodológico, con coherencia entre objetivos, hipótesis operativas y procedimientos técnicos y con criterios de rigurosidad: trazabilidad entre problema, variables y resultados, reproducibilidad del entorno, medición objetiva del desempeño y validez interna y externa. El enfoque combina la observación de eventos generados por un estímulo simulado y controlado con el desarrollo tecnológico del prototipo (Capítulo V), en dimensiones cuantitativas y cualitativas.

4.2 Enfoque metodológico

La investigación adopta un enfoque mixto con predominio cuantitativo (Hernández Sampieri et al., 2014), estructurado en dos dimensiones complementarias:

4.2.1 Dimensión cuantitativa

Orientada a medir el comportamiento del sistema mediante indicadores verificables: cantidad de eventos capturados, frecuencia de ataques por unidad temporal, tiempo de procesamiento por evento, volumen de IoCs generados y tasa de estructuración de datos; permite contrastar las hipótesis operativas en términos cuantitativos.

4.2.2 Dimensión cualitativa

Orientada a caracterizar el comportamiento observado en las sesiones registradas, con categorías definidas a priori —tipo de actividad, complejidad de los comandos y TTPs reconocibles— y mediante la reconstrucción de la sesión (§6.3.3). Como el ataque fue diseñado y ejecutado por los propios autores (§4.11), la «intención del atacante» es conocida y no se infiere del tráfico: esta dimensión verifica que el pipeline registra y estructura el ciclo completo de la sesión, no que descubre intención.

4.3 Tipo de investigación

El estudio es una prueba funcional controlada de un artefacto tecnológico, articulada como estudio de caso (§4.4), con un estímulo determinista diseñado y ejecutado por el propio equipo: aplicada (resuelve un problema concreto), descriptiva (caracteriza el comportamiento del pipeline ante el ataque simulado) y evaluativa (mide el desempeño frente a los criterios de validación del sistema), consistente con investigaciones tecnológicas en entornos dinámicos (Wohlin et al., 2012). No es observacional en sentido estricto ni exploratorio de la variabilidad de amenazas reales, porque el estímulo es un ataque simulado con temporización fija y el atacante es el propio equipo. El diseño es transversal y de intervención técnica controlada.

4.4 Diseño de investigación

El diseño articula la naturaleza multicategorial del tipo de investigación con el problema planteado en un estudio de caso (Yin, 2018): la evaluación de la arquitectura como validación funcional de su cadena de captura → procesamiento → inteligencia en un laboratorio controlado, sin exposición entrante desde Internet. La recolección se estructura como una prueba controlada de corta duración con cuatro componentes. Entorno: el descripto en el §4.11. Estímulo: ataque íntegramente simulado, por Telnet desde el anfitrión en L0 (scripts/attack_simulator.py) y por SSH desde attack-runner, con una tarea programada, desde W1. Observación: dos lotes en L0, las dos ventanas automáticas y los contrastes C1, C2 y C3. Registro: los eventos se persisten en PostgreSQL y los conteos se verifican con las consultas del §6.3.5. El trabajo comparte rasgos con Design Science Research (DSR): produce un artefacto y lo evalúa en iteraciones sucesivas (L0, W1 y W2) con correcciones puntuales, más tres contrastes con reglas escritas de antemano (C1, C2 y C3), declarados como ajustes posteriores a la observación de los datos. Se eligió el estudio de caso porque el interés no es proponer una teoría de diseño, sino validar el funcionamiento de una integración de componentes existentes en un entorno acotado y con evidencia verificable.

4.5 Problema de investigación

La pregunta de investigación es la del §1.2. Se la operacionaliza en tres dimensiones: (1) la generación de inteligencia de amenazas, (2) la estructuración de eventos técnicos y (3) la reducción del tiempo de procesamiento, en comparación con un enfoque manual no automatizado. Queda fuera el fortalecimiento de la capacidad de respuesta defensiva, que el trabajo no mide (§8.2.2): solo el tiempo se compara con un enfoque manual (P1), y las demás dimensiones se contrastan con umbrales absolutos, porque el diseño no incluye grupo de comparación.

4.6 Hipótesis general e hipótesis operativas

La hipótesis general y las cuatro hipótesis operativas (P1 a P4) con sus umbrales se enuncian en el §1.6, y su operacionalización y sus criterios de validación están en los apartados 4.8 y 4.9. Que el sistema fortalezca la respuesta defensiva de una organización no se mide aquí: requeriría tráfico real y analistas.

Regla de decisión. La hipótesis general se considera sostenida si se confirman las cuatro hipótesis operativas y parcialmente sostenida si alguna no se confirma; en ese caso se indica qué dimensiones respalda la evidencia. Cuando P3 o P4 se contrastaron en más de una validación, el veredicto es el de la ventana de veredicto (W2) y las anteriores son antecedente; el de P1 y P2 es el del lote L0, con la ventana W2 y los tres contrastes (C1, C2 y C3) como controles. Los contrastes no modifican los veredictos. Se fijaron al analizar los resultados, no de antemano, la categoría «parcialmente sostenida» y la definición de «ataque de interés» (sesión con autenticación exitosa); lo fijado de antemano fueron los umbrales de P1 a P4 y la operacionalización de P3 por event_id.

Términos propios. La Tabla 4.1 resume las validaciones y el papel de cada una en el análisis; a continuación se definen los demás términos propios de este trabajo.

Tabla 4.1: Validaciones realizadas y papel de cada una en el análisis.

Código y fecha	Qué fue	Papel en el análisis
L0 · 25/09/2026	Lote 1 (sin eventos: ingesta inactiva) y lote 2 (5 sesiones, 65 eventos); cinco corridas del simulador Telnet cada uno	Sustenta P1 y P2; primer contraste de P3 y P4 (§6.2.2)
W1 · 28/09/2026	Ventana automática de ≈ 7 h 32 min, con 10 sesiones con autenticación exitosa	Antecedente: no determina veredictos (§6.2.4)
W2 · 30/09/2026	Ventana automática de 8 h 07 min, con 41 sesiones con autenticación exitosa	Ventana de veredicto de P3 y P4, designada después de observar los resultados (§6.2.4)
C1 · 02/10/2026	Ventana automática de 67 min, con 15 sesiones y reglas escritas antes del primer evento (publicadas después)	Contraste: no modifica los veredictos (§6.2.5)
C2 · 05–06/10/2026	Ventana automática de 14 h, con 66 sesiones y registro previo publicado antes del inicio	Contraste; observa el disparo de producción de las 08:00 (§6.2.6)
C3 · 08–09/10/2026	Ventana automática de 12 h, con 49 sesiones y registro previo publicado antes del inicio	Contraste de la configuración final del 06/10 (§6.2.7)
R1 · 10–11/10/2026	Ventana automática de 19 h 11 min, con 200 sesiones reales de un dataset público (1.020 comandos) reproducidas por los autores y registro previo publicado antes de la primera sesión	Validación ampliada: no modifica los veredictos (§6.13)
Réplica C	Réplica independiente por una persona ajena al equipo, en una máquina limpia y con solo el README (ejecutada el 11/10/2026: exitosa, sin intervenciones del equipo)	Replicabilidad; una ejecución exitosa, con asistencia de IA para los comandos (§6.13.4)
27/09 y 29/09/2026	Dos intentos de validación abandonados antes de su cierre; sus tablas se vaciaron	Descartados: ninguno de sus datos integra la tesis (§6.2.4)

Nota. Una ventana automática es una validación continua sin intervención manual desde su inicio. W2 es la ventana de veredicto. Todas las fechas son de 2026. Elaboración propia.

- **Operacionalizaciones de P3.** Por event_id, fijada de antemano y la que determina el veredicto: la sesión tiene al menos una fila de iocs atribuida a uno de sus eventos. Sobre ioc_sessions, exploratoria y definida después de W1: la sesión tiene al menos un indicador registrado en esa tabla. Completitud (P3′), registrada antes de C1: ioc_sessions contiene todos los indicadores que las reglas del extractor derivan del log de Cowrie de la sesión.
- **Modo trigger y modo manual o cli.** Una ejecución en modo trigger es la que dispara la programación (cron) del propio workflow, sin intervención humana; una ejecución en modo manual o cli se lanza a mano desde el editor o desde la línea de comandos de n8n y no se computa como automática.
- Cierre previsto y cierre real. El cierre previsto es la hora de cierre planificada de una ventana; el cierre real, la hora en que efectivamente se detuvo. En W1 y en W2 no coincidieron, y P4 se informa con ambos (Tabla 6.9); en C1, C2 y C3 las ventanas se cerraron a la hora fijada en su registro previo.
- **Línea de base estimada y línea de base medida.** La estimada es la descomposición propia del procesamiento manual de un evento en cinco fases (≈30 s; Tabla VIII-1). La medida es la mediana por evento del triaje y la extracción sobre 20 eventos: 18,44 s con dos integrantes del equipo (02/10/2026, día de C1), y 10,36 s y 11,01 s con cinco y diez participantes ajenos (B1 y B1c; §6.12.1). En este documento «B1» y «B1c» designan solo las mediciones con participantes ajenos al equipo; la del equipo se cita por su fecha.

Decisiones tomadas antes y después de observar los datos. La Tabla 4.2 separa lo que se fijó antes de observar los datos que sustentan los veredictos de lo que se estableció al analizarlos.

Tabla 4.2: Decisiones tomadas antes y después de observar los datos.

Antes de observar los datos	Después de observar los datos
Umbrales de P1 a P4 (50 %, 80 %, 70 % y 90 %)	Ventana de veredicto de P3 y P4 (W2)
Operacionalización de P3 por event_id	Definición de «ataque de interés» (sesión con autenticación exitosa)
Registro previo publicado antes de la ventana en C2, C3 y R1	Categoría «parcialmente sostenida» de la hipótesis general
Reglas de C1, escritas antes de la ventana (publicadas después)	P3′ (completitud por sesión), definida tras W1, y control estricto de P2, aplicado solo como control posterior

Nota. El detalle por hipótesis está en la columna «Fijado» de la Tabla 8.1. Elaboración propia a partir de los apartados 4.6 y 6.9.

4.7 Variables de investigación

Variable independiente: implementación de la arquitectura honeypot + automatización n8n. Variables dependientes, operacionalizadas en la Tabla 4.3: (1) tiempo de procesamiento por evento; (2) porcentaje de eventos correctamente estructurados; (3) cantidad de IoCs generados;

(4) tiempo de generación de reportes, y (5) nivel de utilidad operativa de la información. La variable (4) no tiene hipótesis ni umbral y se informa de forma descriptiva (§6.6); la (5) se evaluó solo en parte.

4.8 Operacionalización con métricas

La Tabla 4.3 presenta la operacionalización de las variables mediante métricas.

Tabla 4.3: Operacionalización de variables mediante métricas.

Variable	Indicador	Métrica	Método
Tiempo de procesamiento	Latencia	Segundos/evento	Timestamp entrada/salida
Estructuración	Calidad de parsing	% eventos válidos	Campos obligatorios no nulos (§4.9.2)
IoCs	Producción y atribución	Cantidad total y % de sesiones con IoC atribuido (P3)	Base de datos (iocs, ioc_sessions)
Reportes	Cobertura y frecuencia	% de sesiones cubiertas por reportes en modo trigger (P4); reportes/día	Logs de n8n y events
Utilidad	Aplicabilidad	Escala cualitativa	Lectura por cinco participantes ajenos al equipo, no por analistas (§7.7.3); no medida con analistas

Nota. Elaboración propia.

4.9 Criterios de validación del sistema

Se establecen criterios explícitos para determinar si la arquitectura cumple su objetivo:

4.9.1 Reduce el tiempo de procesamiento ≥ 50 % frente a estimación manual.

La línea de base se construyó primero como estimación propia (≈30 s por evento; §6.12.1) y después se midió: con dos participantes del equipo, con cinco ajenos y con otros diez. El registro previo de C1 fijó la primera medición como término de comparación de P1 (§6.2.5).

4.9.2 Estructura correctamente ≥ 80 % de los eventos capturados.

Se considera estructurado todo evento insertado en events con tipo (eventid), sesión (session) y marca temporal (timestamp) no nulos, con independencia de la extracción de indicadores. En el esquema (Anexo III), eventid y timestamp son NOT NULL y session no lo es, por lo que el criterio se verifica con una consulta sobre session además de la restricción del esquema. Una operacionalización más exigente —campos obligatorios por tipo de evento, como username y password en cowrie.login.* o input en cowrie.command.input, y comparación campo a campo con el log de Cowrie— se aplicó después de observar los datos como control de robustez de P2; se informa en el §6.9 y no modifica el veredicto, porque no fue el criterio fijado de antemano para L0 y W2. En los tres contrastes (C1, C2 y C3) se escribió antes del primer evento.

4.9.3 Genera IoCs accionables de forma automática, para al menos el 70 % de las sesiones con actividad maliciosa confirmada.

A los fines de la medición, «sesión con actividad maliciosa confirmada» se operacionaliza como sesión con autenticación exitosa, y «genera IoC» como la existencia de al menos una fila en la tabla iocs atribuible a un evento de la sesión mediante event_id.

Esta operacionalización determina el veredicto de P3. Tras observar el resultado de W1 se incorporó la tabla ioc_sessions, que registra cada aparición de un indicador en una sesión; la medición adicional sobre ella (sesiones con al menos una fila) es exploratoria, por haberse definido después de observar los datos, y no sustituye a la anterior (§6.9).

P3′ (reformulación). Definida después de observar W1, P3′ mide la completitud de extracción por sesión: sesiones con autenticación exitosa en las que ioc_sessions contiene todos los indicadores que las reglas del extractor derivan del log de Cowrie, con el mismo umbral del 70 %. Se escribió antes de C1, con registro publicado después de la ventana, y se registró antes de C2 y C3, con registro previo verificable. Determina el resultado de P3′ en cada contraste, no el veredicto de P3, que se toma de W2.

4.9.4 Produce reportes sin intervención humana directa, para al menos el 90 % de los ataques categorizados como de interés.

Solo se computan los reportes generados en modo trigger (disparo programado); las ejecuciones manuales no cuentan como automáticas. En W1 el disparador de report-generator se reconfiguró de forma deliberada a la expresión 0 */2 * * * (cada 2 horas) para completar la prueba en una ventana práctica; el valor de producción del diseño sigue siendo diario (0 8 * * *). Las expresiones cron se evalúan en la zona horaria de n8n (America/Argentina/Mendoza, UTC−3): 0 8 * * * corresponde a las 08:00 ART (11:00 UTC) y 0 */2 * * * dispara a las 01:00, 03:00, 05:00 y 07:00 UTC en las ventanas de validación. El flujo tal como operó se conserva en la carpeta experimentos del repositorio y el de producción se restituyó al terminar, por lo que la medición de P4 corresponde a la configuración de validación y no a la de producción. W2 usó la misma reconfiguración; C1 usó una expresión diaria, 50 11 * * * (11:50 ART), un único disparo por día adelantado para observarlo dentro de la ventana.

4.9.5 Permite identificar patrones de ataque repetitivos. Es un criterio cualitativo, sin hipótesis ni umbral numérico; se verifica en el §6.12.5, con resultado «Sí (parcial)».

Estos criterios transforman el trabajo de descriptivo a evaluativo medible.

4.10 Población, unidad de análisis y muestra

Población: eventos de seguridad generados por el simulador contra los servicios del honeypot en el laboratorio. Unidad de análisis: cada interacción técnica individual (por ejemplo, un login

SSH o una conexión TCP). Las proporciones de P2 se calculan por evento y las de P3, P3′ y P4, por sesión con autenticación exitosa. Muestra: la totalidad de los eventos válidos del período de observación; es válido todo evento estructurado correctamente por el pipeline (parseo, enriquecimiento y persistencia sin errores de esquema). En L0 los eventos persistidos cumplieron el criterio (§6.9), por lo que la muestra es un censo de la validación y no un submuestreo: no se aplican técnicas de muestreo probabilístico ni se generalizan inferencias estadísticas a otras ventanas.

4.11 Escenario de validación

El entorno es un laboratorio controlado, sin exposición entrante desde Internet: redes Docker bridge (§5.5) con todos los puertos publicados exclusivamente en 127.0.0.1, virtualización mediante Docker y registro de los contenedores. El despliegue sigue un modelo on-premise: todos los componentes (honeypots, n8n, PostgreSQL) se ejecutan en un servidor local bajo el control del equipo operador, por tres razones: (a) control total sobre los datos capturados, que incluyen tráfico potencialmente sensible; (b) ausencia de costos recurrentes de infraestructura cloud, acorde con el perfil de recursos limitados del público objetivo (PyMEs e instituciones educativas); y (c) simplicidad operativa para un equipo pequeño (3 personas), sin departamento de seguridad dedicado. Se prioriza la autonomía operativa sobre el procesamiento masivo, y la escalabilidad no se midió en las validaciones sino en una prueba de carga local posterior (§6.12.3).

4.12 Procedimiento

1. Diseño de la arquitectura segmentada en capas (entorno contenerizado sin exposición entrante; segmentación implementada con redes Docker el 05/10/2026). 2. Implementación de la infraestructura Docker. 3. Configuración del honeypot Cowrie. 4. Validación técnica de conectividad y registro de logs. 5. Integración con flujos automatizados en n8n. 6. Verificación de que el entorno no recibe conexiones entrantes desde Internet. 7. Recolección de datos durante el período de observación. 8. Procesamiento automático de eventos. 9. Análisis de resultados y generación de reportes. Los pasos 1 a 3 corresponden a la infraestructura (Figura 5.3); los pasos 5 a 7, al flujo automatizado, descripto en las Figuras I-1 e I-2 (Anexo I).

4.13 Métodos de análisis

El análisis se limita a estadística descriptiva sobre los eventos persistidos y sobre los conteos SQL de las sesiones autenticadas de cada validación (§6.9), sin pruebas de significación ni modelos inferenciales: el volumen no lo justifica y no se generalizan conclusiones fuera de la

ventana. Las métricas descriptivas —frecuencias por tipo de evento y latencias con mínimo, máximo, promedio, mediana y desviación estándar muestral— se calculan directamente en SQL (count, min, max, avg, stddev y percentile_cont(0.5)), con las consultas publicadas en el §6.3.5 y en los documentos de verificación del repositorio. Los intervalos de confianza del 95 % de las proporciones se calcularon con el método de Wilson (1927) mediante el script de cálculo del repositorio. La reconstrucción de la sesión a partir de las marcas temporales sirve al análisis cualitativo.

4.14 Validez y confiabilidad

Validez interna. La instrumentación se controla con registro automático en JSON, sin intervención humana; la historia se mitiga documentando cada cambio de configuración (§6.2.2); la maduración se acota con ventanas continuas y las imágenes de Cowrie y n8n fijadas por digest en docker-compose.yml; y la selección, registrando cada sesión sin filtrado previo. La principal amenaza es la circularidad: los mismos autores construyeron el entorno, diseñaron y ejecutaron el ataque, definieron los criterios de éxito y evaluaron su cumplimiento. Se mitiga con la trazabilidad del registro (el historial de ejecuciones del pipeline y el log de Cowrie son verificables de forma independiente) y reconciliando las cifras contra PostgreSQL con las consultas publicadas (§6.3.5) cuando la base se conservó (no en W1). Antes del lote L0, una etapa de automatización asistida por IA insertó eventos sintéticos (sesiones con prefijo «sim-») en la tabla de eventos; se detectó cruzando las sesiones con los registros reales de Cowrie y se eliminó, con los IoCs asociados y un reporte contaminado, antes de cualquier análisis (su fecha exacta no quedó registrada). Ninguna cifra vigente se calculó sobre esa base (Anexo V.4); desde entonces cada ventana parte de tablas vacías y, desde W2, se concilia contra el log de Cowrie. Validez externa. El alcance se limita a organizaciones con un perfil de recursos similar (servidor único con Docker, APIs gratuitas, sin personal dedicado), y la replicabilidad se apoya en Docker versionado y configuraciones declarativas. Las muestras no capturan ciclos diarios ni semanales de actividad (Nawrocki et al., 2016): los resultados validan la funcionalidad de la arquitectura en el escenario definido, no caracterizan el tráfico hostil. Validez de constructo. Tiempo de procesamiento, tasa de estructuración y volumen y tipo de

IoCs operacionalizan el constructo de inteligencia de amenazas procesable; la métrica de estructuración subestima la información aprovechable de los eventos sin IoC extraíble, que se conservan para el análisis cualitativo. Confiabilidad. Las marcas temporales las generan Cowrie y n8n; la repetibilidad se sustenta en el versionado Git (Docker Compose, workflows JSON, SQL); las fallas de entrega del forwarder quedan en su log ([OK] o [ERR] por envío), y

la tabla error_log recibe, desde la versión de W2, los errores del upsert de ioc-extractor (§5.6.4); en esa validación registró cero filas.

4.15 Consideraciones éticas

El trabajo adopta las siguientes medidas: los honeypots solo registran actividad pasiva, sin interacción ofensiva ni inducción a conductas delictivas; no se recolectan datos personales de terceros; el uso es exclusivamente académico y el entorno es un laboratorio controlado, sin exposición entrante desde Internet (sobre su salida, §5.6.6), con acceso restringido al equipo. En el prototipo, las credenciales capturadas se almacenan en claro, la conexión de n8n a PostgreSQL no usa TLS (PGSSLMODE=disable; §5.6.6) y n8n tiene NODE_TLS_REJECT_UNAUTHORIZED=0, que desactiva la validación de certificados en sus llamadas salientes; el hashing o cifrado de las credenciales y el cifrado en tránsito son requisitos previos a cualquier despliegue expuesto. El tratamiento de los datos se encuadra, de forma preventiva, en la Ley 25.326 y su reglamentación (Decreto 1558/2001): finalidad determinada y lícita (art. 4), limitada a la validación funcional y la investigación académica. Los datos provienen de un entorno simulado por el equipo y no refieren a personas determinadas o determinables (art. 2), y la ley excluye las investigaciones científicas mientras los datos no puedan atribuirse a una persona (art. 28), mientras que un despliegue expuesto requeriría el consentimiento de los titulares o alguna de las excepciones del art. 5, inc. 2. Minimización (art. 4), con solo direcciones IP, credenciales y comandos del laboratorio; seguridad (art. 9), con acceso restringido y almacenamiento local, sin cifrado en reposo ni en tránsito. Conservación limitada a la vigencia de la investigación, y derechos de acceso, rectificación y supresión (arts. 14 y 16). Como las direcciones IP pueden ser datos personales en ciertas jurisdicciones, no se publica información que permita identificar personas físicas (Sokol et al., 2017).

4.16 Limitaciones metodológicas

Ausencia de tráfico real: el estímulo es simulado y no captura la variabilidad de amenazas externas; R1 la mitiga en parte con sesiones reales de un dataset público reproducidas por los autores (§6.13). Dependencia del período de exposición: resultados acotados al tiempo de observación. Cobertura limitada a los protocolos emulados por Cowrie. Posibilidad de detección del honeypot por atacantes sofisticados. Línea de base manual de tamaño mínimo (N = 2, 5 y 10; §6.12.1): es una referencia descriptiva, no una estimación del tiempo de un analista. La utilidad operativa de la información no se evaluó con analistas.

Capítulo V: Diseño e Implementación del Prototipo

5.1 Introducción

Este capítulo describe cómo está construido el prototipo que se valida en el Capítulo VI: la arquitectura por capas, las herramientas, los componentes desplegados y la red, los cuatro workflows de n8n y el modelo de datos. El diseño de la investigación está en el Capítulo IV y los resultados de aplicarlo, en el Capítulo VI.

5.2 Arquitectura del prototipo

La Figura 5.1 representa la disposición lógica de las herramientas en capas y la Figura 5.3, el despliegue real de contenedores. La Figura 5.2, con VLAN y reglas de firewall entre segmentos, es el diseño objetivo; L0 a C1 usaron una red Docker única y, desde C2 (05/10/2026), las tres capas se implementan con redes Docker separadas, sin VLAN ni firewall (§5.5).

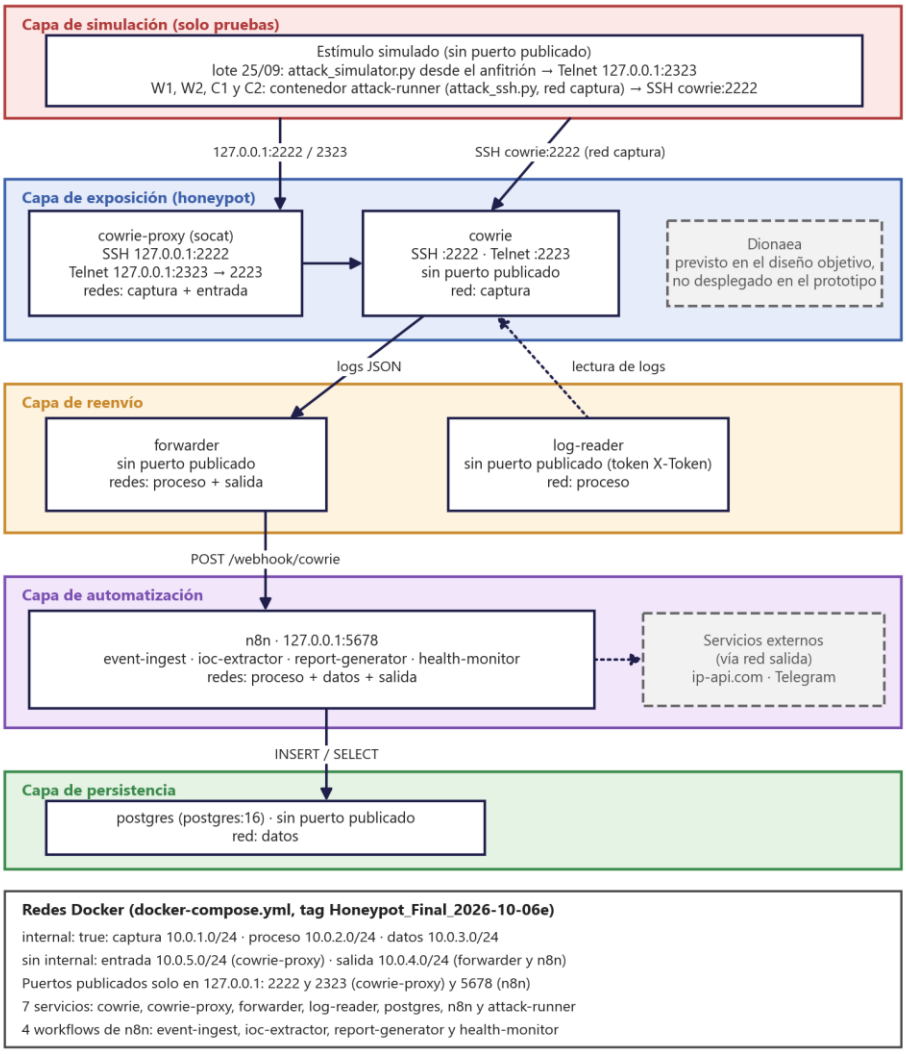


Figura 5.1: Arquitectura general de la solución por capas, en la configuración final. Dionaea figura con línea punteada como componente previsto en el diseño objetivo, no desplegado (§5.4). En esta configuración log-reader y postgres no publican puertos, Cowrie publica los suyos mediante cowrie-proxy, n8n ejecuta cuatro workflows (incluido health-monitor) y las redes Docker son captura, proceso y datos (internal) más entrada y salida (Tabla 5.3, §5.5). El simulador atacó por Telnet (puerto 2323) en el lote L0 y por SSH (puerto 2222) en las validaciones W1, W2, C1, C2 y C3. Fuente: Elaboración propia.

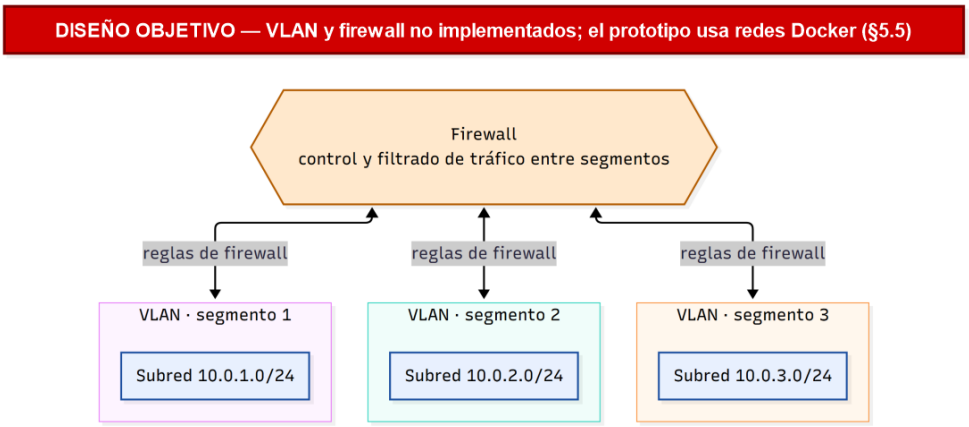


Figura 5.2: Topología de red con segmentación VLAN y reglas de firewall — diseño objetivo. El prototipo no implementa VLAN ni firewall: las validaciones L0 a C1 usaron una red Docker única; desde el 05/10/2026 (validación C2) sus tres capas se implementan con redes Docker (10.0.1.0/24, 10.0.2.0/24 y 10.0.3.0/24, rangos privados; Rekhter et al., 1996), con la red de datos como internal, y desde el 06/10/2026 las tres son internal, más una red de entrada y otra de salida (§5.5). Fuente: Elaboración propia.

5.3 Herramientas utilizadas

La Tabla 5.1 resume las herramientas utilizadas y su rol en la arquitectura.

Tabla 5.1: Herramientas utilizadas y previstas, y su rol en la arquitectura.

Herramienta	Versión	Propósito	Rol en la arquitectura
Cowrie	v3.0.15 (etiqueta de la imagen). Imagen fijada por digest (sha256:fc57120d…)	Honeypot SSH/Telnet	Captura de intentos de autenticación y comandos
Dionaea	— (no desplegado; diseño objetivo)	Honeypot multi-protocolo	Captura de malware y conexiones SMB/ HTTP/FTP
n8n	2.39.10 (n8n --version en el contenedor). Imagen fijada por digest (sha256:94d70bcb…)	Plataforma de automatización	Ingesta, parseo, enriquecimiento y almacenamiento
Docker	No registrada durante las ventanas	Contenedores	Aislamiento y despliegue de componentes
PostgreSQL	postgres:16 (sin digest); la imagen local del 07/10/2026 contiene la 16.14	Base de datos relacional	Persistencia de eventos e IoCs
ip-api.com	—	API de geolocalización	Enriquecimiento: país, consultado desde el nodo Code de event-ingest (capa gratuita, 45 req/min, según la documentación del proveedor, no verificada aquí)
AbuseIPDB	—	API de reputación IP	(no implementada; diseño objetivo)
VirusTotal	—	API de análisis de hashes	(no implementada; diseño objetivo)

Nota. Elaboración propia. Las versiones de Cowrie, n8n y PostgreSQL se consultaron el 07/10/2026 en las imágenes de la versión final del 06/10/2026 (docker image inspect y n8n --version); no se registraron durante las ventanas de validación.

Todas las APIs externas utilizadas pertenecen a capas gratuitas o públicas, sin costos de licenciamiento, lo que asegura la replicabilidad sin presupuesto para integraciones externas. El prototipo incluye además un servicio de reenvío (forwarder) que lee los eventos de Cowrie y los envía al webhook de n8n; su registro de entregas es la fuente del diagnóstico del §6.4. Parte del flujo de validación se ejecutó con herramientas de automatización asistidas por IA, cuyo uso y alcance se declaran y se auditan en el Anexo V (§V.4).

5.4 Componentes desplegados

El docker-compose.yml define siete servicios: cowrie, forwarder, log-reader, postgres, n8n, attack-runner y, desde el 06/10/2026, cowrie-proxy, el contenedor de reenvío TCP que publica los puertos de Cowrie (§5.5). Las validaciones L0 a C2 se hicieron con los seis primeros; C3 y R1, con los siete. El prototipo validado incluye, por lo tanto, un único honeypot. La Tabla 5.2 resume los componentes y su exposición.

Tabla 5.2: Servicios del prototipo, dirección en la red de laboratorio y puertos publicados.

Servicio	Función	IP en honeypot-net	Puertos publicados (host → contenedor)
cowrie	Honeypot SSH/Telnet de media interacción	172.18.0.3	Ninguno desde el 06/10/2026 (los publica cowrie-proxy); antes, 127.0.0.1:2222 → 2222 (SSH) y 127.0.0.1:2323 → 2223 (Telnet)
cowrie-proxy	Reenvío TCP de los puertos de Cowrie (solo desde el 06/10/2026)	— (no existía en L0)	127.0.0.1:2222 → 2222 (SSH); 127.0.0.1:2323 → 2223 (Telnet)
forwarder	Lee los eventos de Cowrie y los envía por HTTP POST al webhook de n8n	172.18.0.7	Ninguno
log-reader	Servicio auxiliar de lectura de registros	172.18.0.5	127.0.0.1:9000 → 9000
postgres	Persistencia (tablas events, iocs, reports, error_log, ioc_observations, ioc_sessions)	172.18.0.2	127.0.0.1:5433 → 5432
n8n	Orquestación de los tres workflows de L0 a C1 y de los cuatro desde el 05/10/2026 (se suma health-monitor, §5.6.4)	172.18.0.6	127.0.0.1:5678 → 5678
attack-runner	Contenedor de ejecución de ataques de prueba	— (estado Exited (0))	Ninguno

Nota. Direcciones IP asignadas por Docker al momento de la inspección (docker network inspect); los bindings se confirmaron con docker ps -a. Las direcciones son las del lote L0, en el que el simulador se ejecutó desde el host contra 127.0.0.1:2323 y no desde attack-runner. La red fue 172.19.0.0/16 en W1 y W2 y 172.18.0.0/16 en C1, y en las tres el simulador se ejecutó desde attack-runner. Los puertos de Cowrie los publica un proxy, cowrie-proxy, que no existía de L0 a C2 (C3 y R1 corrieron con él, aunque el tráfico del simulador no pasó por él; apartado 6.2.7) y por el que, con tráfico externo, Cowrie vería la IP del proxy (§5.5). Elaboración propia.

5.5 Red e instrumentación

En L0 a C1 todos los servicios compartieron una única red bridge (honeypot-net), con subred asignada por Docker (172.18.0.0/16 en L0 y 172.19.0.0/16 en W1 y W2). No existían VLAN, subredes adicionales, directivas internal ni reglas de filtrado entre servicios: la red era plana y un contenedor comprometido podría, en principio, haber alcanzado a los demás. La Figura 5.3 representa ese despliegue.

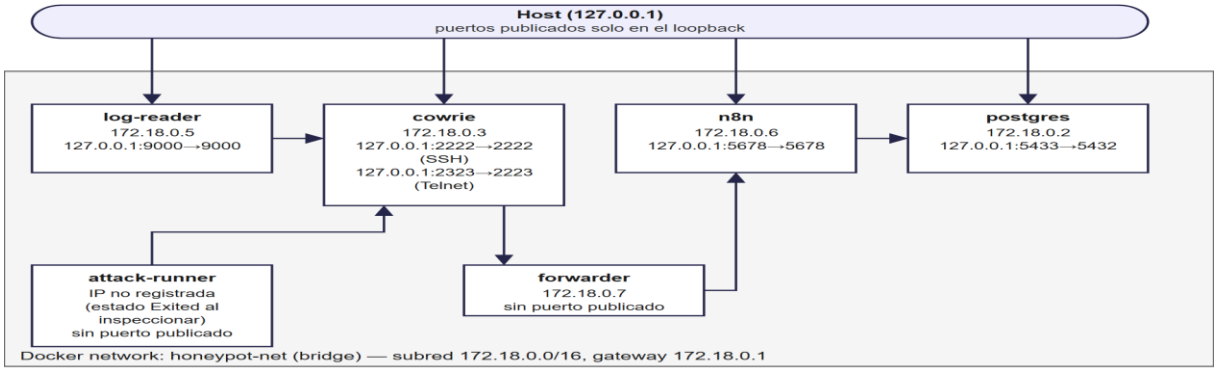


Figura 5.3: Diagrama de contenedores Docker con puertos y red, con las direcciones asignadas por Docker en el lote L0 (subred 172.18.0.0/16). La subred fue 172.19.0.0/16 en W1 y W2 y 172.18.0.0/16 en C1. En las tres el simulador se ejecutó desde attack-runner (§5.5). Fuente: Elaboración propia.

En todas las configuraciones, los puertos se publican solo sobre 127.0.0.1: ningún servicio es alcanzable desde otra máquina de la red física ni desde Internet. Después de C1, las tres capas de la Figura 5.2 se implementaron con redes Docker en lugar de VLAN y firewall. La red de captura (10.0.1.0/24) reúne cowrie y attack-runner; la de proceso (10.0.2.0/24), forwarder, log-reader y n8n; la de datos (10.0.3.0/24), n8n y postgres, que dejó de publicar su puerto. Docker no enruta entre redes bridge distintas: en pruebas fijadas de antemano, la red de captura no alcanzó a n8n ni a postgres, el forwarder no alcanzó a postgres, postgres no salió a Internet y el

pipeline persistió 48 de 48 eventos. Después las tres redes pasaron a internal: true; solo n8n y el forwarder se conectan además a una red de salida, para ip-api.com y Telegram, y Cowrie publica sus puertos mediante cowrie-proxy.

La Tabla 5.3 distingue la configuración de cada validación de la configuración final: los veredictos de las hipótesis se obtuvieron con las dos primeras columnas, no con la última; C3 (§6.2.7) contrasta la última.

Tabla 5.3: Configuración validada y configuración final del prototipo.

Aspecto	Validaciones de L0 a C1	Validación de C2 (05/10/2026)	Configuración final (06/10/2026)
Redes	Una red bridge plana (honeypot-net), sin VLAN ni reglas entre servicios	Tres redes Docker (captura, proceso y datos); solo la de datos con internal	Las tres redes con internal: true, más una red de entrada y otra de salida
Puertos de Cowrie	Publicados en 127.0.0.1 directamente por Cowrie	Ídem	Publicados por un contenedor de reenvío TCP (cowrie-proxy); Cowrie ve la IP del proxy
PostgreSQL	Puerto publicado en 127.0.0.1	Sin puerto publicado; alcanzable solo desde n8n	Ídem
Contenedores	Sin endurecimiento específico	Ídem	cap_drop: ALL, no-new-privileges, límites de memoria, CPU y procesos, y healthchecks (salvo attack-runner)
Acceso a log-reader	Sin token	Sin token	Token obligatorio (LOG_READER_TOKEN)
Monitoreo y alertas	Sin monitor de extremo a extremo	health-monitor cada 5 min, con alertas solo en error_log	health-monitor con aviso también por Telegram
Forwarder	Lee el log desde el final; no reintenta si n8n no responde	Ídem	Guarda la posición del log y reintenta cada evento hasta cinco minutos

Nota. Elaboración propia a partir de las secciones 5.5 y 5.6. Los veredictos de P1 a P4 provienen de la primera columna (L0 y W2); C1, C2 y C3 son contrastes. La configuración final se probó primero con pruebas propias y se observó después en una ventana propia, C3.

5.6 Implementación de workflows n8n

Esta sección describe los workflows tal como están exportados en la carpeta n8n/workflows/ del repositorio. El workflow health-monitor (health-monitor.json, 12 nodos) se describe en el §5.6.4.

5.6.1 Workflow de ingesta de eventos (event-ingest.json, 3 nodos)

Webhook (n8n-nodes-base.webhook, POST /webhook/cowrie) → Process and Insert (n8n-nodes-base.code) → Postgres: Insert Event (n8n-nodes-base.postgres). El nodo Code normaliza el evento de Cowrie a las columnas de events y obtiene el país de la IP de origen mediante this.helpers.httpRequest() a ip-api.com, con un tiempo de espera de 2 s y el valor «Desconocido» ante falla (hasta el 29/09/2026 usaba fetch(), no disponible en el entorno de n8n; §6.8.3); el nodo Postgres inserta la fila con processed = false. No hay nodos de validación, de reputación ni de manejo de errores, y el disparo por webhook exige que el workflow esté activo (§6.4).

5.6.2 Workflow de extracción de IoCs (ioc-extractor.json, 6 nodos)

Schedule (scheduleTrigger, cron */15 * * * *) y Manual (Execute Workflow) (executeWorkflowTrigger) → Postgres: Eventos sin procesar → Extraer IoCs (Code) → Postgres: Upsert IoCs, cuya salida de error lleva a Postgres: Registrar error. La lectura toma hasta 500 eventos con processed = false, incluida la columna session. El nodo Code descarta los ítems sin identificador (el ítem {"success": true} que entrega la lectura cuando no hay pendientes) y extrae, por evento, la IP de origen de los eventos cowrie.session.connect, la credencial usuario:contraseña de todo evento que la contenga, los comandos que coinciden con un patrón de descarga o ejecución y los hashes SHA-256 presentes en el mensaje. Una única sentencia del nodo Postgres: Upsert IoCs inserta los indicadores en iocs con ON CONFLICT (type, value) DO UPDATE … RETURNING id, registra en ioc_sessions una fila por indicador (ioc_id, event_id, session), registra en ioc_observations si el evento produjo un indicador y marca el evento como processed = true. La confianza queda en el valor por defecto (BAJO) y no se consultan servicios externos.

5.6.3 Workflow de generación de reportes (report-generator.json, 5 nodos)

Schedule (cron 0 8 * * *) y Manual (Execute Workflow) → Postgres: Datos del reporte → Ensamblar Reporte (Code) → Postgres: Guardar Reporte. Calcula una ventana móvil de 24 horas y guarda el resultado como JSONB en reports.content, junto con period_start y period_end.

5.6.4 Manejo de errores

Parcial. Desde W2, el nodo Postgres: Upsert IoCs de ioc-extractor tiene la salida de error activada (onError: continueErrorOutput) y la deriva a Postgres: Registrar error, que inserta el workflow, el nodo y el mensaje en error_log. En W2 la tabla terminó con cero filas. Los workflows event-ingest y report-generator no definen ramas de error. Para las fallas silenciosas, después de C1 se incorporó el workflow health-monitor, que corre cada 5 minutos: compara los eventos que Cowrie registró entre 20 y 2 minutos antes, leídos de log-reader, con los persistidos en events, busca eventos sin procesar con más de 35 minutos y escribe una alerta en error_log ante cualquier diferencia. En pruebas fijadas de antemano alertó al desactivar event-ingest y al detener el forwarder, y dejó de alertar tras la recuperación. Con la credencial inválida de C1 no dejó registro, porque la base era inaccesible, y al recrear el entorno detectó una pérdida real de 22 eventos que el forwarder no recuperaba. Esos tres hallazgos se corrigieron: health-monitor envía ahora cada alerta también por Telegram y avisa si no puede consultar la base o log-reader; el forwarder guarda la posición del log y la retoma tras un reinicio (con el forwarder detenido durante una corrida se persistieron 69 de 69

eventos, frente a 0 de 61 antes) y reintenta cada evento hasta cinco minutos si n8n no responde (55 de 55 con n8n reiniciándose, frente a 21 de 33 sin reintentos).

5.6.5 Alertas vía Telegram

La alerta de cada ataque por Telegram reside en el forwarder; el aviso de fallas del pipeline lo envía health-monitor (§5.6.4). La prueba, con plan fijado de antemano, confirmó tres defectos: el resultado de cada envío no se registraba; Telegram rechazó el resumen de una sesión con «<» en un comando, porque el texto se envía como HTML sin escapar; y el envío era sincrónico, de modo que con Telegram inalcanzable se perdieron las 20 alertas y la mediana de latencia de la ingesta pasó de 1,9 s a 16,9 s. Se corrigieron con el registro de cada intento sin el token, el escape de los valores del atacante y el envío en un hilo aparte. El token y el chat ID están solo en .env, y las alertas están activadas.

5.6.6 Credenciales de servicio y control de acceso

Los workflows acceden a PostgreSQL mediante una credencial configurada en n8n; el repositorio publica solo un archivo de ejemplo. La versión final no contiene credenciales reales, pero el historial público conserva la contraseña de PostgreSQL de laboratorio, que era el valor por defecto del compose, y la de login de n8n. Ambas se rotaron y el historial no se reescribió, porque cambiaría los hashes de los commits y los tags que cita este trabajo. Eran credenciales de un laboratorio sin exposición entrante desde Internet. En las validaciones, con la red sin segmentar, nada impedía a Cowrie alcanzar a PostgreSQL más que sus credenciales; con la segmentación posterior, PostgreSQL solo es alcanzable desde n8n (§5.5).

Seguridad del propio pipeline. El prototipo está endurecido dentro del laboratorio, pero no se evaluó frente a tráfico real ni se diseñó para exponerse a Internet. (a) Antes, las consultas SQL de n8n se armaban concatenando valores controlados por el atacante, un vector de inyección SQL; ninguno de los 3.097 eventos de las validaciones hasta C1 contiene apóstrofos ni barras invertidas, por lo que los resultados no se vieron afectados. Después, los cuatro workflows pasaron los valores como parámetros del nodo Postgres: en pruebas fijadas de antemano, la versión anterior hizo fallar el reporte diario y la nueva guardó los 21 eventos con sus indicadores y el reporte. (b) Antes, docker-compose.yml no limitaba recursos ni capacidades y solo PostgreSQL tenía healthcheck; ahora todos los servicios tienen cap_drop: ALL (postgres conserva cinco capacidades que necesita su imagen), no-new-privileges y límites de memoria, CPU y procesos, y, salvo attack-runner, healthcheck. (c) En la configuración final, Cowrie, attack-runner, log-reader y postgres no tienen salida a Internet; n8n y el forwarder sí, y n8n consulta ip-api.com por HTTP sin cifrar. (d) log-reader exige un token y no publica su puerto. (e) N8N_ENCRYPTION_KEY es obligatoria (docs/DETALLES_CUERPO.md).

Quedan pendientes el TLS hacia PostgreSQL (PGSSLMODE=disable), el sistema de archivos de solo lectura y los secretos fuera de .env (§8.4).

5.7 Modelo de datos

El esquema relacional consta de seis tablas en PostgreSQL: events, iocs, reports y error_log, correspondientes a las etapas de captura, extracción de IoCs, generación de reportes y manejo de errores descriptas en la sección anterior, e ioc_observations e ioc_sessions, que registran la trazabilidad del extractor por evento y por sesión. La restricción UNIQUE (type, value) de iocs implementa la deduplicación mediante UPSERT, y la relación con cada sesión se registra en ioc_sessions. El archivo db/schema.sql del repositorio define las seis tablas y los ocho índices; el modelo se presenta en la Figura III-1 y el DDL, en el Anexo III.

Capítulo VI: Resultados de la Validación Funcional

6.1 Introducción

Este capítulo presenta los resultados de la validación funcional del prototipo, obtenidos en el lote L0, en las ventanas automáticas W1 y W2 y en los contrastes C1, C2 y C3 (Tabla 4.1), sobre el entorno descripto en el Capítulo V; W2 es la ventana de veredicto de P3 y P4 (§4.6). Todas las cifras provienen de consultas directas a la base PostgreSQL del laboratorio, de los registros de los contenedores y de la interfaz de línea de comandos de n8n, y se acompañan de las consultas que permiten reconstruirlas (§6.3.5). Las fechas, las rutas de evidencia, los tags y las expresiones de detalle de los Capítulos V a VII no figuran en el texto: están en BITACORA.md y en docs/DETALLES_CUERPO.md del repositorio.

El capítulo adopta tres convenciones. Primero, el denominador de las proporciones por sesión (P3 y P4) es el número de sesiones con autenticación exitosa persistidas en la tabla events (en W1, cuya base no se conservó, las registradas en el log de Cowrie; §6.2.4); el de las proporciones por evento (P2), el de eventos persistidos y, en el criterio estricto, el de eventos con campos obligatorios por tipo (Anexo VII.2). Las sesiones ejecutadas que no llegaron a la base se informan aparte, como resultado del lote y no como fracasos de la hipótesis. Segundo, una ejecución disparada a mano (modo cli o manual) no se computa como automática. Tercero, las ventanas anteriores a L0 no aportan evidencia a este capítulo (§6.2.3 y Anexo V); la evidencia procede de las seis validaciones de la Tabla 4.1.

6.2 Diseño del entorno de validación

6.2.1 Flujo operativo de la validación

El procedimiento tuvo seis pasos. Primero, se registró una línea de base de conteos en events e iocs. Segundo, se ejecutaron cinco corridas del simulador, con 30 s de espera entre una y otra. La separación observada fue mayor, de 47,6 a 65,8 s, porque cada corrida dedica tiempo a capturar los logs y a generar su informe (Tabla 6.3). Tercero, se verificaron los conteos y los logs del forwarder y de n8n. Cuarto, se ejecutaron ioc-extractor y report-generator. Quinto, se verificaron las tablas iocs y reports. Sexto, se verificó el estado de activación de los workflows con n8n list:workflow --active=true.

El ataque es íntegramente simulado (§4.4). El script de simulación, un cliente Telnet, se conecta a Cowrie en 127.0.0.1:2323 en cada corrida. Realiza dos intentos de autenticación fallidos y uno exitoso, y ejecuta seis comandos con temporización fija, sin operador humano. Es una amenaza a la validez externa (§6.11).

6.2.2 Período de observación

El lote L0 se ejecutó en dos tandas de cinco corridas del simulador. La primera (lote 1) no dejó eventos en la base, porque el workflow de ingesta estaba inactivo y el webhook devolvió HTTP 404 a las 65 entregas. Tras activarlo, la segunda (lote 2) persistió 65 eventos de 5 sesiones. Después se ejecutaron a mano ioc-extractor y report-generator (cronología en la Tabla VI-1, Anexo VI.3).

6.2.3 Relación con las ventanas de validación anteriores

Las ventanas del 10–11/08, del 10/09 y del 16/09/2026 se documentan en los Anexos II, III y V como antecedente del desarrollo. Sus cifras de P3 y P4 se retiran, porque no pueden reconstruirse con las consultas ni con los artefactos publicados. Ningún veredicto depende de ellas. La única excepción es la medición de tiempos de la sesión del 10–11/08/2026 (Anexo III, Tabla III-1), citada en el §6.3.2 solo como referencia complementaria de P1.

6.2.4 Ventanas automáticas W1 y W2

El lote L0 dejó P3 y P4 sin veredicto, por lo que se realizaron dos ventanas automáticas, sin intervención manual desde su inicio (Tabla 6.2). W1 (≈ 7 h 32 min; 10 sesiones con autenticación exitosa, reconstruidas desde el log de Cowrie) se informa como antecedente. El resultado de P4 dependía del momento de cierre: fue de 5 de 9 sesiones con el cierre previsto a las 6 h y de 9 de 10 con el cierre real. W2 (de las 23:53:57 UTC del 29/09 a ≈ 08:01 UTC; 8 h 07 min) es la ventana de veredicto de P3 y P4. Registró 1.998 eventos, 41 sesiones con autenticación exitosa, 21 indicadores y 4 reportes en modo trigger, sin pérdidas entre la captura y la persistencia y con error_log vacío. Su cierre real incluyó dos sesiones posteriores al cierre previsto. Con el cierre previsto, P4 sería de 35 de 39 sesiones (89,7 %), también por debajo del umbral. Los dos intentos descartados y los ajustes hechos tras observar los datos se declaran como amenazas a la validez (§6.11); la cronología está en el Anexo VI.1.

6.2.5 Contraste C1, con reglas escritas de antemano

C1 respondió a tres debilidades de las ventanas anteriores: el cierre no fijado de antemano, el disparo diario nunca observado y la medición de P3 definida a posteriori. Sus reglas se fijaron por escrito antes del primer evento, pero el registro se publicó después de la ventana: su anterioridad descansa en la declaración de los autores. Duró 67 minutos, sobre una instalación nueva y con el disparo diario de report-generator adelantado. Cowrie registró 629 eventos de 103 sesiones, 15 con autenticación exitosa; las otras 88 son sesiones del propio simulador con credenciales inválidas. La tabla events contiene los mismos 629. El extractor generó 26 indicadores y 211 filas en ioc_sessions, y el reporte diario se ejecutó una vez en modo trigger, con error_log vacío. Ese día se midió también la línea de base manual (§6.12.1). La

configuración, los tres desvíos y la preparación están en el Anexo VI.2 y en docs/ DETALLES_CUERPO.md.

6.2.6 Contraste C2, con registro previo

C2 buscó observar el disparo de producción y medir la exactitud de la extracción. Su registro previo se publicó antes del inicio, condición que verificaba el propio script de inicio. Corrió sin modificar ningún workflow: report-generator conservó el cron de producción. La ventana fue de las 18:14:25 ART del 05/10, con las tablas vacías, a las 08:15 ART del 06/10, cierre fijado de antemano. El simulador corrió cada 15 minutos de las 19:30 a las 07:45 ART (50 corridas).

Cowrie registró 3.218 eventos y events contiene los mismos 3.218, sin claves sin par. Hubo 66 sesiones con autenticación exitosa, 32 indicadores en iocs y 1.108 filas en ioc_sessions. El workflow ioc-extractor se ejecutó 56 veces y el workflow health-monitor, 168, todas con éxito y sin alertas ni ejecuciones manuales. La tabla error_log terminó vacía. El workflow report-generator se ejecutó una vez: el disparo de producción de las 08:00:00 ART del 06/10, en modo trigger y con éxito, observado por primera vez. Los resultados por hipótesis están en el §6.9.

6.2.7 Contraste C3, con registro previo y sobre la configuración final

C3 validó con una ventana propia la configuración final del 06/10, que ni C2 ni las pruebas de correcciones habían cubierto (Tabla 5.3, última columna): redes internal, token en log-reader, forwarder con posición guardada, alertas de Telegram en un hilo aparte, consultas parametrizadas y health-monitor con aviso por Telegram. Su registro previo se publicó antes del inicio y los umbrales son los de C2, con una comprobación de la configuración al cierre. La ventana fue de las 20:13:59 ART del 08/10, con las tablas vacías, a las 08:15 ART del 09/10, cierre fijado de antemano. El simulador corrió cada 15 minutos de las 22:30 a las 07:45 ART (38 corridas, sin errores) y atacó por SSH desde attack-runner (10.0.1.4) con las doce credenciales válidas de C1 y C2. No se modificó ningún workflow: report-generator conservó el cron de producción.

Cowrie registró 2.386 eventos de 397 sesiones, 49 de ellas con autenticación exitosa, y events contiene los mismos 2.386, sin claves sin par. Hubo 32 indicadores en iocs y 817 filas en ioc_sessions. El workflow ioc-extractor se ejecutó 48 veces y el workflow health-monitor, 144, todas con éxito y sin alertas ni ejecuciones manuales. La tabla error_log terminó vacía. El workflow report-generator se ejecutó una vez: el disparo de producción de las 08:00:00 ART del 09/10, en modo trigger y con éxito, observado por segunda vez. Se enviaron 793 alertas de Telegram y ninguna fue rechazada: 397 de conexión, 348 de login fallido y 48 de 49 resúmenes; el restante no salió por un error SSL, que el forwarder registró y no reintentó. Al cierre corrían los seis servicios de ejecución continua; attack-runner termina después de cada

corrida. Los puertos publicados eran solo 127.0.0.1:2222, :2323 y :5678 y las redes de captura, proceso y datos eran internal (Tabla VI-2).

C3 es un contraste y no modifica los veredictos (§4.6). Sus límites son los de C1 y C2 (Tabla 6.9): tráfico simulado desde una única dirección interna, con guion fijo. Además, el simulador se conecta a Cowrie dentro de la red de captura, por lo que el tráfico no pasó por los puertos de cowrie-proxy, de los que solo se verificó la publicación en 127.0.0.1. Fue una sola noche de 12 h sobre la misma arquitectura, no una réplica independiente, y el resumen perdido muestra que las alertas de Telegram son de mejor esfuerzo. Los resultados por hipótesis están en el §6.9.

6.3 Análisis cuantitativo

6.3.1 Resumen de conteos

La Tabla 6.1 resume los conteos de sesiones por lote L0.

Tabla 6.1: Sesiones por lote, persistencia y cobertura (denominador: sesiones persistidas).

Lote	Ejecutadas	Persistidas	Con login exitoso	Con ≥ 1 fila en iocs atribuible por event_id	Cubiertas por reporte en modo trigger
Lote 1 (01:28–01:46 ART)	5	0	—	—	—
Lote 2 (01:59–02:14 ART)	5	5	5	1 (e355bc9ba0d1)	0

Nota. Elaboración propia a partir de los conteos de PostgreSQL y de la Tabla 6.6.

Tabla 6.2: Resultados de las ventanas W1 y W2.

Indicador	W1	W2
Ventana (UTC)	00:57:55 – 08:30 (7 h 32 min)	29/09 23:53:57 – 30/09 ≈08:01 (8 h 07 min)
Frecuencia del simulador	Cada hora	Cada 15 min
Eventos (procesados al cierre)	405 (405)	1.998 (1.930)
Sesiones con autenticación exitosa	10	41
Indicadores en iocs	21	21
Filas en ioc_sessions	— (tabla inexistente)	654
Reportes en modo trigger	4	4
Ejecuciones de ioc-extractor (éxito / error)	30 (7 / 23)	33 (33 / 0)
Filas en error_log	— (sin escritor)	0

Nota. Elaboración propia a partir de los documentos de verificación y los CSV de evidencia del repositorio. Columna W1: reconstruida desde el log de Cowrie y el historial de n8n, porque la base de esa ventana no se conservó (§6.2.4); el valor de eventos procesados al cierre proviene del documento de verificación de W1.

Tabla 6.3: Sesiones persistidas del lote 2.

Sesión (events.session)	Eventos	Primer evento (UTC)	Último evento (UTC)	Duración
e355bc9ba0d1	13	04:59:58.114	05:02:08.450	130,3 s
54cf066a37cd	13	05:03:01.241	05:05:11.601	130,4 s
5e6fe1c13050	13	05:05:59.178	05:08:09.527	130,3 s
d1e20e2ea9b9	13	05:09:15.299	05:11:25.621	130,3 s

Sesión (events.session)	Eventos	Primer evento (UTC)	Último evento (UTC)	Duración
a3f0aaaa0cca	13	05:12:14.805	05:14:25.184	130,4 s
Total	65			

Nota. La tabla events contiene además una fila sin sesión (04:58:31 UTC), correspondiente al ping sintético de prueba del webhook; se excluye de todos los cómputos. La coincidencia de 5 × 13 = 65 con el delta medido confirma que no hubo pérdidas en el lote 2. Elaboración propia.

Cada sesión generó exactamente 13 eventos: cowrie.session.connect, dos cowrie.login.failed, un cowrie.login.success, cowrie.session.params, seis cowrie.command.input, cowrie.log.closed y cowrie.session.closed.

6.3.2 Métricas de procesamiento

La latencia de entrega al webhook de n8n, medida con el log del forwarder, es inferior a un segundo por evento (resolución del log); su equivalencia con la persistencia la confirma la consulta V1 del §6.3.5 sobre los 65 eventos del lote 2 (delta created_at − timestamp): media 348,02 ms, mediana 330,56 ms, mínimo 100,16 ms, máximo 798,34 ms, desviación estándar muestral 160,08 ms (Anexo III, Tabla III-1b). Esta medición es la base cuantitativa principal de P1.

Como referencia complementaria, la sesión del 10–11/08/2026, cuyo agregado proviene de un registro anterior del proyecto (las 13 mediciones individuales no se conservan), informó una media de 297 ms (Anexo III, Tabla III-1), del mismo orden de magnitud aunque no directamente comparable con la medición principal.

En la validación W2, la misma diferencia created_at − timestamp sobre los 1.998 eventos de la ventana dio una media de 1.608,76 ms (mediana 1.037,16 ms; mínimo 412,20 ms; máximo 4.922,22 ms; desviación estándar muestral 984,98 ms), y la duración de las 1.998 ejecuciones de event-ingest registrada por n8n, una media de 1.299,42 ms (mediana 687,50 ms; máximo 3.269 ms) (docs/evidencia/latencia_2026-09-30.csv y docs/evidencia/ ejecuciones_duracion_2026-09-30.csv). El aumento respecto de L0 es consistente con la consulta de país, que desde el 29/09/2026 agrega una solicitud HTTP a ip-api.com por evento (§6.8.3), y con la llegada en ráfagas de los eventos de cada corrida, que n8n procesa de a uno. Ningún evento superó los 15 s del umbral de P1.

6.3.3 Temporización de las sesiones

Las cinco sesiones tienen duraciones prácticamente idénticas (130,3–130,4 s). Los intentos de autenticación se registran con ≈22 s de separación y los seis comandos, con 10–11 s. Esta regularidad es la de un cliente con temporización fija y confirma que la actividad es la del simulador; no admite inferencias sobre comportamiento humano.

6.3.4 Credenciales observadas

En el lote L0, la base de usuarios de Cowrie (cowrie/userdb.txt) definía dos credenciales válidas: admin:test123 y guest:guest123 (en las validaciones W1 y W2 son seis; §6.2.4). En cada sesión se intentaron tres combinaciones (Tabla 6.4).

Tabla 6.4: Credenciales intentadas en el lote 2.

Credencial	Resultado	Intentos (5 sesiones)
admin:123456	Fallido	5
admin:admin	Fallido	5
admin:test123	Exitoso	5

Nota. Elaboración propia.

Política de credenciales. Las credenciales capturadas se almacenan en claro en events.password e iocs.value y se exhiben en claro en este documento, por su valor analítico: contar las más frecuentes, usarlas en listas de bloqueo o detectar reutilización exige conservar el valor original. Son credenciales de laboratorio, definidas por el equipo, que no pertenecen a ninguna persona ni constituyen datos personales en los términos del art. 2 de la Ley 25.326; en un despliegue expuesto a Internet podrían vincularse con personas reales (§9.4).

6.3.5 Consultas

Las cifras de este capítulo se obtuvieron con consultas SQL sobre la base del laboratorio (conteos y sesiones, atribución por sesión de P3, cobertura por reportes de P4 y verificaciones V1 a V3), cuyo texto completo está en el Anexo VII.5; sus salidas literales están en la carpeta de evidencia y en los documentos de verificación del repositorio.

6.4 Observaciones generales del entorno

El hallazgo operativo más relevante es la falla silenciosa del lote 1: las cinco corridas del simulador terminaron sin errores del lado del cliente, pero ninguno de los 65 eventos llegó a la base. El log del forwarder registró la respuesta HTTP 404 de n8n en cada entrega, porque el workflow event-ingest no estaba activo y un workflow con disparador Webhook solo registra su URL de producción cuando está activado. La falla fue invisible para el simulador, que valida la respuesta de Cowrie y no la persistencia, y el sistema no la registró; desde el 05/10/2026 health-monitor detecta este tipo de falla (§5.6.4). Se corrigió activando el workflow por CLI y reiniciando el contenedor, y el lote 2 persistió el 100 % de sus eventos.

6.5 Caracterización de la actividad observada

6.5.1 Naturaleza de las interacciones

La actividad registrada es la de cinco réplicas de un mismo guion automatizado: misma IP de origen, mismas tres credenciales en el mismo orden, mismos seis comandos y misma temporización. La muestra carece de variabilidad y no permite conclusiones sobre tácticas de atacantes; su función es verificar que cada etapa del pipeline procesa correctamente una sesión completa, de manera repetida y sin pérdidas: los 13 eventos de cada sesión reconstruyen el ciclo completo de una intrusión en el honeypot

(conexión, autenticación, parámetros, comandos y cierre), granularidad que permite derivar indicadores (IP y credenciales).

6.5.2 Procedencia de la IP de origen

Todas las sesiones registran como origen 172.18.0.1, el gateway de la red bridge: el tráfico generado en el host y dirigido a 127.0.0.1:2323 llega a Cowrie con la dirección del gateway. Es un artefacto de la topología del laboratorio y no identifica a ningún atacante, por lo que carece de sentido geolocalizarla o evaluar su reputación (§6.8.3). En las validaciones W1, W2, C1, C2 y C3, el simulador se ejecutó como contenedor en la misma red, por lo que el origen registrado es la dirección interna de ese contenedor (172.19.0.5 en W1, 172.19.0.7 en W2, 172.18.0.7 en C1, 10.0.1.3 en C2 y 10.0.1.4 en C3), que tampoco identifica a un atacante.

6.6 Métricas de evaluación del sistema

La Tabla 6.5 resume los indicadores de validación del lote 2.

Tabla 6.5: Indicadores de validación del lote 2.

Indicador	Valor observado	Fuente
Sesiones ejecutadas / persistidas (lote 2)	5 / 5	Tabla 6.1
Eventos persistidos	65 (13 por sesión)	Tabla 6.3
Pérdidas en el lote 2	0	Delta de conteos
Pérdidas en el lote 1	65 de 65 (workflow de ingesta inactivo)	§6.4
Latencia de entrega por evento	< 1 s (L0)	§6.3.2
Eventos con tipo, sesión y marca temporal	65 / 65	§6.9
IoCs únicos generados	4 (1 ip, 3 credential)	Tabla 6.7
Sesiones con IoC atribuido por event_id	1 / 5 (20 %)	Tabla VII-1
Reportes generados en la validación	1 (modo cli)	§6.7
Ejecuciones programadas observadas	0	§6.7

Nota. Elaboración propia.

Para las variables de la Tabla 4.3 que el lote 2 no permite cuantificar, la validación W2 aporta: 21 IoCs únicos en 8 h 07 min (20 credenciales y una dirección IP; §6.2.4) y 4 reportes en

modo trigger, equivalentes a 12 reportes por día con el cron de validación de 2 horas y a 1 por día con el de producción. Cada ejecución de report-generator duró entre 265 y 1.407 ms (media 715,25 ms) y cada una de ioc-extractor, entre 432 y 3.118 ms (media 1.117,85 ms), según el historial de n8n con hora de fin (docs/evidencia/ejecuciones_duracion_2026-09-30.csv); en cambio, la utilidad operativa de la información no se evaluó con analistas (§4.16).

6.7 Resultados del procesamiento automatizado

La Tabla 6.6 distingue, para cada workflow, el modo en que efectivamente se ejecutó durante la validación.

Tabla 6.6: Modo de ejecución de los workflows en la validación L0.

Workflow (ID)	Disparador definido	Ejecución observada	Modo	Resultado verificado en PostgreSQL
event-ingest (T1MtUBk3OW79VT9q)	Webhook POST /webhook/cowrie	65 invocaciones por el forwarder (lote 2)	Automático (webhook)	+65 filas en events
ioc-extractor (aDnuBfhuyiR2PK5g)	Cron */15 * * * *	1 ejecución con n8n execute	Manual (cli)	iocs: 0 → 4 filas
report-generator (LnAKjWiSmnVvfrZO)	Cron 0 8 * * *	1 ejecución con n8n execute	Manual (cli)	reports: 1 → 2 filas (nueva fila id = 2)

Nota. Elaboración propia.

Solo la ingesta operó de forma automática. La extracción de indicadores y la generación del reporte se ejecutaron a mano mediante el CLI de n8n (docker exec n8n n8n execute --id=…), equivalente al botón «Execute Workflow» del editor. Ambas concluyeron con éxito, lo que prueba que su lógica funciona contra la base real, pero no su disparo programado: no estaban activos y se activaron después por CLI, y tras reiniciar el contenedor, n8n list:workflow --active=true devolvió los tres workflows. No se dejó transcurrir tiempo suficiente para observar una ejecución del extractor cada 15 minutos ni la diaria de las 08:00 del generador: el estado «activo» está verificado; el disparo automático, no.

El reporte id = 2 cubre una ventana móvil de 24 horas (period_start = 24/09/2026 05:16:53 UTC; period_end = 25/09/2026 05:16:53 UTC) que contiene las cinco sesiones del lote 2. La tabla reports contenía previamente una fila (id = 1) cuyo origen no está documentado en esta validación; se excluye de todo cómputo.

6.8 Generación de inteligencia de amenazas

6.8.1 Indicadores generados y deduplicación

Tabla 6.7: Contenido de la tabla iocs tras la ejecución de ioc-extractor.

type	value	confidence	source
ip	172.18.0.1	BAJO	cowrie
credential	admin:123456	BAJO	cowrie

type	value	confidence	source
credential	admin:admin	BAJO	cowrie
credential	admin:test123	BAJO	cowrie

Nota. Elaboración propia.

El extractor procesó los eventos pendientes y produjo 4 indicadores únicos. La cifra no indica pérdida de datos: la restricción UNIQUE (type, value) de iocs colapsa en una fila cada valor repetido, y las cinco sesiones compartieron la misma IP y las mismas tres credenciales. La consecuencia es central para P3: cada indicador conserva un único event_id —el de su primera observación—, de modo que la tabla registra qué valores se observaron, pero no en cuántas ni en cuáles sesiones; aquí los cuatro quedaron atribuidos a la sesión e355bc9ba0d1 (Tabla VII-1). La deduplicación es deseable para un catálogo y, a la vez, impide medir la cobertura por sesión con la tabla tal como está diseñada (§6.9).

6.8.2 Distribución por tipo

De los 4 indicadores, 1 es de tipo ip y 3 de tipo credential. No se generaron indicadores de tipo hash, porque ningún comando del guion descargó archivos, ni de tipo url o domain, que el extractor no implementa (solo ip, credential, command y hash). En las validaciones W1 y W2, iocs terminó con 21 indicadores: 20 credenciales y una dirección IP. Hasta W2 ningún comando del simulador cumplió el patrón de descarga o ejecución, por lo que esos tipos no se ejercitaron; en C1 el simulador agregó comandos benignos que sí lo cumplen y el extractor generó 4 indicadores de tipo comando (§6.2.5).

6.8.3 Geolocalización

El nodo Code de event-ingest consulta el país de la IP de origen a ip-api.com y guarda el resultado en events.country (valor «Desconocido» si la consulta falla). Hasta el 29/09/2026 usaba fetch(), que el entorno de código de n8n no expone, por lo que fallaba siempre: en L0 y W1 la geolocalización no funcionó, con independencia de que la IP fuera privada. Se corrigió con this.helpers.httpRequest() y se probó con eventos sintéticos con tres IP públicas, que resolvieron al país esperado (§6.2.4). En W2 todos los eventos provienen de una dirección interna, por lo que country = «Desconocido»: la geolocalización quedó verificada en prueba, pero no ejercida por tráfico real. Se limita al país.

6.8.4 Reputación, artefactos, confianza y hallazgos

Ningún workflow consulta servicios de reputación (AbuseIPDB u otros) ni analiza artefactos: ambas capacidades son parte del diseño objetivo y no están implementadas. Los cuatro

indicadores del lote tienen confianza BAJO, el valor por defecto de la columna confidence: ningún nodo la asigna según reglas, por lo que la escala ALTO/MEDIO/BAJO no se ejercitó. El resultado relevante es de procedimiento: el pipeline convierte eventos crudos en un catálogo deduplicado, tipado y consultable.

6.9 Validación de las hipótesis operativas

P1 y P2 se evalúan sobre las 5 sesiones persistidas del lote 2 de L0 (65 eventos). P3 y P4 se contrastaron en L0, en W1 (10 sesiones con autenticación exitosa) y, de forma definitiva (§4.6), en W2 (41 sesiones). C1 (N = 15), C2 (N = 66) y C3 (N = 49) tienen reglas escritas de antemano: se presentan al final de cada hipótesis y no modifican los veredictos de W2. Los intervalos son de Wilson (1927) al 95 %; son amplios y ninguna proporción puede generalizarse fuera del laboratorio.

P1 — Reducción de tiempo. El umbral es que el tiempo por evento sea inferior al 50 % del manual. Se midió la latencia de ingesta, de la captura a la persistencia. Como referencia manual se usó primero una estimación propia de ≈30 s por evento y, desde C1, la mediana medida del triaje y la extracción: 18,44 s con dos integrantes del equipo, 10,36 s con cinco participantes externos y 11,01 s con otros diez, estos últimos con registro previo (§6.12.1). La latencia media fue de 348 ms en L0 (1,2 % de los 30 s), de 1,6 s en W2 (5,4 %), de 1,5 s en C1 (7,9 % de 18,44 s), de 1,37 s en C2 (13,2 % de 10,36 s) y de 1,70 s en C3 (16,4 % de 10,36 s): siempre por debajo del 50 %. P1 se confirma para la etapa de ingesta; la extracción y el reporte no se computan, porque en L0 se ejecutaron a mano. Es una comparación entre tareas no equivalentes que, con N = 2, 5 y 10, solo ilustra un orden de magnitud (Tabla 6.8; Anexo VII.1).

P2 — Estructuración de datos. El umbral es que al menos el 80 % de los eventos queden estructurados, es decir, insertados en events con eventid, session y timestamp no nulos (§4.9.2). En L0 lo cumplieron 65 de 65 (100 %; IC 95 % [94,4 %; 100 %]): P2 se confirma. Como control posterior, sin efecto sobre el veredicto, se comparó cada fila de events con el evento del log de Cowrie en siete campos: los 65 eventos de L0 y los 1.998 de W2 coinciden; en C1 coinciden 628 de 629 (629/629 con el criterio estricto), en C2 3.211 de 3.218 y en C3 2.382 de 2.386. Las doce diferencias son un comando con un espacio final que el comparador recorta solo del lado de la base, y no una pérdida de datos. El 100 % es en buena medida estructural, porque eventid y timestamp son NOT NULL (Tabla 6.9); el criterio estricto por tipo de evento está en el Anexo VII.2.

P3 — Generación de IoCs. (a) Resultado. El umbral es que al menos el 70 % de las sesiones con autenticación exitosa (§4.9.3) tengan un indicador atribuido por event_id, es decir, al menos una fila de iocs ligada a uno de sus eventos. El resultado fue 1 de 5 (20 %) en L0, 6 de

10 (60 %) en W1 y 6 de 41 (14,6 %; IC 95 % [6,9 %; 28,4 %]) en W2, la ventana de veredicto: P3 no se confirma. Las sesiones de W1 se reconstruyeron desde el log de Cowrie, porque esa base no se conservó.

(b) Causa. La medición dependía del esquema del catálogo, no del funcionamiento del extractor: cada fila de iocs admite un único event_id y la restricción UNIQUE (type, value) deja atribuida solo a la primera sesión que observa cada valor (§7.2.3). La tabla ioc_sessions, creada tras ver el resultado de W1, registra indicadores en 39 de las 41 sesiones de W2 (95,1 %), pero es una medición exploratoria, casi estructural, y no determina el veredicto.

(c) P3′, completitud por sesión (reformulación). P3′ exige que ioc_sessions contenga todos los indicadores que las reglas del extractor derivan del log de Cowrie de cada sesión con autenticación exitosa, con el mismo umbral del 70 %; se definió después de observar W1 (§4.9.3). La completitud fue de 15 de 15 en C1 (IC 95 % [79,6 %; 100 %]), de 66 de 66 en C2 (IC 95 % [94,5 %; 100 %]) y de 49 de 49 en C3 (IC 95 % [92,7 %; 100 %]); por event_id fue de 9 de 15, 14 de 66 y 15 de 49. La exactitud fue de 32 de 32 y de 1.108 de 1.108 en C2 y de 32 de 32 y de 817 de 817 en C3. P3′ mide la fidelidad del extractor a sus propias reglas, no el valor de la inteligencia, y no reemplaza el veredicto de P3.

(d) Lección de diseño. Una métrica por sesión exige una relación indicador–sesión en el modelo de datos desde el inicio. El detalle está en el Anexo VII.3.

P4 — Reportes automatizados. El umbral es que al menos el 90 % de las sesiones con autenticación exitosa queden cubiertas por reportes generados en modo trigger (§4.9.4). En L0 no se observó ningún disparo programado: 0 de 5 en modo trigger, aunque el reporte manual cubrió las 5 sesiones; el resultado fue «no verificada», no «refutada». En W1, con un disparador de 2 horas, se cubrieron 9 de 10 sesiones (90 %; IC 95 % [59,6 %; 98,2 %]); con el cierre previsto de 6 h habrían sido 5 de 9, y por esa sensibilidad al cierre W1 es solo antecedente. En W2 se cubrieron 35 de 41 (85,4 %; IC 95 % [71,6 %; 93,1 %]) y P4 no se confirma: las seis sesiones sin cobertura son posteriores al último disparo (07:00 UTC) y no reflejan una falla de ejecución, y las cuatro ejecuciones programadas se produjeron sin intervención manual. Como cada reporte abarca las 24 horas previas, la cobertura queda garantizada por construcción una vez que ocurre cualquier disparo posterior: así ocurrió en C1 (15 de 15), en C2 (66 de 66) y en C3 (49 de 49), que verifican que el disparo programado ocurre sin intervención manual, no el cumplimiento del umbral, y no modifican el veredicto (Anexo VII.4).

6.10 Cumplimiento de objetivos

Tabla 6.8: Cumplimiento de los objetivos específicos.

Objetivo específico	Estado	Evidencia
1. Analizar el marco conceptual de honeypots, deception security e inteligencia de amenazas	Cumplido	Capítulos II y III
2. Diseñar una arquitectura de red segmentada e implementar el prototipo en un entorno contenerizado sin exposición a Internet	Cumplido (segmentación implementada después de las ventanas de veredicto; validada en C3)	Segmentación con tres redes Docker implementada después de C1 y de las ventanas de veredicto (C2 la ejercitó) y validada en su configuración final en C3, con las tres redes internal; entorno sin conexiones entrantes desde Internet (§5.5). La VLAN con firewall es diseño objetivo (Figura 5.2) y no forma parte del texto del objetivo
3. Implementar un honeypot de media interacción orientado a capturar intentos de acceso, credenciales y comandos ejecutados tras la autenticación	Cumplido	Cowrie (SSH y Telnet) desplegado y validado en seis validaciones (el lote L0 y las ventanas W1, W2, C1, C2 y C3) y en R1, con captura de intentos de acceso, credenciales y comandos posteriores a la autenticación (§5.4). Dionaea queda como diseño objetivo (§1.5)
4. Integrar los eventos con flujos automatizados en n8n	Parcialmente cumplido	Ingesta automática 65/65 (L0) y sin pérdidas en W2, C1, C2 y C3; extracción y reporte en modo trigger en W1 y W2 (W2: 33/33 ejecuciones del extractor sin error; W1: 23 de 30 con error en disparos sin eventos pendientes, sin pérdida de datos, defecto corregido el 29/09; Tabla 6.2, Anexo VI.1). La atribución de indicadores por sesión (P3, 6/41) y la cobertura por reportes (P4, 35/41) no alcanzan sus umbrales (§6.9)
5. Incorporar al pipeline un proceso de enriquecimiento contextual de los eventos mediante geolocalización de la IP de origen y verificar su funcionamiento	Cumplido (verificado en prueba con IP públicas; no ejercido con tráfico real)	Geolocalización por país corregida y verificada en prueba con IP públicas (§6.8.3); no se ejerció con tráfico real, porque todo el tráfico de las validaciones fue interno. La reputación de IP y el análisis de artefactos son diseño objetivo (§1.5)
6. Evaluar la utilidad operativa mediante métricas	Parcialmente cumplido	§6.3 (N = 5, N = 10 y N = 41): se midieron procesamiento, estructuración, extracción y cobertura de reportes; el reporte se evaluó con cinco participantes ajenos al equipo, no con analistas, y la utilidad percibida (3,0 sobre 5) quedó por debajo del umbral de 4,0
7. Elaborar recomendaciones para replicabilidad institucional	Cumplido	§8.4 y Capítulo IX
Objetivo general. Diseñar e implementar una arquitectura de honeypots integrada con n8n para automatizar la recolección, el procesamiento y la generación de indicadores y reportes de amenazas, verificados mediante prueba funcional controlada	Parcialmente cumplido	Recolección, procesamiento y emisión programada de reportes verificados (§6.9); no se demuestra inteligencia con valor operativo: P3 y P4 no se confirman, utilidad percibida del reporte 3,0/5 (< 4,0; §7.7.3) e indicadores de ataques simulados o reproducidos por los autores

Nota. Elaboración propia.

Con la redacción reformulada de los objetivos 3 y 5 (§1.4.2), se cumplen cinco objetivos específicos (1, 2, 3, 5 y 7) y dos se cumplen solo en parte (4 y 6). El objetivo 2 pasa a

«Cumplido» porque la segmentación con tres redes Docker internal se implementó, después de las ventanas de veredicto, y C3 la validó; la VLAN con firewall es diseño objetivo y no figura en el texto del objetivo. El objetivo 3 se evalúa con seis validaciones de Cowrie (L0, W1, W2, C1, C2 y C3), y el objetivo 5 se verificó en prueba con IP públicas sin ejercerse con tráfico real.

El objetivo general se cumple solo parcialmente. Se verificaron la recolección, el procesamiento y la emisión programada de reportes (esta con el disparador de validación de 2 horas; §6.9), pero no se demuestra inteligencia con valor operativo: P3 y P4 no se confirman, el reporte obtuvo una utilidad percibida de 3,0 sobre 5 (por debajo del umbral de 4,0; §7.7.3) y los indicadores provienen de ataques simulados o de sesiones reales reproducidas por los autores. C1, C2 y C3 alcanzaron los umbrales de P3′ y de P4 —esta por construcción—, pero no modifican los veredictos de W2.

6.11 Limitaciones de los resultados y amenazas a la validez

Tabla 6.9: Limitaciones de los resultados y amenazas a la validez.

Limitación	Tipo	Efecto sobre los resultados
Ataque 100 % simulado por script desde una única IP interna, con guion idéntico en L0 y credenciales variadas de un conjunto finito (6 cuentas válidas en W1 y W2; 12 en C1, C2 y C3). En R1, comportamiento real de un dataset público, reproducido por los autores	Validez externa	Los resultados validan el funcionamiento del pipeline, no su desempeño frente a amenazas reales; no se infiere comportamiento de atacantes. R1 mitiga en parte la variabilidad del estímulo: reprodujo comportamiento real de un dataset público, pero desde la misma IP interna (§6.13)
N = 5 sesiones (réplicas del mismo guion) en L0; N = 10 en W1; N = 41 en W2; N = 15 en C1; N = 66 en C2 y N = 49 en C3	Validez de conclusión estadística	Intervalos amplios; la lectura por valor de P3 no discrimina
Disparo programado observado con el cron de validación (cada 2 horas) en W1 y W2 y con una expresión diaria adelantada en C1. El de producción se observó dos veces: el 06/10/2026 (C2, 14 h) y el 09/10/2026 (C3, 12 h)	Validez de constructo (P4)	P4 igualó el umbral en W1 y no en W2: depende de cuántas sesiones quedan después del último disparo de la ventana. C1, C2 y C3 se cumplen por construcción (§6.9)
Esquema de iocs orientado a catálogo: UNIQUE (type, value) y un único event_id por indicador	Validez de constructo (P3)	Con la operacionalización del §4.9.3, los IoCs quedan atribuidos solo a la primera sesión que los observó
Medición de P3 sobre ioc_sessions definida después de observar el 60 % de W1, y casi estructural	Validez interna y de constructo (P3)	Se informa como exploratoria y no determina el veredicto: toda sesión procesada aporta su IP y su credencial, y la medición refleja cobertura de extracción, no calidad ni novedad de la inteligencia (§6.9)
Manejo de errores parcial en las validaciones: solo el upsert de ioc-extractor escribía en error_log	Validez interna	Una falla como la del lote 1 (workflow de ingesta inactivo) no quedaba registrada; desde el 05/10/2026 la detecta health-monitor y desde el 06/10 la notifica por Telegram (§5.6.4)
Imágenes de Cowrie y n8n fijadas por digest, con digests distintos de los registrados en la BITACORA el 11/08/2026	Reproducibilidad	La discrepancia entre ambos registros de digests está documentada en la BITACORA; una replicación debe usar los digests de docker-compose.yml

Limitación	Tipo	Efecto sobre los resultados
Correcciones previas a cada ventana y dos ventanas descartadas (27/09 y 29/09; Anexo VI)	Validez interna (historia)	Ninguna ventana válida tuvo cambios de versión ni de configuración durante su ejecución; la ampliación de cuentas y la tabla ioc_sessions se decidieron a la vista de datos de P3 (ajustes posteriores a la observación). No hubo pérdida de datos
Regla de cierre de ventana no prefijada	Validez interna	Las ventanas W1 y W2 se cerraron a las 6–8 h, sin hora fija, y P4 es sensible al cierre (§6.2.4); el veredicto usa W2, donde ninguno de los dos cierres alcanza el umbral. C1, C2 y C3 se cerraron a la hora fijada en su registro previo
Línea de base manual medida con dos integrantes del equipo autor y quince participantes externos (cinco en B1 y diez en B1c)	Validez de constructo (P1)	N = 2 integrantes del equipo y N = 5 + 10 externos, con 20 eventos. Los externos usaron de forma consecutiva el equipo de un autor, y los cinco primeros recibieron antes su explicación del flujo de trabajo. La estimación previa de ≈30 s se conserva como referencia (§6.12.1)
Criterio de P2 cercano a las restricciones del esquema (eventid y timestamp son NOT NULL)	Validez de constructo (P2)	El 100 % podía alcanzarse en buena parte por construcción. Se mitigó con un control posterior estricto por tipo de evento y comparación campo a campo con el log de Cowrie (§6.9), sin modificar el veredicto
Comparación no equivalente en P1	Validez de constructo (P1)	La línea de base manual (≈30 s) abarca cinco fases (Tabla VIII-1) y la medición automática solo la ingesta; la extracción y el reporte se cronometraron por ejecución y no por evento (§6.6). P1 se confirma solo para la ingesta (§6.9, Anexo VII.1); la comparación completa requiere medir ioc-extractor por evento y contar con analistas ajenos al equipo
Reporte id = 1 preexistente de origen no documentado	Trazabilidad	Excluido de los cómputos
Automatización asistida por IA en la operación del laboratorio	Validez interna	Declarada en el §5.3 y auditada en el Anexo V.4: las cifras vigentes parten de tablas vacías y se concilian contra el log de Cowrie

Nota. Elaboración propia.

6.12 Comparación con el procesamiento manual

El valor de la automatización en seguridad solo puede establecerse mediante una comparación cuantitativa contra la alternativa que reemplaza: el procesamiento manual de eventos. Este análisis la descompone en cinco dimensiones, una por apartado, y complementa los veredictos (§6.9), que no dependen de ella.

6.12.1 Línea de base de procesamiento manual

Para establecer un término de comparación, se descompuso el procesamiento manual de un evento de honeypot en cinco fases, siguiendo el ciclo de gestión de incidentes de NIST SP 800-61 Rev. 2 (Cichonski et al., 2012), en su fase «Detección y Análisis» (§3.2); la Rev. 3 (Nelson et al., 2025) la reemplazó, y aquí se usa la Rev. 2 como referencia del ciclo clásico. Los rangos de tiempo de cada fase son una estimación de la experiencia del equipo. Los supuestos consideran un analista con formación en ciberseguridad (perfil SOC nivel 1-2), sin especialización en threat intelligence, que procesa los eventos individualmente, sin batching, en jornadas de 8 horas, sin herramientas de automatización y con acceso a los logs crudos JSON de Cowrie. La Tabla VIII-1 (Anexo VIII) presenta la descomposición completa por fase, con los rangos de tiempo estimados y su justificación.

Las fases 1 y 2 de la tabla (≈10,5 s en sus puntos medios) son las comparables con la ingesta medida; las fases 3 a 5 corresponden a tareas que ejecutan ioc-extractor y report-generator, cuyas ejecuciones de W2 duraron en promedio 1.117,85 ms (por corrida de hasta 500 eventos) y 715,25 ms (§6.6). Procesar a mano los 65 eventos del lote L0 requeriría ≈32,5 minutos-hombre (65 × 30 s), un volumen que puede resolverse manualmente. La comparación de P1 enfrenta dos magnitudes no equivalentes —la latencia de ingesta de una máquina y el tiempo que una persona tarda en triar un evento y extraer sus campos—, por lo que ilustra un orden de magnitud y no mide un ahorro de trabajo analítico; la ventaja por escala queda como hipótesis a verificar con tráfico real (§8.4).

Medición de la línea de base. Para contrastar la estimación se midieron el triaje y la extracción de campos (fases 1 y 2) sobre 20 eventos reales de la validación W2, en una página de cronometraje que muestra el JSON crudo. Dos integrantes del equipo los midieron en C1, con una mediana por evento de 18,44 s (media 29,94 s, elevada por unos pocos eventos muy lentos). Como medición complementaria no registrada de antemano, cinco participantes ajenos al equipo la repitieron el 05/10/2026, con una mediana de 10,36 s (medianas individuales de 14,06 a 9,11 s), y una ampliación posterior con registro previo (B1c) sumó diez participantes más, con una mediana de 11,01 s (intervalo bootstrap del 95 %, calculado después, de 10,44 a 11,93 s); con los quince externos la mediana es de 10,92 s. El paso 1 fue correcto en 196 de 200 eventos y no hubo errores en los campos extraídos. La medición del equipo queda por encima de la estimación previa de las fases comparables (≈10,5 s), por lo que la Tabla VIII-1 subestimaba el trabajo manual, y la de los externos queda en su orden de magnitud; los externos usaron de forma consecutiva el equipo de un autor, que les explicó antes el flujo, y sus medianas individuales tienden a descender, lo que sugiere aprendizaje. Con N = 2, N = 5 y N = 10 son valores descriptivos, no admiten inferencia y no representan el tiempo de un analista (§7.4). Las medianas individuales, el piloto descartado y la clasificación de severidad (fases 4 y 5) están en el Anexo VIII.

6.12.2 Tabla comparativa de métricas

La Tabla 6.10 contrasta las métricas del sistema automatizado contra las estimaciones manuales. Las fórmulas de mejora se especifican en las notas al pie.

Tabla 6.10: Comparación de métricas entre procesamiento manual estimado y sistema automatizado.

Métrica	Manual (medida y estimada)	Automatizado (real)	Reducción relativa (indicativa)	Notas
Tiempo de procesamiento por evento	18,44 s medidos (fases 1–2); ≈30 s estimados (fases 1–5)	1.453 ms (C1); 1.609 ms (W2); 348 ms (L0) [1]	92,1 %; 91,3 %; 98,1 % (frente a 18,44 s) [a]	Tareas no equivalentes (§6.11)
Capacidad de procesamiento pura	≈1.560 eventos/día (28.800 s ÷ 18,44 s); ≈960 con la estimación [2]	≈19.800 eventos/día como cota inferior secuencial (28.800 s ÷ 1,453 s) [3]; ≥ 28 eventos/s sostenidos en la prueba de carga, ≈806.400 por jornada de 8 h (28 × 28.800)	≈12,7× (cota secuencial)	Cota secuencial por latencia media; la capacidad la mide la prueba de carga (§6.12.3)
Capacidad efectiva con enriquecimiento	≈960 eventos/día [2]	No medida [4]	n/a	Solo hay consulta de país (ip-api.com, 45 consultas/min según el proveedor); no se midió

Notas.

[1] Medición principal: consulta V1 (§6.3.5) sobre los 65 eventos persistidos del lote L0, delta created_at − timestamp (media 348,02 ms; mediana 330,56 ms; mínimo 100,16 ms; máximo 798,34 ms; desviación estándar muestral 160,08 ms; Anexo III, Tabla III-1b). La sesión del 10–11/08/2026 (Anexo III, Tabla III-1) se cita como referencia complementaria.

[2] Cálculo: con la línea de base medida, 28.800 s ÷ 18,44 s ≈ 1.560 eventos/día; con la estimación, 480 min ÷ 0,5 min/evento = 960.

[3] Estimación a partir de la latencia media: 28.800 s ÷ 1,453 s/evento ≈ 19.800 eventos/día en jornada de 8 h (C1, con la consulta de país operativa); con la media de W2 (1,609 s), ≈17.900. Supone procesamiento secuencial; la prueba de carga del 05/10/2026, sin la consulta de país, sostuvo al menos 28 eventos por segundo (§6.12.3), y la cifra de 19.800 se conserva solo para comparar con la estimación manual.

[4] No medida: la reputación IP no está implementada y la capacidad con consulta de país depende del límite de ip-api.com para IP distintas (según la documentación del proveedor).

[a] Reducción relativa calculada como (valor_manual − valor_automático) / valor_manual × 100 (menor es mejor). Es una comparación indicativa entre tareas no equivalentes (§6.11).

Nota: esta comparación se basa en una línea de base manual estimada a partir de la estructura del ciclo de manejo de incidentes de NIST SP 800-61 Rev. 2 (Cichonski et al., 2012) y de la experiencia propia del equipo (Tabla VIII-1). No proviene de un experimento controlado con grupo de analistas ni de cifras publicadas por SANS, IBM Security, Verizon o ENISA; los valores porcentuales son una aproximación conservadora, y la línea de base se midió después (§6.12.1).

6.12.3 Análisis de escalabilidad

La escalabilidad del pipeline automatizado se evaluó con una proyección frente al esfuerzo manual equivalente (§6.12.1, Tabla 6.10) y con una prueba de carga local. Dos restricciones

podrían afectar el rendimiento: la geolocalización, a partir de ≈65.000 IP únicas por día, y la ingesta, a partir de unos 20 eventos por segundo, donde la latencia mediana empezó a crecer en la prueba de carga.

1. APIs de geolocalización (rate limiting). El nodo Code de event-ingest consulta ip-api.com, con un límite de 45 consultas por minuto en su capa gratuita, según la documentación del proveedor, no verificada aquí. Con ≈65.000 eventos por día, con IPs mayoritariamente únicas, habría demoras por throttling. Mitigación: caché local de consultas geo con TTL de 24 horas o plan comercial (≈$10-15 USD/mes).

2. Capacidad de ingesta. Una prueba de carga local, con plan fijado de antemano, envió al webhook seis escalones de 120 eventos sintéticos, de 1 a 50 eventos por segundo, con la consulta de país anulada. Se persistieron los 720 eventos. La mediana de latencia fue de 35 a 45 ms hasta 10 eventos por segundo y de 125 ms a 20; a 28,1, la tasa máxima que alcanzó el generador, fue de 1,44 s (p95 de 1,92 s). Con el criterio registrado (100 % persistido y p95 de hasta 5 s), la capacidad sostenida es de al menos 28 eventos por segundo (≈100.000 por hora), sin que el pipeline llegara a saturarse. Mitigación: concurrency control de n8n o fragmentación en workflows con colas intermedias.

6.12.4 Análisis de precisión

La precisión del parseo automatizado se contrastó con tasas de error estimadas por el equipo para el análisis manual, no tomadas de la literatura (desglose en la Tabla III-6, Anexo III). Los 65 eventos del lote 2 de L0 se insertaron sin errores de estructura, y no se observaron payloads malformados ni campos obligatorios faltantes, porque todo el tráfico provino del simulador. En el lote 1 se perdieron los 65 eventos por la inactividad del workflow de ingesta (§6.4), una falla de operación y no de parseo que el pipeline no detecta porque event-ingest carece de manejo de errores. El rango estimado para el procesamiento manual no proviene de una fuente externa verificable, y la línea de base medida no lo corrobora: los participantes no cometieron errores de extracción (§6.12.1).

6.12.5 Verificación de criterios de éxito

Los criterios 4.9.1 a 4.9.4 corresponden a P1 a P4 y se verifican en el §6.9: P1 y P2 se confirmaron, y P3 y P4 no. El criterio 4.9.5, cualitativo y sin umbral, se cumple parcialmente. Se identificaron credenciales débiles reiteradas y una secuencia de reconocimiento posexplotación (§6.8). Ambas son patrones del guion del simulador: validan el extractor y no la identificación de TTPs en tráfico real. Formalizar esa identificación exigiría, por ejemplo, un pipeline de aprendizaje automático.

6.13 Validación ampliada: R1 y réplica C

Este apartado reúne dos validaciones que no entran en los veredictos: R1, con comportamiento de atacantes reales tomado de un dataset público, y la réplica independiente C. Ninguna modifica los veredictos de P1 a P4, que siguen siendo los de L0 y W2 (Tabla 8.1).

6.13.1 R1: muestra y procedimiento

R1 responde si el pipeline captura, estructura e indexa bien el comportamiento de atacantes reales y no solo el guion de los autores. Su estímulo son 200 sesiones del período 2024 del dataset público de Wang et al. (2025), elegidas con una semilla fija entre 5.540 sesiones elegibles de 6.658 (de 1 a 30 comandos, sin caracteres de control y con credenciales de formato simple). Una sesión del dataset reúne los comandos únicos de un origen durante el período. La muestra suma 1.020 comandos, de 1 a 28 por sesión.

Un reproductor (attack-runner/replay_dataset.py) se conecta por SSH a Cowrie desde la red de captura, se autentica con las credenciales del primer login exitoso de cada origen en el registro crudo del dataset y escribe los comandos en el orden original, con la demora original acotada entre 0,3 y 5 s (813 de los 1.020 comandos tienen demora original; los demás, 1 s) y esperando el prompt tras cada uno. Su registro de lo enviado es la verdad de referencia. Reprodujo 10 sesiones cada 15 minutos entre las 21:30 y las 02:15 ART, con el disparo de producción de las 08:00 dentro de la ventana, que duró 19 h 11 min y cerró a las 08:15 ART del 11/10/2026. Cowrie no tiene salida a Internet, por lo que los wget y curl del dataset fallaron y solo quedaron registrados. La configuración fue la de C3, con las cuentas de la muestra agregadas a userdb.txt. El registro previo (docs/PREREGISTRO_R1.md, tags Honeypot_R1_prereg y Honeypot_R1_prereg2) se publicó antes de la primera sesión; la única corrección anterior al inicio, en el script de inicio, se declara en su sección 9.

6.13.2 R1: resultados frente a los criterios registrados

La Tabla 6.11 resume los criterios de R1, fijados antes de iniciar, y sus resultados. Todos los que tienen umbral se cumplieron.

Tabla 6.11: R1: criterios registrados de antemano y resultados.

Criterio	Umbral registrado	Resultado	Cumple
Captura: comandos enviados que aparecen como cowrie.command.input en events	≥ 95 %	1.018/1.020 = 99,8 % (IC 95 % [99,3 %; 99,9 %]); igual contra el log de Cowrie	Sí
Ingesta: eventos de Cowrie persistidos en events	100 %, sin claves sin par	3.110/3.110 = 100 % (IC 95 % [99,9 %; 100 %]); 0 claves sin par	Sí
P2 estricto: criterio por tipo y fidelidad campo a campo con el log de Cowrie	Ambas ≥ 80 %	3.110/3.110 y 3.110/3.110 (100 %; IC 95 % [99,9 %; 100 %])	Sí
P3′: completitud por sesión	≥ 70 %	200/200 = 100 % (IC 95 % [98,1 %; 100 %])	Sí
Indicadores de tipo comando: precisión y exhaustividad contra la verdad de referencia	Sin umbral	Catálogo: 133/133 y 133/133 (IC 95 % [97,2 %; 100 %]). Por sesión: 285/285 y 285/285 (IC 95 % [98,7 %; 100 %])	—

Criterio	Umbral registrado	Resultado	Cumple
P4: disparo de producción de las 08:00 ART	Sin umbral en R1	Una ejecución en modo trigger, con éxito, a las 11:00:00 UTC; reporte guardado; cobertura 200/200 (por construcción)	—

Nota. Intervalos de Wilson al 95 %. Los criterios y los umbrales se fijaron en el registro previo de R1 (docs/PREREGISTRO_R1.md), antes de la primera sesión; los resultados son los del script registrado (docs/evidencia/r1/resultados_r1.json). La verdad de referencia usa el patrón del propio extractor. Elaboración propia.

Las 200 sesiones produjeron 294 indicadores (1 IP, 160 credenciales y 133 comandos) y 685 filas en ioc_sessions. La latencia de persistencia fue de 1,61 s de media (máximo, 2,94 s). El extractor corrió 76 veces y health-monitor, 230, sin errores ni alertas, sin ejecuciones manuales y con error_log vacío; el forwarder hizo 3.110 entregas sin errores y Telegram envió 200 alertas de conexión y 200 resúmenes, sin rechazos. Al cierre, los seis servicios corrían con los puertos y las redes de la configuración final. Con la operacionalización original de P3 (por event_id), 168 de 200 sesiones (84,0 %) tienen una fila de iocs atribuida; es un valor descriptivo, porque el veredicto de P3 es el de W2.

Frente a las técnicas ATT&CK que el dataset asigna a las sesiones de la muestra (el pipeline no las etiqueta), las más frecuentes son T1070 Indicator Removal (159 sesiones), T1222 File and Directory Permissions Modification (139) y T1098 Account Manipulation (139); todas las sesiones con esas tres técnicas tienen al menos un indicador de tipo comando, mientras que solo 23 de las 36 con T1087 Account Discovery lo tienen. De los 1.020 comandos enviados, 735 no cumplen el patrón del extractor (cd, 139; uname, 77; cat, 65, entre otros, y 37 de cada uno de sh, shell, enable y system), por lo que no generan indicador de comando.

Los dos comandos no capturados son de una misma sesión que había enviado passwd: la emulación de Cowrie lo deja esperando una contraseña, y las dos líneas siguientes quedaron en el log como entradas de ese diálogo (cowrie.command.success) y no como cowrie.command.input. Este diagnóstico es posterior al registro previo.

6.13.3 Limitaciones de R1

El comportamiento es real, pero lo reproducen los autores: la conexión, el orden de las sesiones, la temporización acotada a 5 s y la espera del prompt son decisiones del reproductor. El origen sigue siendo una única dirección interna, por lo que no se ejerce la geolocalización ni la diversidad de orígenes. Las descargas fallan por diseño, porque Cowrie no tiene salida a Internet: no hay archivos descargados ni hashes. Las repeticiones de un mismo comando no se reproducen y las sesiones de Telnet del dataset se reproducen por SSH. Las credenciales se filtraron por formato (1.000 de 6.658 sesiones quedaron fuera), de modo que la muestra no es aleatoria sobre todos los ataques de 2024. La precisión y la exhaustividad de los indicadores de comando se miden contra el patrón del propio extractor, y R1 comparte arquitectura y autores con las demás validaciones.

6.13.4 Réplica independiente C

Una persona ajena al equipo y a la dirección del trabajo, que no participó en el desarrollo ni leyó la tesis, hizo la réplica el 11/10/2026 siguiendo solo el README del tag de código (Honeypot_Final_2026-10-08g), según el protocolo preparado de antemano (docs/

PROTOCOLO_REPLICA.md). Usó un equipo propio sin contenedores ni imágenes del proyecto (Windows 11 Pro, 63,78 GB de RAM, Docker 29.8.2 y Compose v5.5.1, con experiencia previa intermedia en Docker), clonó el repositorio en ese tag (árbol limpio, userdb.txt del tag) y se ayudó con una IA para automatizar la ejecución de los comandos del README. El equipo no intervino en ningún momento.

Resultado. Tras una corrida del simulador de 45,8 s (16 sesiones, 0 errores), los eventos de Cowrie de la corrida fueron 102 (140 líneas finales menos 38 de base) y las filas nuevas de events, 102: resultado exitoso según el criterio del protocolo. Desde el clon hasta el conteo final pasaron 58 min 44 s. La persona señaló dos dificultades, resueltas en segundos: la imagen de Cowrie no incluye shell, por lo que contó las líneas con docker cp y python3, y el simulador ataca al levantar el stack, por lo que la línea base no es cero (38 líneas de Cowrie sin persistir). Propuso agregar al README un fragmento que automatice la comparación, la advertencia sobre la línea base y la ruta del log. Su registro está en docs/evidencia/rep/ (tag Honeypot_Rep_evidencia).

Límites. Es una sola réplica, con una persona y un equipo, y mide el arranque y la paridad de eventos, no los veredictos. La IA ejecutó los comandos, por lo que no mide a una persona sin asistencia. No se registraron los tiempos del primer arranque del stack ni el detalle de 43 min entre el clon y el archivo .env. El simulador informó 3 inicios de sesión correctos de 16 y no se pudo verificar la cuenta del tercero: el registro lo atribuyó a una cuenta de R1, pero el userdb.txt del tag no la contiene, y la salida del simulador no se conservó. No afecta a la comparación de eventos, que cuenta todos los eventos de Cowrie. La réplica no modifica ningún veredicto.

Capítulo VII: Discusión

Este capítulo interpreta los resultados del capítulo anterior en el contexto del problema planteado y de los objetivos de la tesis.

7.1 Interpretación de la comparación con el procesamiento manual

Los resultados confirman que la integración reduce el tiempo de la etapa de ingesta frente al triaje manual (§6.12.1) y estructura los eventos sin errores (P1 y P2); P3 y P4 no se confirman en la ventana de veredicto, y los contrastes con reglas escritas de antemano no modifican esos veredictos. La automatización low-code resulta así una alternativa viable para capturar y normalizar eventos en organizaciones con recursos limitados; la capacidad efectiva con enriquecimiento no se midió.

7.2 Interpretación de resultados

7.2.1 Análisis cuantitativo de eventos

Más allá del veredicto de cada hipótesis (§6.9), los valores admiten cuatro lecturas. Primero, la actividad se concentra en intervalos breves y regulares (Tabla 6.3), propios del escenario simulado; para la planificación defensiva conviene priorizar la detección de autenticaciones exitosas con credenciales débiles por sobre el volumen bruto de eventos. Segundo, el 100 % de eventos Telnet del lote y de SSH en las ventanas automáticas refleja el servicio atacado por el simulador, no la prevalencia de cada protocolo en Internet, y las credenciales triviales del guion (Tabla 6.4) ilustran los accesos que contraseñas robustas y MFA bloquearían. Tercero, la geolocalización no aporta información, porque el origen siempre fue una dirección privada del laboratorio (§6.5.2). Cuarto, la prueba de carga local sostuvo al menos 28 eventos por segundo sin pérdidas, con la consulta de país anulada.

7.2.2 Análisis cualitativo de comportamiento

En las cinco sesiones del lote L0 los intervalos entre comandos son regulares (10–11 s) y la espera entre la autenticación y el primer comando es mayor (≈32 s; §6.3.3). Esa regularidad casi exacta es incompatible con la variabilidad de una persona tecleando en tiempo real: el comportamiento observado es el del script del equipo, no el de un atacante.

7.2.3 Poder discriminante de las hipótesis y lección de diseño

Poder discriminante de P1 y P2. Las dos hipótesis confirmadas aportan menos evidencia de la que sugiere su resultado, y funcionan más como criterios de aceptación de ingeniería que como

hipótesis de investigación. P1 compara tareas no equivalentes —la ingesta automática de un evento frente al triaje y la extracción manuales— y su umbral del 50 % se cumple casi por construcción; P2, en su definición original, depende en buena parte del esquema de la base, porque eventid y timestamp son NOT NULL (§6.9). Las hipótesis que podían fallar, P3 y P4, no se confirmaron; una formulación más exigente se propone en el §8.4.

Lección de diseño: catálogo deduplicado y cobertura por sesión. El fracaso de P3 no revela una propiedad del fenómeno estudiado sino un defecto del diseño, que pudo preverse: la restricción UNIQUE (type, value) de la tabla iocs conserva un único event_id por indicador, de modo que cada valor queda atribuido solo a la primera sesión que lo observó (§6.9; Anexo VII), aunque la información no se pierde. La cobertura por sesión debió medirse desde el principio sobre una relación indicador–sesión, de muchos a muchos, que el esquema original no representaba; la tabla ioc_sessions, incorporada después, permite medirla pero no sustituye al criterio fijado de antemano (§4.9.3). No se corrigió el esquema para repetir la ventana de veredicto porque el criterio de P3 se fijó de antemano y cambiarlo después de ver los datos invalidaría el contraste; por eso la completitud por sesión se informa aparte como P3′, una reformulación definida tras W1 y aplicada en C1 a C3 que no modifica el veredicto de P3.

7.3 Comparación con literatura existente

Los resultados son consistentes con lo que señala la literatura —escaneo automatizado constante, uso de credenciales comunes y ataques oportunistas—, con tres matices. El volumen no es comparable con el de sensores expuestos a tráfico hostil real (Nawrocki et al., 2016); la diferencia refleja el diseño de la validación (§4.4), no una limitación del pipeline. En despliegues expuestos el puerto de SSH (22) recibe actividad de ataque sostenida (SANS Internet Storm Center, s.f.), por lo que un sensor real registraría SSH, que el prototipo expone; también expone Telnet (2323 en el host), cuya actividad real no se cita. Respecto de la automatización, el extractor operó en modo trigger en las ventanas automáticas, pero en la ventana de veredicto no alcanzó la cobertura por sesión del §4.9.3; esa orientación coincide con la encuesta de Shackleford (2015), que plantea la necesidad de procesos más automatizados que vinculen la detección con la respuesta. Frente a Modern Honey Network, la arquitectura agrega procesamiento automatizado de eventos y un enriquecimiento mínimo (país) que no se comparó con el de MHN.

Frente a T-Pot, la plataforma vigente más completa (más de veinte honeypots, visualización con el Elastic Stack y mapas de ataques, desplegada con Docker; Deutsche Telekom Security GmbH, 2023), el prototipo es mucho más modesto: un solo honeypot, sin panel de visualización y con un enriquecimiento mínimo. T-Pot concentra la captura y la visualización,

mientras que aquí los eventos se normalizan en un modelo propio del que se derivan indicadores y reportes con flujos editables en n8n. No se comparó con T-Pot en la práctica (Tabla III-13) y no se afirma que el prototipo lo supere.

7.4 Limitaciones del estudio

Los resultados deben interpretarse a la luz de nueve limitaciones, organizadas por dimensión en la Tabla 7.1 con su impacto y su mitigación; la primera, el volumen, condiciona todo lo demás. Ninguna invalida las conclusiones dentro del alcance definido, pero todas condicionan su generalización. Las limitaciones operativas específicas de la validación están en la Tabla 6.9 (§6.11).

Tabla 7.1: Limitaciones del estudio: impacto y mitigación.

Limitación	Impacto	Mitigación
De volumen: de 5 a 66 sesiones simuladas por validación (Tabla 6.9), todas desde una única IP interna y con un conjunto finito de credenciales	No permite análisis estadístico ni patrones temporales de mediano plazo; la literatura reporta picos de frecuencia de ataque en otros servicios, por ejemplo diarios y semanales en el spam de mensajería instantánea (Nawrocki et al., 2016)	Una ventana de observación con tráfico real queda fuera del alcance de este trabajo
Metodológica: línea de base manual con pocos participantes (Tabla 6.9)	Referencia descriptiva sobre 20 eventos y sin la fase de enriquecimiento (§6.12.1)	La estimación previa (23–36 s por evento, Tabla VIII-1), de elaboración propia, se conserva como referencia; una medición con analistas en sus propios equipos y con más eventos establecería la comparación directa
Técnica: dependencia de APIs gratuitas con rate limiting (ip-api.com: 45 consultas/min, según la documentación del proveedor, no verificada aquí)	A volúmenes elevados con IPs únicas habría demoras por throttling; el workflow no incorpora reintentos	Incorporación de reintentos y de una caché local con TTL de 24 horas (trabajo futuro; mitigación descripta en §6.12.3)
De entorno: ataque controlado, sin tráfico hostil real de Internet	La ausencia de tráfico benigno y de volumen real simplifica artificialmente el procesamiento (Tabla 6.9)	Los resultados se limitan al entorno definido; validar con tráfico real queda fuera del alcance de este trabajo
De protocolos: cobertura limitada a los protocolos de Cowrie (Telnet en L0; SSH en W1, W2, C1, C2 y C3)	Quedan fuera HTTP/HTTPS, RDP, SMB, bases de datos (MSSQL, MySQL) e IoT (MQTT, CoAP); los patrones identificados no son representativos de esos protocolos	Alcance delimitado a los protocolos emulados; la extensión de cobertura queda como trabajo futuro
De validación: ausencia de ground truth para ML	Sin etiquetas de validación es imposible calcular precisión, recall o F1-score; las discusiones sobre clustering y anomalías son hipotéticas	Etiquetado manual de sesiones (estimado: 40-80 horas de analista; estimación propia) como requisito previo

Limitación	Impacto	Mitigación
De generalización: ámbito acotado a PyMEs y educación	Resultados no generalizables a entornos enterprise (millones de eventos/día, PCI-DSS/HIPAA/GDPR, SIEM/SOAR corporativos)	Antes de adopción a gran escala se requiere evaluar el escalado horizontal, la redundancia de sensores y el cumplimiento regulatorio
De medición de P3 (Tabla 6.9)	La completitud se mide contra las reglas del propio extractor y no indica la calidad de la inteligencia	Se informa como verificación de integridad, junto con la operacionalización original (§6.9)
Origen del atacante no preservado: con cowrie-proxy, Cowrie ve la IP del proxy	El IoC de tipo ip, la geolocalización y la futura reputación pierden sentido para el tráfico que entra por el proxy.	Antes de exponer el sistema, preservar la IP real (protocolo PROXY o publicación directa de los puertos); en las validaciones el origen era una sola IP interna y la limitación no se manifestó.

Nota. Elaboración propia.

Limitación de seguridad del prototipo. Además de las nueve limitaciones de la tabla, el prototipo se endureció solo dentro del laboratorio (red segmentada, consultas parametrizadas y contenedores sin privilegios; §5.6.6) y no se evaluó frente a tráfico hostil real.

7.5 Implicaciones prácticas

Para las PyMEs, la arquitectura muestra que es posible montar en laboratorio un pipeline local de captura, procesamiento y reporte con herramientas de código abierto o gratuito y recursos mínimos, sin plataformas SOAR comerciales; que produzca inteligencia útil frente a tráfico real no se probó. Para las instituciones educativas, aporta una metodología reproducible para enseñar ciberseguridad defensiva en un laboratorio controlado, sin exposición entrante desde Internet.

7.6 Evaluación crítica de n8n como plataforma de automatización CTI

7.6.1 Ventajas observadas

Cuatro ventajas de n8n se confirmaron durante el desarrollo: 1. Curva de aprendizaje baja: el editor visual permitió construir el workflow event-ingest (3 nodos) en ≈4 horas, frente a 12-16 horas que el equipo estima para un pipeline equivalente en Python puro (estimación propia, no medida). 2. Integraciones nativas: los nodos Webhook, Code, PostgreSQL y Schedule Trigger cubren la ingesta, la transformación y la persistencia. 3. Orquestación multi-workflow: la ingesta, el procesamiento por lotes y los reportes se separaron en tres workflows sin orquestador externo, y desde W1 operaron de forma automática (§6.2.4, Tabla 6.2). 4.

Comunidad activa: más de 400 nodos disponibles (n8n GmbH, s.f.) cubren la mayoría de los escenarios CTI sin conectores personalizados.

7.6.2 Limitaciones encontradas

Cuatro limitaciones de n8n se manifestaron durante la operación del pipeline.

1. Procesamiento secuencial por defecto: la concurrencia se habilita con concurrency control, aunque su configuración no es intuitiva. En la prueba de carga la latencia mediana empezó a crecer a partir de 20 eventos por segundo, aunque se sostuvieron al menos 28 sin pérdidas (§6.12.3).

2. Debugging complejo en workflows multi-nodo: los errores en nodos intermedios no siempre identifican la causa raíz. Un error de tipado en ioc-extractor insumió ≈3 horas.

3. Consumo de recursos de Node.js: no se midió; en VPS de 1 GB de RAM n8n puede no ser viable junto con los honeypots.

4. Defecto de agregación en el reporte. En una validación anterior, en un segundo entorno (Anexo V), report-generator listó una misma IP de origen dos veces en las IP más frecuentes, con conteos distintos; la consulta versionada agrupa por src_ip y no debería repetirla, y la causa no se determinó. Requiere una corrección antes de considerarse lista para producción (Tabla 8.2).

Robustez de la ingesta. Un hallazgo más grave surgió en el segundo entorno (Anexo V): el forwarder dejó de entregar eventos a n8n en dos intervalos y, de 39 sesiones, solo 10 quedaron persistidas (≈29 perdidas), sin que el pipeline lo registrara; la misma falla silenciosa se repitió en C1, con la credencial de PostgreSQL inválida. Por eso se agregaron dos correcciones: health-monitor compara lo que Cowrie registró con lo persistido, y el forwarder guarda la posición del log y reintenta cada evento (§5.6.4), ambas probadas con pruebas fijadas de antemano. C3 observó después la configuración final sin pérdidas (2.386 de 2.386 eventos; §6.2.7), pero no cubrió la recuperación ante fallas: no se provocó una caída de n8n ni un reinicio del forwarder, por lo que el reintento y la posición guardada siguen verificados solo por esas pruebas.

7.6.3 Trade-offs frente a alternativas

Python puro (scripts, cron y Flask o FastAPI) ofrece control total y menor huella de recursos, pero exige desarrollar a mano el scheduling, los reintentos, el registro y las integraciones; Node-RED comparte la filosofía low-code, con una integración con PostgreSQL menos madura, y StackStorm, más potente en auto-remediación, resultaría excesivo para una PyME por su complejidad operativa (según la documentación, no una prueba propia; Tabla III-13 y

Tabla III-7, Anexo III). Para un equipo de seguridad sin perfil de desarrollo, n8n ofreció el mejor balance entre velocidad de implementación, integraciones y costo operativo; con capacidad de desarrollo, Python puro ofrece ventajas en escalabilidad y depuración. Una evaluación académica reciente de n8n autoalojado señala, en cambio, la curva de aprendizaje y las restricciones de la versión gratuita como limitaciones (Casaclang et al., 2026).

7.7 Calidad y utilidad de los IoCs generados

Los 4 IoCs generados en el lote L0 (§6.8) son el producto central del pipeline: se produjeron mediante una ejecución manual de ioc-extractor y tienen la estructura del esquema (tipo, valor, confianza —por defecto BAJO—, evento de origen, fuente y fecha de creación); la geolocalización, el ASN, la reputación y las etiquetas MITRE ATT&CK no forman parte del esquema implementado. Su valor operativo potencial radica en la especificidad local: a diferencia de los feeds globales (OTX, MISP defaults), un IoC del propio sensor reflejaría actividad dirigida a la infraestructura monitoreada, ventaja que aquí no se ejerce porque la única IP observada es el gateway del laboratorio y las credenciales son las del propio equipo. Con el extractor en modo trigger, iocs terminó con entre 21 y 32 indicadores únicos en las ventanas automáticas (21 en W1 y W2, 26 en C1 y 32 en C2 y C3; Tabla 6.2 y §6.2.5 a 6.2.7).

Según la Pirámide del Dolor (§3.6.4), las direcciones IP del prototipo están entre los niveles más bajos y, además, son internas; las credenciales reutilizadas y los comandos observados describen herramientas y comportamiento y, a criterio de los autores, serían los de mayor valor potencial. Con tráfico simulado, ninguno tiene hoy valor operativo.

7.7.1 Falsos positivos y ruido

Los IoCs provenientes de Cowrie (media interacción; §3.5.3.3) comparten una limitación inherente a los honeypots de interacción baja o media: capturan intentos de ataque, no necesariamente amenazas activas fuera del laboratorio. El riesgo puede mitigarse con tres estrategias, propuestas y no implementadas: umbrales de confianza, correlación temporal y filtrado por tipo de IoC. En L0 todo el tráfico es simulado y todos los indicadores son, por construcción, verdaderos, por lo que la tasa de falsos positivos no es evaluable con este corpus (Tabla 7.2). El único contexto implementado es el país de la IP de origen (§6.8.3); sin enriquecimiento, un IoC de IP tiene escaso significado operativo, y completarlo es una prioridad del trabajo futuro.

7.7.2 Comparación con IoCs de feeds públicos

Los IoCs generados por el sistema presentan ventajas y desventajas frente a los disponibles en plataformas de inteligencia colectiva (Tabla 7.2):

Tabla 7.2: Comparación de IoCs locales frente a feeds públicos de inteligencia colectiva.

Dimensión	IoCs locales (este sistema)	Feeds públicos (OTX, MISP)
Frescura	T+0 (detección en tiempo real)	Variable; sin cifra verificada en este trabajo
Especificidad	Dirigidos a infraestructura monitoreada	Genéricos, multi-organización
Contexto	Bajo en el prototipo (solo país); alto en el diseño objetivo	Variable según el publicador
Volumen	4 en el lote L0; 21 en W1 y W2, 26 en C1, 32 en C2 y 32 en C3 (gestionable)	Variable; sin cifra verificada en este trabajo (requiere triage)
Falsos positivos	No evaluable (tráfico simulado)	Variable; sin cifra verificada en este trabajo
Correlación cruzada	Solo datos locales	Correlación multi-feeding

Nota. Elaboración propia.

La ventaja esperable de los IoCs locales es la frescura y la especificidad, que esta validación, con tráfico simulado, no pudo verificar; la de los feeds públicos es la correlación cruzada, que permite saber si una IP ya fue reportada por otras organizaciones. La integración óptima es complementaria: IoCs locales para detección temprana y feeds públicos para validación y enriquecimiento.

7.7.3 Utilidad del reporte para un lector ajeno al sistema

Para el objetivo 6, los cinco participantes de B1 (E1 a E5; ajenos al equipo, no analistas) leyeron, uno por vez, el reporte diario real de la validación C2 y un fragmento de log, y respondieron preguntas con umbrales fijados antes de aplicarlas (docs/evidencia/evaluacion/). Acertaron 25 de 30 preguntas sobre el reporte (83,3 %; IC 95 % [66,4 %; 92,7 %]; umbral 80 %), con una mediana de 11,8 s (umbral 60 s), y reconocieron en 10 de 10 respuestas lo que el reporte no informa. La utilidad percibida fue de 3,0 sobre 5, por debajo del umbral de 4,0 (claridad y confianza en los números, 2,8). Las preguntas sobre el log dieron 0 de 10 con el criterio registrado. El reporte era el JSON del sistema, N = 5 no admite inferencia y el primer participante empezó antes de la publicación del plan.

Capítulo VIII: Conclusiones

8.1 Síntesis del problema abordado

Este trabajo partió de la dificultad de las organizaciones con recursos limitados para obtener inteligencia de amenazas propia y de la pregunta sobre si una arquitectura de honeypots integrada con una plataforma de automatización low-code puede generar esa inteligencia con baja intervención humana. Para responderla se construyó un prototipo compuesto por Cowrie, un servicio de reenvío de eventos, n8n y PostgreSQL, desplegado con Docker Compose con puertos publicados solo en loopback, y se lo sometió a un ataque simulado y controlado cuyos resultados se reconciliaron contra la base de datos y el log de Cowrie cuando la evidencia se conservó (§6.2.4).

8.2 Evaluación de hipótesis y contribuciones

La Tabla 8.1 consolida la verificación de cada hipótesis operativa con su umbral (§4.9), su operacionalización y el resultado observado en el lote inicial L0 (P1 y P2) y en la ventana de veredicto W2 (P3 y P4), con la ventana W1 como antecedente; los tres contrastes (C1, C2 y C3) se agregan en cada fila.

Tabla 8.1: Verificación de las hipótesis operativas frente a los resultados (P1 y P2: lote L0, N = 5 sesiones, 65 eventos; P3 y P4: validación W2, N = 41 sesiones; antecedente: W1, N = 10; contrastes con reglas escritas de antemano: C1, N = 15; C2, N = 66, y C3, N = 49).

Hipótesis (criterio §4.9)	**Umbral exigido**	**Operacionalización**	**Resultado observado**	**Veredicto**	**Fijado**
P1 — Reducción de tiempo	Menos del 50 % del tiempo manual: ≈30 s estimado (→ 15 s); 18,44 s medido para triaje y extracción (→ 9,22 s); 10,36 s con participantes externos (→ 5,18 s)	Latencia de ingesta por evento (captura → persistencia)	Media 348,02 ms por evento (lote L0, N = 65; Tabla III-1b); referencia complementaria 10–11/08: 297 ms, no verificable (Tabla III-1); W2: media 1.608,76 ms, máximo 4.922,22 ms (N = 1.998); C1: media 1.453,16 ms (7,9 % de la línea de base medida; N = 629); C2: 1.370,85 ms (13,2 % de 10,36 s; N = 3.218); C3: 1.697,89 ms (16,4 % de 10,36 s; N = 2.386)	Confirmada para la etapa de ingesta	Antes de los datos (umbral)
P2 — Estructuración de datos	≥ 80 % de los eventos	Fila en events con eventid, session y timestamp no nulos	65/65 (100 %; IC 95 % [94,4 %; 100 %]); control posterior estricto y fidelidad con el log de Cowrie: 1.998/1.998 en W2; con registro previo, 3.218/3.218 y 3.211/3.218 en C2; 2.386/2.386 y 2.382/2.386 en C3	Confirmada	Antes de los datos (umbral)
P3 — Generación de IoCs	≥ 70 % de las sesiones con login exitoso	Sesión con ≥ 1 fila en iocs atribuible por event_id	6/41 (14,6 %; IC 95 % [6,9 %; 28,4 %]) en W2; 6/10 (60 %) en W1; 1/5	No confirmada	Antes de los datos: umbral y operacionalización por event_id. Después de los datos: ventana de veredicto (W2)
			en L0 (extracción ejecutada manualmente). Medición exploratoria sobre ioc_sessions, definida a posteriori: 39/41 (95,1 %). Por event_id en los contrastes: 9/15 (C1), 14/66 (C2) y 15/49 (C3). La completitud por sesión se informa en P3′.		

Hipótesis (criterio §4.9)	Umbral exigido	Operacionalización	Resultado observado	Veredicto	Fijado
P3′ — Completitud por sesión (reformulación de P3)	≥ 70 % de las sesiones con login exitoso	Sesión con todos los indicadores que las reglas del extractor derivan del log de Cowrie en ioc_sessions	C1: 15/15 (100 %; IC 95 % [79,6 %; 100 %]); registro publicado después de la ventana. C2: 66/66 (IC 95 % [94,5 %; 100 %]); exactitud 32/32 y 1.108/1.108. C3: 49/49 (IC 95 % [92,7 %; 100 %]); exactitud 32/32 y 817/817	Cumplida en C1, C2 y C3; reformulación posterior a W1 que no modifica el veredicto de P3	Después de los datos (definida tras W1); con registro previo antes de C2 y C3
P4 — Reportes automatizados	≥ 90 % de las sesiones con login exitoso	Sesión contenida en un reporte generado en modo trigger	35/41 (85,4 %; IC 95 % [71,6 %; 93,1 %]) en W2; 9/10 (90 %) en W1; 0/5 en L0. Cron de validación de 2 horas; 4/4 ejecuciones en modo trigger en cada ventana (W1 y W2). C1, disparo diario 50 11 * * *: 15/15 (100 %); 06/10 (C2) y 09/10 (C3), disparo de producción 0 8 * * * a las 08:00:00: 66/66 y 49/49; todos por construcción	No confirmada en la ventana de veredicto (igualó el umbral en W1, 9/10 con límite inferior del IC de 59,6 %, y lo alcanzó por construcción en C1, C2 y C3)	Antes de los datos: umbral. Después de los datos: ventana de veredicto (W2)
Hipótesis general (§1.6)	Sostenida si se confirman las cuatro hipótesis operativas (§4.6)	Contraste conjunto de P1 a P4	P1 y P2 confirmadas; P3 y P4 no confirmadas en la ventana de veredicto	Parcialmente sostenida (categoría formulada al analizar los resultados, §4.6)	Después de los datos: categoría «parcialmente sostenida»

Nota. Intervalos de Wilson al 95 %. El registro previo de C1 no se publicó antes de la ventana; los de C2 y C3, sí (Tabla 4.1). Fijado: «Antes de los datos» indica lo establecido antes de observar los datos que sustentan el veredicto, y «Después de los datos», lo establecido al analizar los resultados (§4.6); la definición de «ataque de interés» (sesión con autenticación exitosa), común a P3 y P4, se fijó después de los datos. Elaboración propia a partir del §6.9.

8.2.1 Resultados confirmados

P1 y P2 se confirman (Tabla 8.1). P1 compara la latencia de ingesta de una máquina con el tiempo de triaje y extracción de una persona, tareas no equivalentes, por lo que con un umbral del 50 % se cumple casi por construcción y no mide un ahorro de trabajo analítico (§7.2.3); el 100 % de P2 es en buena medida estructural. La cadena captura → reenvío → ingesta → persistencia funcionó sin pérdidas una vez activado el workflow de ingesta (§6.2.4 a 6.2.6).

8.2.2 Resultados no confirmados

P3 no se confirma en la ventana de veredicto (Tabla 8.1): la medición dependía del esquema del catálogo (restricción UNIQUE (type, value) con un único event_id por indicador; §7.2.3), no del funcionamiento del extractor. P3′, la completitud por sesión definida después de observar la ventana W1, se cumple en los tres contrastes (C1, C2 y C3), pero mide la fidelidad del extractor a sus propias reglas y no cambia el veredicto.

P4 no se confirma en la ventana de veredicto (Tabla 8.1): las sesiones sin cobertura son posteriores al último disparo del cron de validación, y los contrastes con el disparo diario y el de producción la alcanzan por construcción, porque cada reporte abarca las 24 horas previas; lo verificado es que el disparo programado ocurre sin intervención manual, no el cumplimiento del umbral.

Criterio de aceptación de la hipótesis general. Se la consideraba sostenida si se confirmaban las cuatro hipótesis operativas (§4.6). Con P1 y P2 confirmadas y P3 y P4 no confirmadas en la ventana de veredicto, queda parcialmente sostenida, categoría formulada al analizar los resultados. Con la regla binaria original (sostenida o no sostenida), la hipótesis general no se sostiene en su totalidad, porque P3 y P4 no se confirman: la conclusión no depende de esa categoría. La evidencia respalda la viabilidad técnica de automatizar la captura, la normalización, la persistencia, la extracción programada de indicadores y la emisión programada de reportes, pero no alcanza los umbrales de atribución por sesión ni de cobertura por reportes. C1, C2 y C3 alcanzaron los umbrales de P1, P2 y P4 (esta última por construcción) y el de P3′, pero no cambian esta calificación.

Alcance de lo demostrado. El trabajo demuestra la viabilidad técnica del pipeline en laboratorio: un honeypot, n8n y PostgreSQL pueden capturar, estructurar y reportar eventos de forma automatizada. No demuestra la generación de inteligencia de amenazas con valor operativo, porque los indicadores producidos provienen de ataques simulados o de sesiones reales reproducidas por el propio equipo, ni que el sistema fortalezca la capacidad de respuesta defensiva (§1.2), lo que requiere tráfico real y analistas. La única evaluación del reporte con lectores ajenos al sistema (cinco participantes que no son analistas) dio una utilidad percibida de 3,0 sobre 5, por debajo del umbral de 4,0, y 0 de 10 respuestas correctas sobre el fragmento de log con el criterio registrado (§7.7.3). Los veredictos de P1 a P4 se obtuvieron con la configuración de las validaciones L0 a W2 (red plana); la configuración final, endurecida y segmentada, se validó después en C3 y no modifica esos veredictos.

Validación ampliada (R1). Con 200 sesiones reales de un dataset público (1.020 comandos) reproducidas desde una dirección interna, el pipeline persistió los 3.110 eventos de Cowrie sin claves sin par, capturó 1.018 de los 1.020 comandos enviados (99,8 %), cumplió P2 estricto (100 %) y obtuvo una completitud por sesión de 200 de 200. R1 no modifica los veredictos de P1 a P4, que son los de L0 y W2: muestra que los criterios técnicos se sostienen con comandos más variados, no que el sistema produzca inteligencia con valor operativo (§6.13).

8.2.3 Aportes del trabajo

El trabajo aporta un prototipo reproducible y descripto tal como fue construido; un marco de evaluación con umbrales explícitos y con las consultas que permiten reconstruir cada cifra; la

identificación de dos requisitos operativos que el diseño no contemplaba —el estado de activación de los workflows y la monitorización de extremo a extremo—, puestos de manifiesto por la falla del lote 1 y atendidos después con un monitor de extremo a extremo; y la lección de diseño que dejó P3: una métrica por sesión requiere una relación indicador–sesión desde el modelo de datos inicial. Tres elementos son transferibles a otros prototipos: el protocolo de validación con registro previo publicado antes de la ventana (C2, C3 y R1); la conciliación por SQL de las cifras contra el log de la fuente, en este caso Cowrie; y health-monitor como patrón para detectar fallas silenciosas de extremo a extremo, como una ingesta inactiva que no genera errores.

8.3 Síntesis global

8.3.1 Tabla de correspondencia objetivos-resultados

La correspondencia entre objetivos específicos y resultados está en la Tabla 6.8 (§6.10). Con los objetivos 3 y 5 reformulados (§1.4.2), los objetivos 1, 2, 3, 5 y 7 se cumplieron; los objetivos 4 y 6 y el objetivo general se cumplieron parcialmente, por las razones que allí se detallan.

8.3.2 Evaluación global

El prototipo cumple su función central en la captura y la ingesta, y demuestra la extracción y el reporte en modo trigger. No alcanza el umbral de P3 con la operacionalización definida de antemano: un catálogo deduplicado no registra observaciones por sesión (§7.2.3). La emisión programada de reportes se verificó con un disparador más frecuente que el de producción y su cobertura quedó por debajo del umbral en la ventana de veredicto (§6.9). Todas las muestras son simuladas o reproducidas por los autores y provienen de una única IP interna: los hallazgos sostienen la viabilidad técnica del enfoque en laboratorio, no su desempeño frente a tráfico real.

8.4 Trabajo futuro

Las líneas de trabajo futuro se priorizan en la Tabla 8.2 (§8.5) y son de dos tipos: de investigación (la ventana de varios días, la métrica del valor de la inteligencia por sesión, la ampliación de la muestra y la línea de base con analistas) y de ingeniería (el manejo de errores, el endurecimiento y el enriquecimiento). Además de las acciones de esa tabla, quedan las siguientes.

1. Adoptar como criterio principal de P2 la definición estricta —campos obligatorios por tipo de evento y fidelidad campo a campo con el log de Cowrie—, aplicada en esta tesis solo como control posterior (§6.9), y fijarla antes de una nueva validación.

2. Observar el extractor en una operación prolongada. La corrección del 29/09/2026 eliminó los errores en las 33 ejecuciones de la ventana W2; resta verificarla con tráfico real y volúmenes mayores.

3. Revisar el tratamiento de datos antes de exponer el sistema. En un despliegue expuesto a Internet, las direcciones IP y credenciales capturadas pueden vincularse con personas reales: corresponde definir plazos de retención, controles de acceso y, si procede, seudonimización, de acuerdo con la Ley 25.326; además, almacenar las credenciales capturadas con hash o cifrado y habilitar TLS en la conexión de n8n a PostgreSQL (hoy PGSSLMODE=disable; §4.15).

4. Alinear los digests. Las imágenes de Cowrie y n8n ya se fijan por digest en docker-compose.yml; queda por reconciliar la discrepancia con los digests registrados en la BITACORA el 11/08/2026.

5. *Repetir la réplica independiente C.* Una persona ajena al equipo la hizo una vez con éxito (§6.13.4), pero con una IA que ejecutó los comandos y sin los tiempos de todos los pasos; conviene repetirla con otras personas y máquinas, sin asistencia de IA, registrando el tiempo de cada paso y las intervenciones de los autores.

8.5 Trabajo futuro priorizado

8.5.1 Matriz de priorización

Tabla 8.2: Priorización del trabajo futuro.

Prioridad	Acción	Hipótesis u objetivo afectado	Esfuerzo estimado
1	Ventana de varios días, más larga que el período de 24 h del reporte, con el disparo de producción (observado dos veces: el 06/10/2026 en 14 h y el 09/10/2026 en 12 h), para que la cobertura de P4 no quede garantizada por construcción	P4	Bajo (varios días de funcionamiento)
2	Métrica del valor de la inteligencia por sesión, sobre sesiones etiquetadas, que distinga los indicadores informativos de los que toda sesión aporta por construcción, como la IP de conexión	P3	Bajo
3	Ramas de error en event-ingest y report-generator, cola persistente en el forwarder para interrupciones de n8n de más de cinco minutos y corrección de la agregación de top_ips en report-generator	Robustez (§6.4)	Medio
4	Endurecimiento restante: sistema de archivos de solo lectura y secretos	Seguridad (§5.6.6)	Medio

Prioridad	Acción	Hipótesis u objetivo afectado	Esfuerzo estimado
5	Muestra ampliada con orígenes distintos (R1 ya cubrió guiones y credenciales reales desde una única IP interna), línea de base manual con analistas ajenos al equipo y evaluación del reporte con analistas, presentado con tablas o gráficos (§7.7.3)	Validez externa	Medio
6	Reputación de IP, análisis de artefactos, geolocalización con tráfico real y segundo honeypot	Diseño objetivo (§1.5)	Alto

Nota. Elaboración propia.

8.5.2 Criterios de priorización

El orden responde a dos criterios: primero, las acciones que permiten resolver hipótesis no confirmadas o confirmadas solo con el cron de validación (P3 y P4); después, las que reducen riesgos operativos y amplían la validez de los resultados. Las capacidades nuevas (reputación, artefactos y un segundo honeypot) van al final porque no condicionan la verificación de las hipótesis ya formuladas.

8.6 Cierre

Este trabajo muestra que un honeypot integrado con n8n puede capturar, normalizar y persistir eventos de ataque con latencias muy inferiores al triaje y la extracción manuales (tareas no equivalentes; §6.9), sin pérdidas en las validaciones con evidencia conservada, y que el pipeline puede extraer indicadores y emitir reportes de forma programada. No alcanzó los umbrales de atribución por sesión (P3) ni de cobertura por reportes (P4), y la hipótesis general queda parcialmente sostenida. Lo demostrado es la viabilidad técnica del pipeline en laboratorio, con ataques simulados o reproducidos por el propio equipo; no se demuestra inteligencia de amenazas con valor operativo. Las cifras se reconstruyen con las consultas y los archivos del repositorio (Tabla 4.1; Anexo X).

Capítulo IX: Recomendaciones

9.1 Introducción

Las recomendaciones de este capítulo se derivan de las limitaciones de la validación (Tabla 7.1) y se presentan por destinatario: instituciones educativas, PyMEs e investigadores. Las líneas de trabajo futuro del propio prototipo están en el §8.4 y la Tabla 8.2, y no se repiten aquí. Como la evidencia procede de un laboratorio con ataques simulados o reproducidos, ninguna recomendación se contrastó con tráfico real.

9.2 Recomendaciones para instituciones educativas

Se recomienda implementar la arquitectura como laboratorio académico para la enseñanza de ciberseguridad defensiva: el Capítulo VI mostró que la captura y la ingesta operan de forma automática y que la extracción de IoCs y los reportes, ejecutados manualmente el 25/09/2026 y en modo trigger después (§6.7), funcionan en un laboratorio operado por los propios autores. No se evaluó si un equipo sin experiencia previa en automatización puede desplegarlo y operarlo; por eso se lo recomienda como laboratorio de enseñanza y no como servicio. El despliegue requiere infraestructura mínima (un servidor con Docker y conectividad a Internet) y ningún licenciamiento comercial; se estima que alcanza un perfil con conocimientos básicos de Linux, redes, Docker y seguridad informática, estimación que no se verificó con usuarios.

9.2.1 Costos estimados

La Tabla 9.1 detalla los costos estimados de despliegue.

Tabla 9.1: Costos estimados de despliegue.

Componente	Costo estimado	Tipo
Servidor/VPS (4GB RAM, 2 vCPU)	$10-20 USD/mes	Infraestructura
Docker + Cowrie	Gratuito	Software de código abierto
n8n (self-hosted)	Gratuito	Licencia fair-code
PostgreSQL	Gratuito	Software open source
Total mensual estimado	$10-20 USD	Operativo

Nota. Elaboración propia. Los montos son una estimación orientativa del equipo, sin cotización verificada ni fuente de precios.

9.3 Recomendaciones para PyMEs

Se propone evaluar la arquitectura de honeypots como capa adicional de detección temprana, complementaria de la infraestructura de seguridad existente (firewalls, antivirus, EDR). La evidencia de este trabajo proviene de un laboratorio con ataques simulados o reproducidos y

sin tráfico real, por lo que estas recomendaciones son hipótesis de trabajo que deben contrastarse en un entorno propio antes de adoptarse.

1. Monitoreo de perímetro: detección de escaneos y accesos no autorizados. 2. Generación de inteligencia local: IoCs específicos de la infraestructura de la organización. 3. Enriquecimiento de eventos: geolocalización (implementada); lo previsto, como la reputación de IPs, está en la Tabla 8.2. 4. Reportes automatizados: resúmenes periódicos de actividad (disparo verificado con el cron de validación de 2 horas y, el 06/10/2026 y el 09/10/2026, con el diario de producción; §6.9).

9.4 Recomendaciones para investigadores

Se recogen solo extensiones de investigación que el trabajo no abordó: integración con MISP para el intercambio de inteligencia de amenazas (Wagner et al., 2016); aprendizaje automático para clasificar eventos, una vez superadas la limitación de volumen y la ausencia de ground truth (Tabla 7.1; Sommer y Paxson, 2010); comparación con otras plataformas de automatización (Node-RED, StackStorm, Shuffle; Tabla III-13); y automatización de la respuesta (bloqueo temporal de IPs), dashboard en tiempo real y exportación de IoCs en STIX/ TAXII. El diseño, las configuraciones y los flujos de n8n están documentados en los anexos, lo que permite replicar el estudio.

9.5 Matriz resumen de recomendaciones

La Tabla 9.2 sintetiza las recomendaciones de este capítulo por destinatario y prioridad.

Tabla 9.2: Matriz resumen de recomendaciones.

Recomendación	Destinatario	Prioridad	Esfuerzo estimado	Impacto esperado
Implementar honeypots como sensores	PyMEs	Media	Bajo	Medio
Automatizar procesamiento con n8n	PyMEs	Media	Medio	Medio
Desplegar laboratorio académico	Instituciones educativas	Alta	Bajo	Medio
Generar IoCs locales	PyMEs	Media	Bajo	Medio
Integrar reputación de IP y fuentes externas (§9.3, ítem 3)	PyMEs / Investigadores	Media	Medio	Alto
Implementar ML para clasificación	Investigadores	Baja	Alto	Muy alto
Integrar con MISP	Investigadores	Baja	Alto	Muy alto

Nota. Las prioridades de esta tabla (por destinatario) y las de la Tabla 8.2 (por hipótesis u objetivo afectado) usan criterios distintos y no son directamente comparables entre sí. La prioridad y el impacto de las recomendaciones para PyMEs y para instituciones educativas se estimaron a la baja porque la evidencia procede de un laboratorio con ataques simulados o reproducidos y sin tráfico real: son estimaciones del equipo, no resultados medidos.

Anexos

Anexo I: Diagramas de arquitectura

Este anexo contiene los diagramas de secuencia (Figura I-1) y de flujo de datos (Figura I-2) que detallan el procesamiento descripto en el §4.12; las figuras de arquitectura y de topología están junto al texto (Figuras 5.1, 5.2 y 5.3). Las figuras de este anexo representan los tres workflows de event-ingest, ioc-extractor y report-generator; health-monitor (§5.6.4) no se grafica.

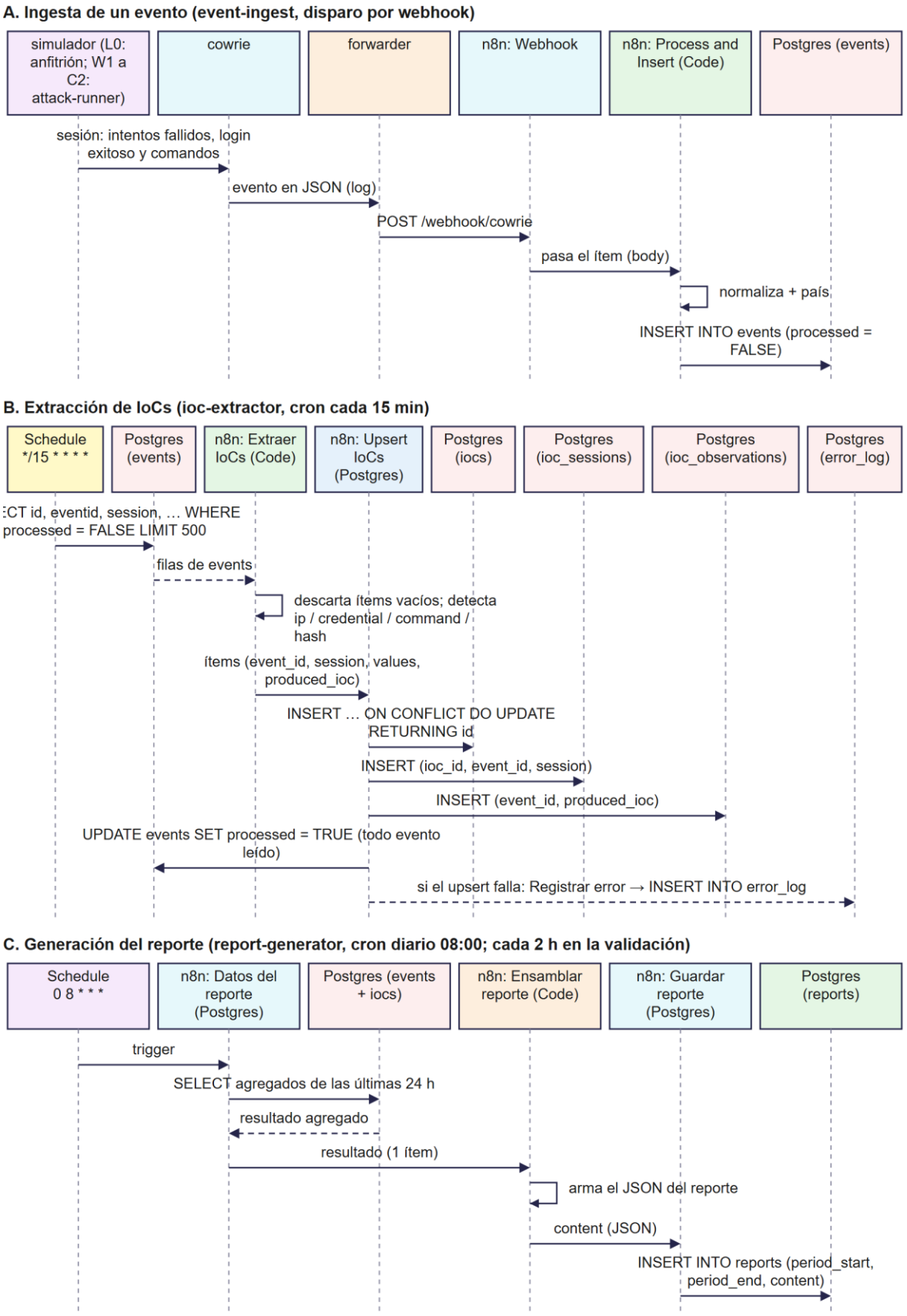


Figura I-1: Diagrama de secuencia del procesamiento de un evento de ataque, en tres partes: (A) ingesta, (B) extracción de IoCs y (C) generación del reporte; la parte B incluye el registro en ioc_sessions, el filtro de ítems vacíos y la rama de error hacia error_log. Fuente: Elaboración propia.

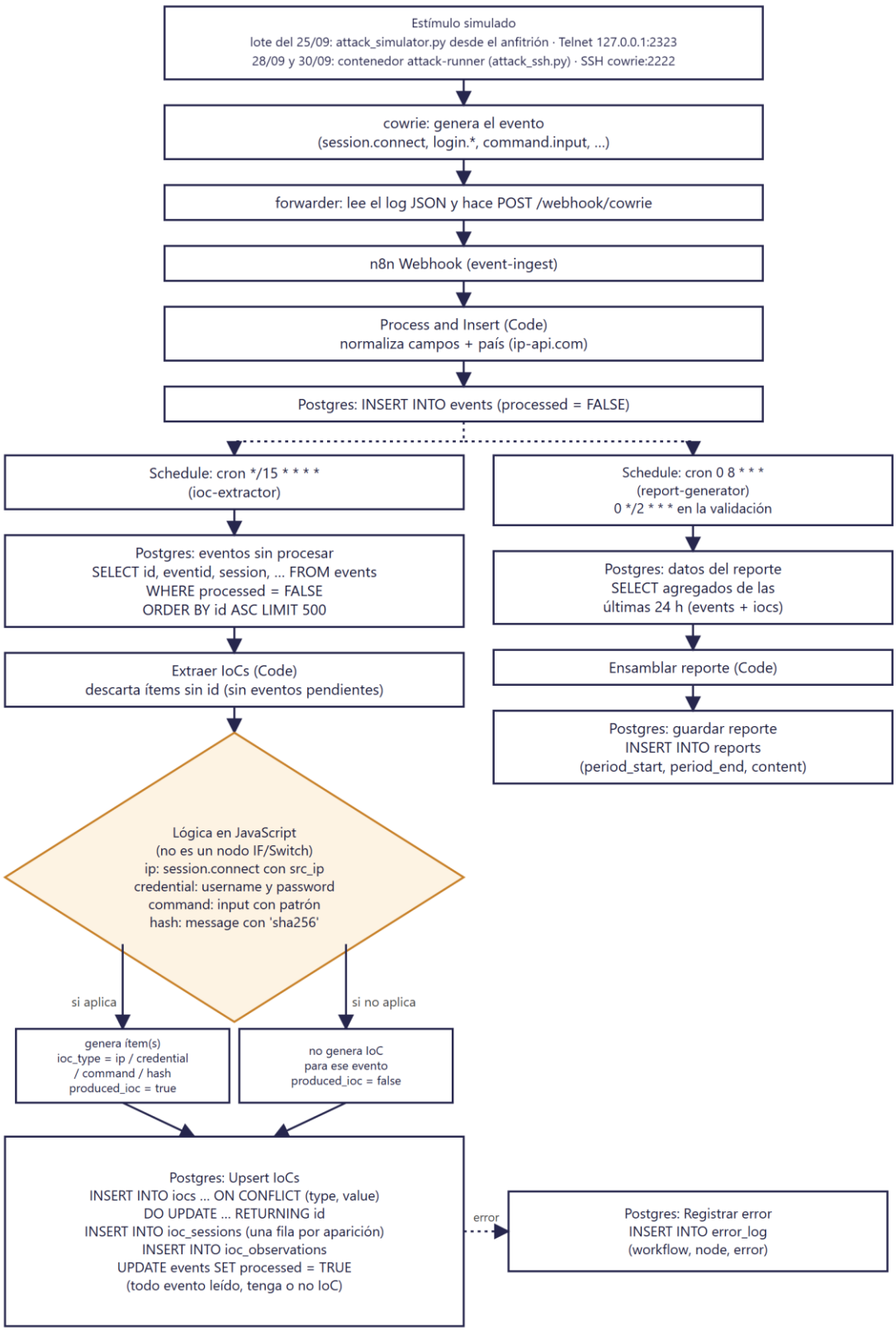


Figura I-2: Flujo de datos desde la captura del evento hasta la generación del reporte, con el extractor vigente en la validación del 30/09/2026 (filtro de ítems vacíos, registro en ioc_sessions y rama de error hacia error_log). Fuente: Elaboración propia.

Anexo II: Workflows n8n exportados

Nota de alcance. Los workflows del tag de la portada son cuatro: los tres descriptos en el §5.6.1 a 5.6.3 (event-ingest, 3 nodos; ioc-extractor, 6 nodos; report-generator, 5 nodos) y health-monitor (12 nodos, §5.6.4), incorporado el 05/10/2026. Sus archivos JSON exportados, con la definición completa de nodos, conexiones, parámetros y configuraciones, se encuentran en el directorio n8n/workflows/ del repositorio público del proyecto (https://github.com/ lucasnorton01/Honeypot_Final_Nocturne_Society, tag Honeypot_Final_2026-10-08g, commit 930acf617ca8c2cb20860aec812deccb074afd5e), donde queda documentado además el detalle de payloads, consultas SQL y configuraciones técnicas. Las ventanas de L0 a C1 se ejecutaron con versiones anteriores de los tres primeros workflows, identificadas por tag en la Tabla X-1 (Anexo X). El repositorio conserva además dos pruebas de desarrollo anteriores al 25/09/2026, una verificación con varios orígenes y su ampliación de muestra del mismo día. Son antecedente del proceso y no sustentan ningún veredicto (§6.2.3); su síntesis es material complementario (Tabla II-1 de docs/ ANEXOS_EXTENDIDOS.md, en el repositorio). El volcado completo de la tabla events de la verificación multi-origen y el detalle de credenciales y comandos por sesión de la ampliación no se reproducen. Las páginas de divulgación que el repositorio traía en Landing page/ y API/ public/landing/, y que describían un diseño y unos veredictos anteriores a los de esta tesis, se retiraron en el tag Honeypot_Final_2026-10-08 y siguen en el historial y en los tags anteriores. El código, los workflows y la evidencia de las ventanas L0 a C2 de ese tag son idénticos a los del tag Honeypot_Final_2026-10-06e (commit e858cc4); difiere solo en documentación, en scripts y archivos de análisis de B1c y de B4, en la evidencia de C3 (docs/evidencia/b4/), en el retiro de la landing page y en dos cambios de la demo (API/public/index.html).

Anexo III: Datos crudos, tablas completas y tablas de apoyo conceptual

Nota de alcance. Los datos de este anexo corresponden a la sesión de referencia del 10–11/08/2026 y a ventanas posteriores anteriores al 25/09/2026, salvo la Tabla III-1b, que corresponde al lote del 25/09/2026. Las tablas de este anexo conservan la numeración de docs/ ANEXOS_EXTENDIDOS.md; las que faltan en la secuencia (III-1a y III-2 a III-5) son material complementario que no se reproduce. Las Tablas III-8, III-9, III-11 y III-13 se trasladaron desde los Capítulos II y III y no figuran en ese documento; las que eran III-10, III-12 y III-14 pasaron al cuerpo (Tablas 3.2, 3.4 y 3.3). Las Tablas III-1 y III-1b se citan en el cuerpo: III-1 (sesión de referencia del 10–11/08/2026) como referencia complementaria de P1, y III-1b (lote del 25/09/2026, N = 65, consulta V1) como su medición principal (§6.3.2, §6.9, Tabla 6.10). Los recuentos de sesiones y la cobertura por reporte automático de las ventanas anteriores quedan superados por el Capítulo VI (§6.2.3) y su síntesis, complementaria, está en docs/ ANEXOS_EXTENDIDOS.md (Tabla II-1). En particular, el reporte de la sesión del

11/08/2026 se generó por ejecución manual (n8n execute), no por el disparo programado. Este anexo reúne las tablas de detalle migradas desde el Capítulo VI (§6.3 a §6.8) y desde la sección §3.11 (estructuras técnicas y ejemplos de los formatos de IoC).

Modelo de datos (DDL). El archivo db/schema.sql del repositorio es la fuente y define las seis tablas y los ocho índices. El esquema relacional consta de seis tablas en PostgreSQL: events, iocs, reports y error_log, correspondientes a las etapas de captura, extracción de IoCs, generación de reportes y manejo de errores descriptas en §5.6, e ioc_observations e ioc_sessions, que registran la trazabilidad del extractor por evento y por sesión:

```
-- Eventos crudos enriquecidos (pipeline event-ingest)
CREATE TABLE IF NOT EXISTS events (
    id          BIGSERIAL PRIMARY KEY,
    eventid     TEXT        NOT NULL,
    session     TEXT,
    src_ip      INET,
    src_port    INTEGER,
    username    TEXT,
    password    TEXT,
    input       TEXT,
    message     TEXT,
    timestamp   TIMESTAMPTZ NOT NULL,
    processed   BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Indicadores de compromiso extraídos (pipeline ioc-extractor)
CREATE TABLE IF NOT EXISTS iocs (
    id          BIGSERIAL PRIMARY KEY,
    type        TEXT        NOT NULL,          -- ip | credential | command | hash (url y domain previstos,
no implementados)
    value       TEXT        NOT NULL,
    confidence  TEXT        DEFAULT 'BAJO',    -- ALTO | MEDIO | BAJO
    event_id    BIGINT      REFERENCES events(id),
    source      TEXT        DEFAULT 'cowrie',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (type, value)
);
-- Reportes periódicos (pipeline report-generator)
CREATE TABLE IF NOT EXISTS reports (
    id           BIGSERIAL PRIMARY KEY,
    period_start TIMESTAMPTZ,
    period_end   TIMESTAMPTZ,
    content      JSONB,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Registro de errores transversal (patrón de manejo de errores)
CREATE TABLE IF NOT EXISTS error_log (
    id          BIGSERIAL PRIMARY KEY,
    workflow    TEXT,
    node        TEXT,
    error       TEXT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Enriquecimiento geográfico (M2: geolocalización de IPs)
ALTER TABLE events ADD COLUMN IF NOT EXISTS country TEXT;
-- Trazabilidad de lectura de ioc-extractor: un registro por cada evento que
-- el workflow lee, produzca o no un IoC (evita perder de vista los eventos
-- sin indicador extraído, ver A-02 de la auditoría).
CREATE TABLE IF NOT EXISTS ioc_observations (
    id           BIGSERIAL PRIMARY KEY,
    event_id     BIGINT      REFERENCES events(id),
    produced_ioc BOOLEAN     NOT NULL DEFAULT FALSE,
    checked_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Relación indicador-sesión: una fila por cada aparición de un IoC ya
-- conocido en una sesión, sin romper la unicidad de iocs (type, value).
-- Permite medir cobertura de P3 por sesión aunque el mismo indicador ya
-- se haya visto antes en otra sesión (ver Paso 7, limitación de iocs
-- UNIQUE encontrada en la validación del 28/09).
CREATE TABLE IF NOT EXISTS ioc_sessions (
    id         BIGSERIAL PRIMARY KEY,
    ioc_id     BIGINT      REFERENCES iocs(id),
    event_id   BIGINT      REFERENCES events(id),
    session    TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Índices para consultas frecuentes
CREATE INDEX IF NOT EXISTS idx_events_src_ip    ON events (src_ip);
CREATE INDEX IF NOT EXISTS idx_events_timestamp ON events (timestamp);
CREATE INDEX IF NOT EXISTS idx_events_processed ON events (processed);
CREATE INDEX IF NOT EXISTS idx_events_country   ON events (country);
CREATE INDEX IF NOT EXISTS idx_iocs_type       ON iocs (type);
CREATE INDEX IF NOT EXISTS idx_ioc_observations_event_id ON ioc_observations (event_id);
CREATE INDEX IF NOT EXISTS idx_ioc_sessions_session ON ioc_sessions (session);
CREATE INDEX IF NOT EXISTS idx_ioc_sessions_ioc_id  ON ioc_sessions (ioc_id);
```

El campo processed de events evita el reprocesamiento por parte de ioc-extractor (§5.6.2); la restricción UNIQUE (type, value) de iocs implementa la deduplicación mediante UPSERT descrita en el mismo apartado; y error_log.workflow permitiría identificar cuál de los cuatro workflows originó un incidente, y desde la versión del 30/09/2026 recibe los errores del nodo de upsert de ioc-extractor (§5.6.4). La restricción UNIQUE (type, value) se mantiene; la relación con cada sesión se registra en ioc_sessions, cuyo ioc_id proviene del RETURNING id del upsert. El cuerpo conserva la narrativa principal y las tablas resumen clave, y remite a este anexo mediante referencias cruzadas explícitas. El modelo de datos del prototipo se presenta en la Figura III-1, a continuación del DDL.

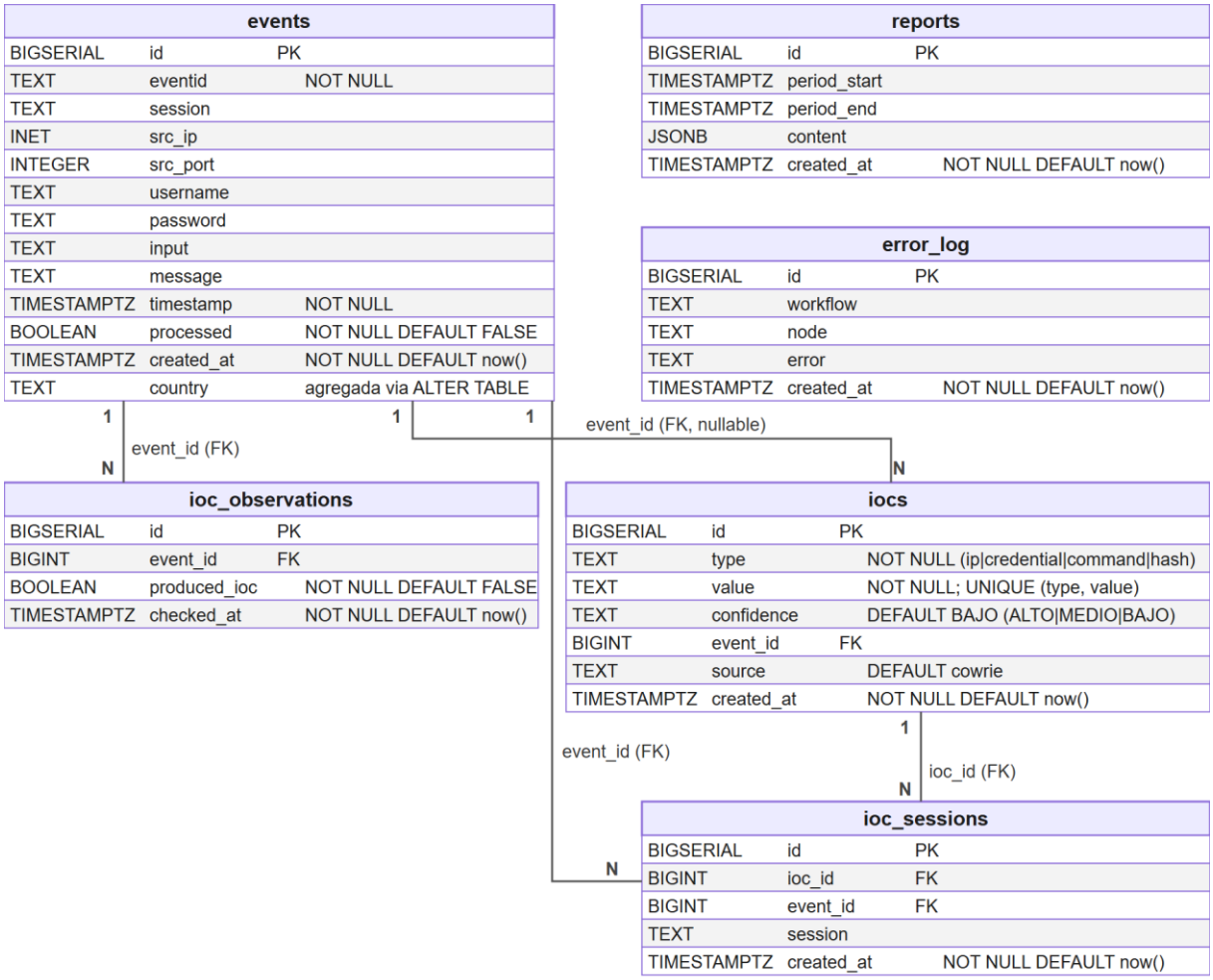


Figura III-1: Modelo de datos del prototipo (seis tablas de PostgreSQL definidas en db/schema.sql). Fuente: Elaboración propia.

A. Datos cuantitativos del Capítulo VI

Tabla III-1: Tiempos de procesamiento por evento (sesión de referencia 10–11/08/2026 — referencia complementaria).

Métrica	Valor
Promedio	297 ms
Mediana (P50)	293 ms
Mínimo	85,5 ms

Métrica	Valor
Máximo	961,6 ms
Desviación estándar muestral	221,2 ms

Nota. Elaboración propia. Según el registro anterior del proyecto, tiempos medidos desde la recepción del evento vía webhook hasta el almacenamiento en PostgreSQL (n = 13 eventos). Con n = 13 no se reportan percentiles extremos (P95/P99); se reportan mínimo, mediana, máximo y desviación estándar muestral, consistentes con el análisis descriptivo de §4.13. Las 13 marcas temporales de la sesión se listan en la Tabla III-1a. Agregado transcripto de un registro anterior; las mediciones individuales no se conservan.

Tabla III-1b: Latencia de persistencia por evento — lote del 25/09/2026 (medición principal de P1).

Métrica	Valor
Promedio	348,02 ms
Mediana (P50)	330,56 ms
Mínimo	100,16 ms
Máximo	798,34 ms
Desviación estándar muestral	160,08 ms

Nota. Elaboración propia. Consulta V1 (§6.3.5), en milisegundos: SELECT count(*), min(ms), avg(ms), max(ms), stddev(ms), percentile_cont(0.5) WITHIN GROUP (ORDER BY ms)

FROM (SELECT EXTRACT(EPOCH FROM (created_at - timestamp)) * 1000 AS ms FROM events WHERE session IN ('a3f0aaaa0cca','d1e20e2ea9b9','5e6fe1c13050','54cf066a37cd',

'e355bc9ba0d1')) t; stddev corresponde a la desviación estándar muestral y percentile_cont(0.5), a la mediana (consulta original en EVIDENCIA_REAL.md del repositorio); ejecutada el 25/09/2026 sobre las 5 sesiones persistidas del lote 2 (N = 65 eventos; ver Tabla 6.3). Verificación de sanidad: 65/65 filas válidas, sin valores nulos ni deltas negativos. Esta medición es la base cuantitativa principal de P1 (§6.9, §6.3.2, Tabla 6.10); la Tabla III-1 (sesión de referencia del 10–11/08/2026) se mantiene como referencia complementaria, sin recalcular ni disponer de sus 13 mediciones individuales publicadas de forma independiente.

En el prototipo todos los IoCs quedan en el nivel de confianza BAJO por defecto (§6.8.4). Los criterios con que se asignarían los niveles ALTO, MEDIO y BAJO son un diseño no implementado; se detallan como material complementario en docs/ ANEXOS_EXTENDIDOS.md (Tabla III-4).

B. Tablas del Capítulo VII (Discusión). Esta sección reúne las tablas de detalle que el cuerpo del Capítulo VII presenta de forma condensada (Tablas III-6 y III-7): precisión de parseo manual vs automatizado y plataformas de automatización para CTI; la comparación con soluciones existentes se sintetiza en la Tabla 2.1 (§2.6) y la comparación de métricas, en la Tabla 6.10.

Tabla III-6: Comparación de precisión de parseo manual vs automatizado.

Tipo de error	Tasa manual estimada[1]	Tasa automática real[2]	Mejora	Fuente de la estimación

Tipo de error	Tasa manual estimada[1]	Tasa automática real[2]	Mejora	Fuente de la estimación
Error de extracción de campos	12–18 % (estimación propia; en B1 se midió 0 % en 20 eventos, §6.12.1)	0 %	n/a	Estimación propia: fatiga del analista en triaje prolongado; no corroborada
Error de clasificación	8–15 % (estimación propia); en la medición del equipo del 02/10/2026, cada integrante tuvo error en 8 de 20 eventos (40 %) con el criterio literal, sobre todo por el formato del valor del indicador	No aplica: el prototipo no clasifica (la confianza de todos los IoCs queda en BAJO, §6.8.4)	n/a	Estimación propia del equipo: precisión de la clasificación manual de incidentes
Error de enriquecimiento (omisión)	5–10 %	No evaluable[3]	n/a	Estimación propia del equipo: omisión de consultas de contexto en el análisis manual

Notas. [1] Rangos estimados por el equipo para analistas SOC de nivel 1-2 en jornadas de 6 h o más; estimación propia, no atribuida a una fuente externa verificable. [2] Porcentaje de eventos con errores sobre el total procesado (§6.12.4). [3] No es evaluable: hasta el 29/09/2026 la consulta de país fallaba en todos los eventos (§6.8.3) y en la validación del 30/09/2026 todo el tráfico provino de una dirección interna, por lo que el país quedó en «Desconocido». La columna «Mejora» compara la tasa manual estimada con la automática observada y hereda la incertidumbre de la estimación manual; no es una medición independiente.

Tabla III-7: Comparación de plataformas de automatización para CTI.

Dimensión	n8n	Python puro	Node-RED	StackStorm
Curva de aprendizaje	Baja	Media-alta	Baja	Alta
Integraciones CTI nativas	Webhook, REST, PostgreSQL, cron	Cualquier librería Python	HTTP, MQTT, PostgreSQL	Sensors, rules, acciones
Escalabilidad	Media (secuencial por defecto; escalable con Docker Swarm o K8s)	Alta (asincrónico, workers)	Limitada (single-threaded por defecto)	Alta (distribuido)
Depuración	Limitada (errores genéricos)	Completa (logging, breakpoints)	Limitada	Buena (trazas de acción)
Comunidad	Comunidad activa y en crecimiento (>400 nodos)	Masiva (PyPI)	Comunidad establecida (cientos de nodos, según la documentación del proyecto)	Mediana (enfocada a empresas)
Madurez CTI	Emergente (pocos casos documentados)	Alta (MISP-STIX, Cuckoo)	Baja	Media (Slack, Jira, PagerDuty)
Huella de recursos	≈ 420 MiB en reposo	80-120 MB de RAM	100-200 MB de RAM	> 1 GB de RAM (multiservicio)
Costo operativo	Gratuito (autoalojado; fair-code, Sustainable Use License)	Gratuito	Gratuito (open source, Apache 2.0)	Gratuito (código abierto)

Nota. Valoración del equipo. La cifra de n8n se midió con docker stats en reposo, sin tráfico, el 07/10/2026, con las imágenes del tag Honeypot_Final_2026-10-06e; las demás celdas de la fila «Huella de recursos» son estimaciones del equipo. Las apreciaciones de curva de aprendizaje, escalabilidad, depuración, comunidad y madurez no se midieron en este trabajo ni se atribuyen a una fuente externa verificable; son un orden de magnitud orientativo, no un resultado de la validación.

C. Tablas de apoyo de los Capítulos II y III. Esta sección reúne las tablas de detalle que el cuerpo remite a este anexo: la línea temporal de hitos de los honeypots (Tabla III-8), el protocolo de búsqueda bibliográfica (Tabla III-9), la comparación ampliada de plataformas de automatización (Tabla III-11) y las alternativas abiertas de orquestación (Tabla III-13). La

taxonomía de honeypots, el ciclo PCPAD y los niveles de inteligencia de amenazas están en el cuerpo (Tablas 3.2, 3.4 y 3.3).

Tabla III-8: Línea temporal de hitos clave en la evolución de honeypots.

Honeypot	Año	Autor/Equipo	Contribución principal	Estado actual
Concepto de honeypot	2002	Lance Spitzner	Sistematización y difusión del concepto y de sus aplicaciones defensivas	Referencia fundacional (Spitzner, 2002)
Honeyd	2002	Niels Provos	Simulación de múltiples sistemas virtuales en una sola máquina; pionero en honeypots de baja interacción	Inactivo
Nepenthes	2005	Equipo Honeynet	Captura automatizada de malware mediante emulación de vulnerabilidades	Inactivo (reemplazado por Dionaea)
Kippo	2009	Upi Tamminen	Honeypot SSH de mediana interacción con sistema de archivos simulado y registro de sesiones	Inactivo (reemplazado por Cowrie)
Dionaea	2009	Honeynet Project	Captura de malware multi-protocolo (SMB, HTTP, FTP, MSSQL, TFTP); sucesor de Nepenthes	Activo
Glastopf	2009	Lukas Rist	Honeypot web de baja/mediana interacción; emulación dinámica de vulnerabilidades web	Inactivo
Cowrie	2015	Michel Oosterhof	Sucesor de Kippo con soporte Telnet, registro JSON, captura de descargas y SFTP	Activo
Snare	2015	Team MUSH	Honeypot de aplicaciones web de media interacción, sucesor de Glastopf; clona páginas web y delega la respuesta en el servidor TANNER	Activo

Nota. Elaboración propia. Para Honeyd, el año es el de lanzamiento de la herramienta; su descripción académica es Provos (2004). Para Cowrie, el año es el del anuncio público (julio de 2015); su desarrollo como bifurcación de Kippo comenzó en 2014. Para Glastopf, el año es el del anuncio en el sitio del Honeynet Project (Honeynet Project, 2009). La columna «Estado actual» es una valoración del equipo, no verificada de forma sistemática en esta tesis.

Tabla III-9: Protocolo de búsqueda bibliográfica.

Elemento	Definición adoptada
Preguntas de investigación	(RQ1) Cómo funcionan los honeypots de baja y media interacción para capturar intentos de ataque; (RQ2) qué tecnologías de automatización (n8n, SOAR) pueden integrarse con honeypots para generar inteligencia de amenazas; (RQ3) qué métricas son relevantes para evaluar la efectividad de un pipeline de automatización de honeypots; (RQ4) qué limitaciones y desafíos presentan los honeypots en entornos productivos.
Bases consultadas	IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink y arXiv (cs.CR); Scopus y Google Scholar se usaron solo para localizar y verificar trabajos (§2.8).
Período	2018-2026.
Cadena base	("honeypot" OR "honeypots" OR "honeynet") AND ("automation" OR "automated" OR "orchestration" OR "workflow") AND ("threat intelligence" OR "attack detection" OR "intrusion detection"), con variantes específicas para Cowrie, SOAR y métricas de evaluación.
Inclusión	Artículos de conferencia, revistas y tesis en inglés o español con texto completo disponible. Se incluyeron además, informes CTI de industria (SANS, Cisco Talos, Akamai, Group-IB, SonicWall, entre otros; ver BITACORA.md sobre las fuentes retiradas), documentación de software libre (GitHub, MISP, Cowrie, n8n, MITRE) y marco legal/ estándares, por su relevancia operativa directa para la arquitectura.
Exclusión	Blogs personales y editoriales sin revisión, trabajos sin revisión de pares, publicaciones anteriores a 2018 y estudios centrados exclusivamente en honeypots de alta interacción. Las excepciones a estos criterios se declaran en el §2.8.
Registro (limitación declarada)	No se llevó un registro cuantitativo contemporáneo por base de datos ni un diagrama PRISMA completo; la descripción de la búsqueda y de sus limitaciones se presenta en §2.8. Del proceso surgieron los antecedentes recientes comparados en la Tabla 2.1 y tres proyectos históricos (The Honeynet Project, HPFeeds, Modern Honey Network), seleccionados por relevancia directa con la arquitectura propuesta.

Nota. Elaboración propia.

Tabla III-11: Comparación ampliada entre SOAR empresarial, n8n y Node-RED.

Dimensión	SOAR empresarial	n8n	Node-RED
Costo	Alto (licencias anuales)	Gratuito (autoalojado; fair-code, Sustainable Use License)	Gratuito (open source, Apache 2.0)
Complejidad	Alta (infraestructura dedicada)	Media (Docker, configuración inicial)	Media-baja (despliegue simple)
Flexibilidad	Limitada a integraciones soportadas	Alta (400+ nodos, API custom, código)	Alta (paquetes contribuidos, código)
Escalabilidad	Escalable pero muy costosa	Media (secuencial por defecto; escalable con Docker Swarm o K8s)	Limitada (single-threaded por defecto)
Curva de aprendizaje	Pronunciada	Baja	Baja
Enfoque	Orquestación y respuesta a incidentes	Automatización de procesos	Flujos de datos y eventos
Seguridad nativa	Alta (RBAC, SOAR, cumplimiento)	Media (credenciales encriptadas, RBAC en cloud)	Básica (sin RBAC nativo)
Comunidad	Empresarial (soporte pago)	Comunidad activa y en crecimiento (>400 nodos)	Comunidad establecida (cientos de nodos, según la documentación del proyecto)
Casos de uso en seguridad	Respuesta a incidentes, orquestación SOC	Ingesta, parseo, enriquecimiento, reportes	Prototipado, IoT, data pipelines

Nota. Las valoraciones de complejidad, flexibilidad, escalabilidad, curva de aprendizaje, seguridad nativa y comunidad son cualitativas del equipo, a partir de la documentación general de cada plataforma, y no se verificaron una por una. La curva de aprendizaje de n8n (baja) es la misma de la Tabla III-7 y se apoya en la experiencia del equipo (≈4 horas para construir event-ingest; §7.6.1).

Tabla III-13: Alternativas abiertas de orquestación y de plataformas de honeypots no evaluadas en esta tesis.

Alternativa	Tipo	Licencia	Relación con esta tesis
Shuffle	SOAR de código abierto con editor visual de flujos	AGPLv3 (backend); MIT (flujos y aplicaciones)	Alternativa directa a n8n, con orientación a seguridad; no desplegada ni medida
TheHive y Cortex	Gestión de casos y análisis de observables	Cortex: AGPL-3.0	Complementarias: un analizador de Cortex podría cubrir la reputación de IP, que el prototipo no implementa
StackStorm	Automatización dirigida por eventos, con motor de reglas y de flujos	Apache 2.0	Alternativa a n8n para el procesamiento; propuesta como comparación futura (§9.4)
T-Pot	Plataforma de más de veinte honeypots con la pila Elastic	GPL-3.0	Alternativa a Cowrie + PostgreSQL; incluye Cowrie y Dionaea; no desplegada ni medida

Nota. Elaboración propia a partir de la documentación de cada proyecto (Shuffle, s.f.; StackStorm, s.f.; TheHive Project, s.f.; Deutsche Telekom Security GmbH, 2023), consultada el 06/10/2026. No se comparó su desempeño con el de n8n.

Anexo IV: Herramienta de demostración para la defensa

Para la defensa oral el equipo desarrolló, con posterioridad al cierre de la evidencia reportada en los Capítulos VI y VIII, una interfaz web (Cowrie Web Interface): una terminal web (xterm.js) y un panel de métricas servidos por Node.js (Express y Socket.IO), que se conecta

por SSH con Cowrie y consulta en modo de solo lectura las tablas events, iocs y reports. Es una capa de presentación: no forma parte del pipeline de captura, procesamiento o generación de reportes descrito en el Capítulo IV y el Anexo II, ni de la evidencia usada para verificar las hipótesis P1-P4 (§6.9, Tabla 8.1). Su uso requiere Node.js ≥ 18 y el stack Cowrie/PostgreSQL en ejecución, y está sujeto a cuatro salvedades: (1) el acceso usa una credencial fija (admin/test123) definida solo para la demostración, no apta para producción y distinta en su propósito de las credenciales documentadas como IoC en los Anexos II y III; (2) toda sesión que el tribunal genere durante la exposición es posterior al cierre de la evidencia y no se incorpora a los conteos de P3 ni P4; (3) sus notificaciones usan un canal de mensajería propio, independiente de report-generator (§5.6), y no forman parte del mecanismo que sostiene la verificación de P4; y (4) las credenciales del canal de alertas y las variables de conexión a la base se gestionan con variables de entorno excluidas del documento y del repositorio público, que solo publica plantillas de ejemplo (.env.example y n8n/workflows/postgres-credential.example.json; §5.6.6).

Anexo V: Ventanas anteriores al 25/09/2026 y limitaciones técnicas identificadas en un segundo entorno

Nota de alcance. Este anexo resume ventanas de validación anteriores al 25/09/2026, en un segundo equipo (16/09/2026) y en repeticiones sobre el entorno original. Sus cifras de P3 y P4 quedan retiradas por no ser reconstruibles y ningún veredicto depende de ellas (§6.2.3); se conserva como registro del desarrollo y de las limitaciones técnicas que dejó.

V.1 Conectividad. En el segundo equipo, los puertos de Cowrie (2222/SSH, 2323/Telnet) no eran alcanzables desde 127.0.0.1 por una limitación de Docker Desktop con WSL2; se atacó desde un contenedor auxiliar en la red interna (“attack-runner”, Python 3.12-slim con paramiko) y se incorporó el archivo de moduli Diffie-Hellman de OpenSSH, cuya ausencia hacía fallar las conexiones SSH.

V.2 Continuidad de la ingesta. De 39 sesiones (16/09/2026) solo 10 quedaron persistidas: el forwarder dejó de reenviar eventos a n8n en dos intervalos (≈29 sesiones perdidas); queda como limitación de robustez (reintento con backoff o cola de mensajes no entregados).

V.3 Tercer intento de ampliación, no exitoso (2 de 15 autenticaciones exitosas y errores de canal en paramiko sin eventos verificables); excluido de todo cálculo, conforme al criterio de no completar datos faltantes.

V.4 Control de calidad. Una etapa de automatización asistida por IA insertó eventos sintéticos directamente en la tabla de eventos (sesiones con prefijo “sim-”, ajeno al formato hexadecimal de Cowrie); se detectaron cruzándolos con los registros reales de sesión y se eliminaron — eventos, IoCs asociados y un reporte agregado contaminado— antes de cualquier análisis. El

episodio corresponde a las ventanas anteriores al 25/09/2026 documentadas en este anexo; su fecha exacta no quedó registrada. Ninguna cifra vigente de esta tesis se calculó sobre esa base: P1 y P2 proceden del lote del 25/09/2026, que partió de una tabla events vacía (Tabla VI-1), y P3 y P4 de la validación del 30/09/2026, con la del 28/09/2026 como antecedente, cada una computada solo sobre las sesiones de su propia ventana (§6.2.4).

V.5 Defecto del extractor de IoCs. La IP de origen se tomaba de cualquier tipo de evento; la condición se corrigió de if (e.src_ip) a if (e.src_ip && e.eventid === 'cowrie.session.connect'). La corrección no elimina el efecto de la restricción de unicidad (type, value) sobre sesiones que comparten la misma IP, que queda como limitación de diseño del esquema.

V.6 Geolocalización (objetivo específico 5). La consulta de país falló en todas las ventanas anteriores al 29/09/2026 porque usaba fetch(), función no disponible en el entorno de código de n8n; su corrección y verificación se describen en el §6.8.3, y el objetivo 5, reformulado, se declara cumplido (Tabla 6.8).

V.7 Segunda validación complementaria de P3 (entorno original). Con el extractor corregido (V.5) se ejecutaron 25 sesiones Telnet (10 autenticadas; 25 de 25 reconciliadas contra PostgreSQL), seis de ellas con comandos del patrón de IoC de tipo comando (wget, curl, nc, python, tftp, chmod, bash, /bin/sh). Sus cifras de P3, como las de las demás ventanas de este anexo, quedan retiradas: los valores vigentes proceden solo de las validaciones del 28/09 y del 30/09/2026 (§6.2.4, §6.9, Tabla 8.1).

Anexo VI: Bitácora de las ventanas de validación

Nota de alcance. Este anexo reúne el detalle cronológico de las ventanas de validación del 28/09, del 30/09 y del 02/10/2026, más el resumen de resultados de C3 (VI.4), trasladado del Capítulo VI para aliviar su lectura: cambios entre ventanas, correcciones previas, intentos descartados, desvíos operativos y configuración. Las cifras que sustentan los veredictos permanecen en los §6.3 y §6.9; aquí se conserva el registro de cómo se obtuvieron. El detalle de C2 está en el §6.2.6 y en docs/PREREGISTRO_B3.md, y el de C3, en el §6.2.7 y en docs/PREREGISTRO_B4.md; aquí solo se resumen los resultados de C3 (VI.4).

VI.1 Ventanas automáticas del 28/09/2026 y del 30/09/2026 (resumen en el §6.2.4).
El lote del 25/09/2026 dejó P3 y P4 sin veredicto: P4, porque no se observó ningún disparo programado, y P3, porque la atribución se concentró en la primera sesión. Para contrastarlas se realizaron dos ventanas automáticas, sin intervención manual desde el inicio de cada ventana. La primera se realizó entre las 00:57:55 y las 08:30 UTC del 28/09/2026 (≈ 7 h 32 min), sobre la versión etiquetada Honeypot_Final_2026-09-28. Se había planificado de 6 a 8 horas, con un cierre previsto a las 6 h (06:57 UTC; BITACORA.md). Con ese cierre, P4 habría sido de 5 de 9 sesiones (55,6 %; IC 95 % [26,7 %; 81,1 %]), porque las sesiones de las 05:45 y las 06:45

UTC solo quedan cubiertas por el reporte de las 07:00 UTC; por esa sensibilidad al momento de cierre, el resultado del 28/09/2026 se informa como antecedente y no sostiene el veredicto (§4.6). Su evidencia (historial de n8n de 440 filas, cruce entre iocs y events de 21 filas, log de Cowrie de la ventana y las salidas literales de cada consulta) está en docs/evidencia/ y docs/VERIFICACION_2026-09-28.md, en el tag Honeypot_Final_2026-10-02-B2.

Cambios respecto del lote del 25/09/2026. (1) ioc-extractor marca como procesado todo evento que lee, tenga o no indicador, lo que resuelve el riesgo de relectura descripto en el Anexo VII.3. (2) Se agregó la tabla ioc_observations, que registra si cada evento leído produjo un indicador; la relación indicador–sesión (ioc_sessions) se incorporó recién para la validación del 30/09/2026. (3) El estímulo pasó del cliente Telnet a attack-runner/attack_ssh.py, que ataca el servicio SSH de Cowrie con credenciales inválidas y con las seis cuentas válidas de cowrie/userdb.txt; lo disparó una tarea horaria del Programador de tareas de Windows (a los :45), que documentó siete corridas entre las 01:45 y las 07:45 UTC, confirmadas por el log de Cowrie. El workflow event-ingest y el forwarder no cambiaron en su lógica. (4) report-generator usó el disparador de validación 0 */2 * * * (horas impares de UTC), restituido después a 0 8 * * * (08:00 ART, 11:00 UTC; §4.9.4); el flujo de la ventana se conserva en experimentos/report-generator-p4-validacion-2026-09-28.json.

Correcciones previas y errores observados. Antes de la ventana se corrigieron dos defectos: el canal SSH del simulador (27/09/2026) y una conversión de tipos en el extractor (28/09/2026, ≈00:12 UTC; BITACORA.md). También se descartó la ventana iniciada el 27/09/2026 a las 22:38 UTC (tablas vaciadas a las 00:57:55 UTC), de modo que ninguno de sus datos integra esta tesis. En esa ventana descartada se amplió de dos a seis el número de cuentas válidas (≈00:35 UTC) tras observar que solo 2 de 6 sesiones exitosas tenían un indicador propio: es un ajuste posterior a la observación de datos de P3 y se declara como amenaza a la validez interna (historia, §4.14). Dentro de la ventana del 28/09 no hubo correcciones ni tráfico manual. El workflow ioc-extractor registró en ella 30 ejecuciones, todas en modo trigger: 7 con éxito, en el primer disparo posterior a cada corrida del simulador, y 23 con el error «column "undefined" does not exist», en disparos sin eventos pendientes; una ejecución adicional (id 869, 08:30:00.048 UTC) es posterior al cierre y no se cuenta. La causa, confirmada el 29/09/2026, es que sin eventos pendientes el nodo de lectura entrega un ítem {"success": true} que el nodo Code procesaba como un evento sin identificador (docs/VERIFICACION_2026-09-28.md, §7). No hubo pérdida de datos: los eventos no procesados se reintentan en el ciclo siguiente. La base de esta ventana no se conservó, porque se truncó antes de la validación del 30/09/2026: sus sesiones y eventos se reconstruyeron desde el log de Cowrie y el historial de n8n (docs/evidencia/), y la atribución de P3 proviene de

iocs_events_2026-09-28.csv; los conteos anteriores al truncado constan en docs/VERIFICACION_2026-09-28.md y no pueden recalcularse.

Correcciones del 29/09/2026, previas a la segunda validación. Se aplicaron, probaron y publicaron tres cambios antes del primer evento de la ventana. (1) El nodo Code del extractor descarta los ítems sin identificador y declara pairedItem, y la salida de error del upsert se deriva a un nodo que inserta en error_log; la prueba unitaria scripts/test_extractor_codenode.js pasó 12 de 12 casos con la corrección y falló en 4 con la versión anterior. (2) La consulta de país de event-ingest, que fallaba siempre porque usaba fetch() —también el 25/09 y el 28/09/2026—, pasó a this.helpers.httpRequest() y se verificó con IP públicas (§6.8.3). (3) Se agregó la tabla ioc_sessions (id, ioc_id, event_id, session, created_at), con una fila por cada aparición de un indicador en una sesión, y el upsert de iocs pasó de ON CONFLICT DO NOTHING a ON CONFLICT (type, value) DO UPDATE … RETURNING id.

Intento descartado. Un primer intento, iniciado el 29/09/2026 a las 21:12:31 UTC, se interrumpió a los ≈18 minutos por un reinicio del anfitrión (actualización obligatoria de WSL2); con el stack caído ≈2 h 17 min, se perdió el disparo de las 23:00 UTC de report-generator. Se descartó entero, sin rellenar el hueco: se vaciaron las tablas y se reinició la ventana.

Validación del 30/09/2026. La ventana válida fue de las 23:53:57 UTC del 29/09/2026 (SELECT now() en PostgreSQL, con las tablas vacías) a ≈08:01 UTC del 30/09/2026 (8 h 07 min), sin reinicios de los servicios ni huecos en el conteo horario de eventos (entre 196 y 278 por hora completa). El simulador corrió cada 15 minutos (33 corridas), rotando la cuenta válida entre las seis de cowrie/userdb.txt, con un 30 % de probabilidad de sumar una segunda, y todas las conexiones provienen de la dirección interna 172.19.0.7 del contenedor. El workflow report-generator usó el disparador de validación de 2 horas, restituido al cierre, y cada reporte abarca las 24 horas previas, por lo que los cuatro reportes se superponen. La versión del sistema no cambió y no hubo tráfico ni ejecuciones manuales. El cierre previsto era las 07:53:57 UTC (BITACORA.md), pero la ventana se cerró a las 08:01 e incluyó dos sesiones autenticadas a las 08:00 UTC. Con el cierre previsto, P4 sería de 35 de 39 sesiones (89,7 %; IC 95 % [76,4 %; 95,9 %]), también por debajo del umbral, y la medición exploratoria sobre ioc_sessions, de 39 de 39.

La ventana produjo 1.998 eventos, 41 sesiones con autenticación exitosa, 21 indicadores en iocs (20 credenciales y una dirección IP), 654 filas en ioc_sessions y 4 reportes (01:00, 03:00, 05:00 y 07:00 UTC). Al cierre se habían procesado 1.930 eventos; los 68 restantes, de la tanda de las 08:00 UTC, aún no habían pasado por el extractor. El CSV events_2026-09-30.csv se exportó después del cierre, cuando el extractor ya había corrido a las 08:15, por lo que todas sus filas figuran con processed = t; los conteos al cierre se obtuvieron filtrando por created_at hasta las 08:01 UTC (docs/VERIFICACION_2026-09-30.md). En el historial de n8n, event-

ingest registró 1.998 ejecuciones por webhook, ioc-extractor 33 y report-generator 4, todas con éxito y las dos últimas en modo trigger, y error_log terminó con cero filas. El log de Cowrie de la ventana contiene los mismos 1.998 eventos que la tabla events: no hubo pérdidas entre captura y persistencia (docs/VERIFICACION_2026-09-30.md, §9). La evidencia y las consultas con sus salidas literales están en docs/evidencia/ y docs/VERIFICACION_2026-09-30.md, en el tag Honeypot_Final_2026-10-02-B2. El aumento de frecuencia del simulador y la tabla ioc_sessions se decidieron después de observar P3 del 28/09/2026 y se declaran como ajustes posteriores a la observación de los datos (§6.11). La Tabla 6.2 (§6.3.1) compara ambas validaciones.

VI.2 Contraste C1 del 02/10/2026, con reglas escritas de antemano (resumen en el §6.2.5).
Las validaciones del 28/09 y del 30/09/2026 dejaron tres debilidades metodológicas: el cierre de la ventana no estaba fijado de antemano, y P4 resultó sensible a él (§6.2.4); el disparo diario de report-generator nunca se observó, porque ambas usaron el disparador de 2 horas (§4.9.4); y la medición de P3 sobre ioc_sessions se definió después de observar los datos (§4.9.3). Para responder a ellas se realizó una tercera validación, cuyas reglas se fijaron por escrito antes del primer evento en docs/PREREGISTRO_B2.md: horario de la ventana, configuración, métricas, umbrales y script de análisis (scripts/b2_analisis.js), con el SHA-256 de cada archivo. Según consta en BITACORA.md, el registro se redactó y su SHA-256 se anotó antes de iniciar la ventana, pero se publicó junto con los resultados en un mismo commit, posterior al cierre: un tercero no puede comprobar la anterioridad por el historial de versiones, y esta descansa en la declaración de los autores (la validación del 05–06/10/2026 lo corrigió, §6.2.6). La validación se ejecutó en una instalación nueva en el equipo de uno de los autores, con volúmenes vacíos, sobre el código del tag Honeypot_Final_2026-10-02 más los cambios registrados. No reemplaza a la ventana de veredicto: sus resultados se informan aparte en el §6.9 y los veredictos del 30/09/2026 no se modifican (§4.6).

Configuración y ventana. La ventana duró 67 minutos, por disponibilidad del equipo anfitrión: inicio a las 13:47:30 UTC (10:47:30 ART; SELECT now() con las tablas vacías) y cierre a la hora registrada, 14:55:10 UTC (11:55:10 ART), sin extensión. El simulador (attack-runner/attack_ssh.py, versión b3-2026-10-02) corrió cada 5 minutos entre las 10:55 y las 11:40 ART (10 corridas), con doce cuentas válidas en cowrie/userdb.txt rotadas por franja de 15 minutos, y en una parte de las sesiones exitosas (5 de 15 en C1, 32 de 66 en C2 y 23 de 49 en C3; el registro previo decía «la mitad») agregó un comando benigno (cifras verificadas sobre docs/evidencia/b2/events.csv, b3/events.csv y b4/events.csv: sesiones con cowrie.login.success que tienen un cowrie.command.input del pool de IoC; 5 de 15 = 33,3 %, 32 de 66 = 48,5 % y 23 de 49 = 46,9 %; todas las sesiones exitosas ejecutaron además comandos de reconocimiento) que coincide con el patrón de IoC de tipo comando (ls /bin, bash --version,

python3 -V, which sh, ls -la /usr/bin). El workflow report-generator usó una expresión diaria, 50 11 * * *: un único disparo por día, como el de producción (0 8 * * *), adelantado a las 11:50 ART para observarlo dentro de la ventana; el flujo de la ventana (experimentos/report-generator-b2-2026-10-02.json) es idéntico al de producción salvo esa expresión, y el de producción se restituyó al cierre. Durante la preparación, la credencial de PostgreSQL importada de la plantilla (ssl: false) hizo fallar todas las inserciones mientras el forwarder registraba [OK], la misma falla silenciosa del lote 1 (§6.4). Se corrigió con ssl: "disable" antes del inicio; luego, una corrida de prueba persistió los 38 eventos que registró Cowrie (BITACORA.md; el historial de n8n muestra 39 ejecuciones, una más, de origen no registrado). Todo esto ocurrió antes de la ventana y las tablas se vaciaron al iniciar, de modo que no afecta ninguna cifra.

Desvíos. Se registraron tres, declarados en BITACORA.md. (1) La corrida de las 10:55 ART terminó a los 22 s con el código 0xC000013A (interrupción de la ventana de consola), después de 5 de sus 9 intentos; no se repitió ni se borró, y las otras nueve terminaron sin error. (2) El paso del cierre que exporta el historial de n8n invocó un intérprete de Python inexistente en el anfitrión; docs/evidencia/b2/ejecuciones.csv se generó después del cierre con la misma consulta, copiando también los archivos WAL de la base de n8n. (3) PowerShell 5 escribió los CSV exportados con marca de orden de bytes (BOM), que se quitó antes de correr el análisis registrado. Ninguno afecta los datos capturados: la base no se modificó durante la ventana, y el script de análisis, el simulador y los flujos coinciden con los hashes del registro previo. Los hashes de attack_ssh.py, userdb.txt y los flujos son los del checkout CRLF de Windows; los demás, los del contenido LF de git.

Resultados. Cowrie registró 629 eventos de 103 sesiones, 15 de ellas con autenticación exitosa, y la tabla events contiene los mismos 629, sin claves sin par (docs/evidencia/b2/ cowrie_ventana.json). El workflow event-ingest registró 629 ejecuciones por webhook; ioc-extractor, 4 (11:00, 11:15, 11:30 y 11:45 ART), y report-generator, 1, a las 11:50:00 ART en modo trigger (62 ms), todas con éxito y sin ejecuciones manuales; error_log terminó vacío. El extractor generó 26 indicadores (21 credenciales, 4 comandos y una dirección IP) y 211 filas en ioc_sessions: es la primera ventana en la que se observan indicadores de tipo comando. La evidencia (tablas exportadas, log de Cowrie, ejecuciones, resultados_b2.json y SHA256SUMS.txt) está en docs/evidencia/b2/, y los resultados por hipótesis, en el §6.9.

Línea de base manual medida. El mismo día se midió la línea de base de P1, hasta entonces estimada, con una página de cronometraje sobre 20 eventos reales de la validación del 30/09/2026; el procedimiento y los resultados se presentan en el §6.12.1. Un primer intento se descartó como piloto, y los datos de ambos intentos, el generador de las páginas y el cálculo (scripts/b1_analisis.js) se publican en docs/evidencia/b1/.

VI.3 Cronología del lote del 25/09/2026 (resumen en el §6.2.2).

La cronología hora por hora del lote (línea de base, lote 1, corrección, ping de prueba, lote 2 y ejecución manual) se detalla en la Tabla VI-1.

Tabla VI-1: Cronología de la validación del 25/09/2026.

Hora ART (UTC−3)	Hora UTC	Hito
01:21:39	04:21:39	Línea de base 1: events = 0; iocs = 0
01:28:50 – 01:46:48	04:28:50 – 04:46:48	Lote 1: 5 corridas del simulador, todas completadas del lado del cliente
Tras el lote 1	Tras el lote 1	Verificación: events = 0. Causa: event-ingest inactivo; el webhook devolvió HTTP 404 a las 65 entregas
Antes de 01:58	Antes de 04:58	Corrección: activación de event-ingest por CLI y reinicio de n8n
01:58:31	04:58:31	Ping sintético de prueba al webhook (HTTP 200; 1 fila, sin sesión)
01:59:42	04:59:42	Línea de base 2: events = 1; iocs = 0
01:59:57 – 02:14:30	04:59:57 – 05:14:30	Lote 2: 5 corridas del simulador
Tras el lote 2	Tras el lote 2	Verificación: events = 66 (Δ = +65); iocs = 0
≈ 02:16:53	≈ 05:16:53	Ejecución manual (CLI) de ioc-extractor y report-generator
Posterior	Posterior	Activación por CLI de ioc-extractor y report-generator; reinicio de n8n; los tres workflows figuran activos

Nota. Horas registradas con date -u y convertidas a ART (la zona America/Argentina/Mendoza no aplica horario de verano). La hora de la ejecución manual se infiere del period_end del reporte generado (05:16:53 UTC). Elaboración propia.

El archivo docs/evidencia/cowrie_lote_2026-09-25.json contiene además 34 sesiones SSH (184 eventos) ajenas a los lotes 1 y 2, registradas el 24/09 y el 25/09/2026 antes y después del lote, de dos clases. Treinta y dos sesiones (24/09 20:57 UTC y 25/09 10:22 y 23:31 UTC, desde 172.18.0.4 y 172.18.0.5; 6 con login exitoso) reproducen, con las mismas cuentas, en el mismo orden y con una sesión cada 2 s, la lista fija de sesiones del simulador attack-runner (attack-runner/attack_ssh.py del tag Honeypot_Final_2026-09-23), sin sus primeras sesiones; son corridas del simulador anteriores a las ventanas y la BITACORA no registra su motivo. Las otras dos (25/09 05:31 UTC, desde 172.18.0.1, sin autenticación y con una petición «GET / HTTP/1.1» al puerto 2222) son conexiones aisladas de origen no registrado. No integran ninguna cifra de este trabajo: las 5 sesiones del lote 2 se identifican por sus identificadores (Tabla 6.3).

VI.4 Resultados del contraste C3 del 08–09/10/2026 (detalle en el §6.2.7).

La Tabla VI-2 resume los resultados de C3, calculados con el script registrado antes de la ventana (scripts/ b4_analisis.js) y publicados en docs/evidencia/b4/resultados_b4.json (tag Honeypot_Final_2026-10-08g).

Tabla VI-2: Resultados de la validación C3 (08–09/10/2026).

Medida	Umbral	Resultado
Ventana	Cierre 11:15 UTC	23:13:59 UTC del 08/10 a 11:15:00 UTC del 09/10; 38 corridas sin errores; 2.386 eventos en Cowrie y en events; 397 sesiones, 49 con autenticación exitosa
P1	< 50 % de 10,36 s	Media 1.697,89 ms (16,4 %); mediana 1.258,75 ms; máximo 5.255,19 ms; ningún evento sobre 15 s
P2	≥ 80 %	Estricto 2.386/2.386 (IC 95 % [99,8 %; 100 %]); idénticos al log de Cowrie 2.382/2.386 (99,8 %)
P3	≥ 70 %; ≥ 95 %	Completitud 49/49 (IC 95 % [92,7 %; 100 %]); por event_id 15/49 (30,6 %); catálogo 32/32; atribución 817/817
P4	08:00 ART; ≥ 90 %	Un disparo, 11:00:00 UTC, en modo trigger y con éxito; un reporte guardado; cobertura 49/49 (IC 95 % [92,7 %; 100 %]), por construcción

Medida	Umbral	Resultado
n8n y Telegram	Sin errores	ioc-extractor 48 y health-monitor 144, sin errores ni alertas; manuales 0; error_log vacío. Telegram: 793 enviadas, 0 rechazadas (resumen 48/49, un SSLError)
Configuración	Final del 06/10	Seis servicios de ejecución continua (attack-runner termina después de cada corrida); solo 127.0.0.1:2222, :2323 y :5678; tres redes internal; sin fallas

Nota. Intervalos de Wilson al 95 %. P3: completitud ≥ 70 % y calidad ≥ 95 %. Elaboración propia a partir de docs/evidencia/b4/resultados_b4.json.

Anexo VII: Detalle de los contrastes de P1 a P4 y consultas SQL

Nota de alcance. Este anexo reúne, trasladados del §6.9 y del §6.3.5 para aliviar su lectura, el detalle de cada contraste por ventana, con las Tablas VII-1 a VII-5, y el texto completo de las consultas SQL. Los veredictos y las cifras principales permanecen en la Tabla 6.8 y en el §6.9. El script scripts/verificar_cifras.js recalcula esas cifras desde docs/evidencia/ (coinciden las 17).

VII.1 P1: contraste con cada línea de base (resumen en el §6.9).

P1 — Reducción de tiempo (tiempo por evento inferior al 50 % del manual estimado). Para la ventana del 30/09/2026, la línea de base manual era una estimación propia de ≈30 s por evento (§6.12.1), por lo que el umbral equivalía a 15 s. El 02/10/2026 la línea de base se midió con dos integrantes del equipo (18,44 s para triaje y extracción; umbral de 9,22 s); ese contraste se informa más abajo. La latencia de entrega observada en el lote 2 es inferior a 1 s por evento (§6.3.2): la consulta V1 sobre los 65 eventos persistidos arrojó una media de 348,02 ms (mediana 330,56 ms; mínimo 100,16 ms; máximo 798,34 ms; desviación estándar muestral 160,08 ms; Anexo III, Tabla III-1b), equivalente al 1,16 % de la línea de base de 30 s. En la validación del 30/09/2026, con la consulta de país operativa, la media fue de 1.608,76 ms y el máximo de 4.922,22 ms (5,4 % y 16,4 % de la línea de base; N = 1.998), también por debajo del umbral (§6.3.2). La medición cubre la etapa de ingesta (captura → persistencia); la extracción y el reporte no se computan porque se ejecutaron manualmente. P1 se confirma para la etapa de ingesta. Si la comparación se limita a las fases 1 y 2 de la línea de base (recepción y extracción de campos), que son las comparables con la ingesta medida, la referencia es de ≈10,5 s en sus puntos medios y de 8 s en el extremo inferior de sus rangos (Tabla VIII-1). Frente a ella, la media equivale al 3,3 % (25/09/2026) y al 15,3 % (30/09/2026) de los puntos medios, y al 4,4 % y al 20,1 % del extremo inferior: el umbral del 50 % también se cumple con la comparación equivalente.

Contraste con la línea de base medida (02/10/2026). El registro previo fijó como métrica de P1 la latencia media de la validación del 02/10/2026 frente a la mediana por evento del paso 1 de la línea de base manual medida (fases 1 y 2 de la Tabla VIII-1): 18,44 s (§6.12.1). La latencia media fue de 1.453,16 ms (mediana 795,64 ms; máximo 4.418,98 ms; N

= 629; ningún evento superó los 15 s), el 7,9 % de la línea de base; el máximo equivale al 24,0 %. Con la misma referencia, las medias del 25/09 y del 30/09/2026 equivalen al 1,9 % y al 8,7 %. P1 se confirma también frente a esa línea de base, para la etapa de ingesta, que proviene de dos integrantes del equipo autor. Las dos mediciones posteriores con participantes ajenos al equipo dan medianas menores: 10,36 s (B1, no registrada de antemano) y 11,01 s (B1c, con registro previo; Anexo VIII). Frente a ellas, la media del 02/10/2026 equivale al 14,0 % y al 13,2 % (el máximo, al 42,7 % de la primera), y la de W2, al 14,5 % de la segunda: P1 se confirma con las tres referencias. Con N = 2, N = 5 y N = 10 no se hace inferencia, y la comparación enfrenta la ingesta automática con el triaje y la extracción humanos, tareas no equivalentes (Tabla 6.9). El 05–06/10/2026, con 10,36 s como referencia registrada, la media fue de 1.370,85 ms (mediana 693,53 ms; máximo 4.440,01 ms; N = 3.218), el 13,2 %: P1 se confirma.

VII.2 P2: control estricto frente al log de Cowrie (resumen en el §6.9).
P2 — Estructuración de datos (al menos el 80 % de los eventos estructurados). Se considera estructurado todo evento insertado en la tabla events con tipo (eventid), sesión (session) y marca temporal (timestamp) no nulos (§4.9.2). La definición es independiente de la extracción de indicadores y de la columna processed, que solo actualiza el extractor. Las 65 filas del lote 2 se agrupan en las 5 sesiones de la Tabla 6.3 (ninguna con sesión nula), y eventid y timestamp están declarados NOT NULL en el esquema. Resultado: 65/65 (100 %; IC 95 % [94,4 %; 100 %]).

P2 se confirma. Como control de robustez, definido después de observar los datos y sin modificar el veredicto, se aplicó además un criterio estricto por tipo de evento (username y password en cowrie.login.*, input en cowrie.command.input, src_ip y src_port en cowrie.session.connect) y se comparó cada fila de events con el evento correspondiente del log JSON de Cowrie (misma sesión, tipo y marca temporal al milisegundo) en siete campos: eventid, session, src_ip, src_port, username, password e input. En el lote 2, los 65 eventos cumplen el criterio estricto (50 de 50 con campos obligatorios por tipo; IC 95 % [92,9 %; 100 %]) y son idénticos al log de Cowrie; el 30/09/2026, los 1.998 eventos cumplen el criterio estricto (861 de 861 con campos obligatorios; IC 95 % [99,6 %; 100 %]) y son idénticos al log de Cowrie (IC 95 % [99,8 %; 100 %]), según scripts/verificar_p2_estricto.js sobre los CSV y los logs publicados en docs/evidencia/. El 02/10/2026, con ambas métricas registradas de antemano, 629 de 629 eventos cumplen el criterio estricto (100 %; IC 95 % [99,4 %; 100 %]) y 628 de 629 son idénticos al log de Cowrie (99,8 %; IC 95 % [99,1 %; 99,97 %]). El 05–06/10/2026, los 3.218 eventos cumplen el criterio estricto y 3.211 son idénticos al log de Cowrie (99,8 %). Las excepciones, una el 02/10 y siete el 05–06/10, son el comando «which sh » con un espacio final, almacenado igual que en Cowrie; la diferencia la introduce el

comparador del script registrado, que recorta los espacios finales solo del valor de la base. Los intervalos de Wilson de P2 tratan cada evento como una observación independiente, pero los eventos se agrupan en sesiones generadas por un mismo guion, de modo que el tamaño muestral efectivo es menor (5, 41, 15 y 66 sesiones) y los intervalos deben leerse como optimistas. Además, el 100 % es en buena medida estructural, porque eventid y timestamp son NOT NULL.

Denominadores del criterio estricto. El criterio exige campos obligatorios solo a los eventos de cuatro tipos (cowrie.login.success, cowrie.login.failed, cowrie.command.input y cowrie.session.connect); los demás (por ejemplo, client.version, kex o closed) lo cumplen sin requisito adicional. Por eso se informan dos bases: los eventos con campos obligatorios y todos los eventos persistidos. En el lote 2: 50 de 50 (IC 95 % [92,9 %; 100 %]) y 65 de 65; el 30/09/2026: 861 de 861 ([99,6 %; 100 %]) y 1.998 de 1.998; el 02/10/2026: 275 de 275 ([98,6 %; 100 %]) y 629 de 629 ([99,4 %; 100 %]); el 05–06/10/2026: 1.406 de 1.406 ([99,7 %; 100 %]) y 3.218 de 3.218 ([99,9 %; 100 %]). Ningún evento con campos obligatorios los tuvo vacíos, de modo que la elección de la base no altera el resultado. Las cifras de 629 y de 3.218 que figuran en otros apartados corresponden a la base «todos los eventos».

VII.3 P3: atribución por sesión, causa raíz y contrastes (resumen en el §6.9).
P3 — Generación de IoCs (al menos el 70 % de las sesiones con actividad maliciosa confirmada). Conforme al §4.9.3, se operacionaliza «sesión con actividad maliciosa confirmada» como sesión con autenticación exitosa (N = 5 sesiones en el lote del 25/09/2026, 10 en la validación del 28/09/2026 y 41 en la del 30/09/2026), y «genera IoC» como la existencia de al menos una fila en iocs atribuible a un evento de la sesión mediante event_id. La Tabla VII-1 presenta el resultado de la consulta de atribución (§6.3.5).

Tabla VII-1: IoCs atribuidos por sesión mediante event_id.

Sesión	IoCs atribuidos	Primer evento (UTC)
e355bc9ba0d1	4	04:59:58.114
54cf066a37cd	0	05:03:01.241
5e6fe1c13050	0	05:05:59.178
d1e20e2ea9b9	0	05:09:15.299
a3f0aaaa0cca	0	05:12:14.805
Sesiones con ≥ 1 IoC atribuido	1 de 5 (20 %)	

Nota. Elaboración propia a partir de la consulta de atribución del §6.3.5.

Primer contraste (lote del 25/09/2026): 1/5 (20 %; IC 95 % [3,6 %; 62,4 %]), frente al umbral del 70 %; incluso el límite superior del intervalo queda por debajo del umbral, de modo que este primer contraste no confirmó P3. Además, el único IoC atribuido se produjo mediante una ejecución manual (CLI) de ioc-extractor (§6.7); con el criterio de automatización estricto

aplicado a P4 (§4.9.4), que excluye las ejecuciones manuales, la cobertura automática verificada habría sido 0/5 (0 %). Este trabajo no recalcula P3 bajo ese criterio, pero deja constancia de la inconsistencia para que una futura revisión adopte el mismo estándar en ambas hipótesis.

Causa raíz. Los cuatro indicadores —la IP 172.18.0.1 y las credenciales admin:123456, admin:admin y admin:test123— quedaron atribuidos a la sesión e355bc9ba0d1, la primera del lote. El resultado se explica por la interacción entre la restricción UNIQUE (type, value) de la tabla iocs y el mecanismo de upsert del workflow ioc-extractor:

1. Cada fila de iocs admite un único event_id, y la restricción impide que exista más de una fila por par (type, value).
2. El extractor lee los eventos pendientes en orden ascendente de id. La primera aparición de cada valor —en los eventos de la primera sesión— crea la fila del indicador y fija su event_id.
3. Las apariciones posteriores del mismo valor en las cuatro sesiones siguientes chocan con la restricción. El upsert no crea filas nuevas ni reasigna el event_id: la observación repetida se absorbe sin dejar rastro en la tabla.
4. En consecuencia, todos los indicadores quedan concentrados en el primer evento observado de cada valor, y las sesiones que repiten valores ya catalogados figuran con cero indicadores atribuidos, aunque el extractor haya procesado sus eventos.

La lectura por valor confirma que la información no se perdió: la IP y las credenciales de las cinco sesiones están presentes en iocs (5/5), pero esa lectura no discrimina, porque la deduplicación garantiza el resultado para cualquier sesión que repita valores ya catalogados, y no es la operacionalización adoptada.

El resultado demuestra empíricamente una limitación del diseño actual, no una falla del extractor: la tabla iocs funciona como catálogo de indicadores únicos, mientras que la hipótesis mide cobertura por sesión. Un catálogo deduplicado registra qué se observó, pero no cuándo ni en cuántas sesiones. La corrección, una tabla de relación indicador–sesión (ioc_sessions: ioc_id, event_id, session) que registra cada observación de un indicador sin romper la unicidad del catálogo, se incorporó después de ver el resultado de W1 y se usó desde la validación del 30/09/2026 (§6.9, Anexo VI.1). Con ese diseño, las cinco sesiones del lote habrían quedado vinculadas a sus indicadores.

Se deja constancia, además, de un riesgo adicional de la versión del extractor vigente en el lote del 25/09/2026, corregido antes de la validación del 28/09/2026 (§6.2.4): lee los eventos con processed = false con un límite de 500 filas y marca como procesados solo los que producen un indicador; los eventos sin indicador permanecen pendientes y se releen en cada ejecución.

Con 66 filas en events, ese límite no pudo alcanzarse en esta validación, pero en una operación prolongada podría impedir que el extractor alcance los eventos nuevos (consulta V3).

Contraste en modo automático (validación del 28/09/2026). Con la misma operacionalización, las 10 sesiones con autenticación exitosa (reconstruidas desde el log de Cowrie, porque la base de esta ventana no se conservó; §6.2.4) arrojan 6/10 (60 %; IC 95 % [31,3 %; 83,2 %]) sesiones con al menos una fila en iocs atribuida por event_id (Tabla VII-2). El resultado está por debajo del umbral del 70 % y, a diferencia del primer contraste, el intervalo incluye el umbral, de modo que con N = 10 la evidencia no permite confirmar ni descartar la hipótesis en términos estadísticos. Por el criterio aplicado en todo el capítulo (proporción observada frente al umbral), P3 no se confirma.

La Tabla VII-2 detalla los IoC atribuidos por event_id a cada una de las 10 sesiones de W1 con autenticación exitosa.

Tabla VII-2: IoCs atribuidos por sesión mediante event_id (validación del 28/09/2026).

Sesión	IoCs atribuidos
288c84e44ce4	0
eeac3a0a6920	1
106fccee7392	0
91e5b5a84891	0
ccd16ec6808b	1
b823986a217b	1
95e63fb49317	1
d29deee8497d	0
4d51d10d620a	1
25b267072c6e	1
Sesiones con ≥ 1 IoC atribuido	6 de 10 (60 %)

Nota. Elaboración propia a partir de la consulta de atribución del §6.3.5 sobre las 10 sesiones con autenticación exitosa, reconstruida con docs/evidencia/sesiones_2026-09-28.csv y docs/evidencia/iocs_events_2026-09-28.csv (procedimiento en docs/VERIFICACION_2026-09-28.md, §9).

Causa raíz. Cuatro de las 10 sesiones figuran sin indicadores atribuidos porque todas sus credenciales ya estaban catalogadas por sesiones anteriores: el archivo cowrie/userdb.txt definía, en esas validaciones, solo seis cuentas válidas (la validación del 02/10/2026 usó doce; §6.2.5), y por la restricción UNIQUE (type, value) de iocs únicamente la primera sesión que usa cada credencial crea una fila propia. La tabla ioc_observations cuantifica la brecha entre catálogo y observaciones: registra 132 eventos para los que el extractor produjo al menos un indicador, frente a 21 filas en iocs (20 credenciales y una dirección IP). Es el mismo mecanismo diagnosticado en el primer contraste, ahora con más sesiones, y confirma que la limitación es del diseño del catálogo y no del extractor. La corrección (tabla de relación indicador–sesión) se implementó para la validación del 30/09/2026 y no se aplicó retroactivamente a esta ventana.

Contraste en la validación del 30/09/2026 (ventana de veredicto). Con la misma operacionalización, 6 de las 41 sesiones con autenticación exitosa tienen al menos una fila en iocs atribuida por event_id: 6/41 (14,6 %; IC 95 % [6,9 %; 28,4 %]). El resultado empeora respecto del 28/09/2026 por el mismo mecanismo: con seis cuentas válidas y la restricción UNIQUE (type, value), solo la primera sesión que usa cada credencial crea una fila propia, de modo que el numerador queda acotado mientras el denominador crece con el volumen. Las seis sesiones atribuidas lo están por la credencial de su evento cowrie.login.success. Veredicto: P3 no se confirma.

Medición exploratoria sobre ioc_sessions. La tabla ioc_sessions, incorporada tras observar el resultado del 28/09/2026, registra cada aparición de un indicador en una sesión. Con ella, 39 de las 41 sesiones (95,1 %; IC 95 % [83,9 %; 98,7 %]) tienen al menos un indicador registrado; las dos restantes son las dos últimas de la ventana (08:00:07 y 08:00:13 UTC), cuyos eventos el extractor aún no había procesado al cierre. Este valor no determina el veredicto, por dos razones. Primero, la medición se definió después de observar los datos de P3 (§4.9.3). Segundo, es casi estructural: cada una de las 39 sesiones tiene exactamente dos filas (la dirección 172.19.0.7 de su evento cowrie.session.connect y la credencial de su intento), como las 327 sesiones de la tabla (654 filas). Lo que verifica es que el extractor procesa todas las sesiones y registra sus indicadores aunque ya estén catalogados; no mide la calidad ni la novedad de la inteligencia producida.

Tabla VII-3: P3 en la validación del 30/09/2026 (N = 41 sesiones con autenticación exitosa).

Medición	Sesiones con indicador (IC 95 %)	Uso
Fila en iocs atribuida por event_id (§4.9.3)	6/41 = 14,6 % [6,9 %; 28,4 %]	Veredicto: no confirmada
Fila en ioc_sessions (definida a posteriori)	39/41 = 95,1 % [83,9 %; 98,7 %]	Exploratoria

Nota. Elaboración propia a partir de docs/VERIFICACION_2026-09-30.md, §4a y §4b, (41 filas por sesión de ambas consultas) y de docs/evidencia/ioc_sessions_2026-09-30.csv (654 filas: cada aparición de un indicador en una sesión, de 327 sesiones).

Contraste C1, con reglas escritas de antemano (02/10/2026). Para esta validación se registró de antemano una operacionalización de completitud: una sesión con autenticación exitosa cuenta si ioc_sessions contiene, al cierre, todos los indicadores que las reglas del extractor derivan del log de Cowrie de esa sesión (la IP de cowrie.session.connect, las credenciales, los comandos que coinciden con el patrón y los hashes), y no solo alguno. Resultado: 15/15 (100 %; IC 95 % [79,6 %; 100 %]), frente al umbral del 70 %, con 5 sesiones que incluían un comando de IoC; por event_id, 9/15 (60,0 %; IC 95 % [35,7 %; 80,2 %]). P3 alcanza el umbral con el criterio registrado, pero el veredicto del 30/09, obtenido con la operacionalización fijada antes de los datos del 28/09/2026 (§4.6), no se modifica. Además, como la completitud se

mide contra las reglas del propio extractor, solo verifica que este las aplica sin omisiones, no la calidad ni la novedad de la inteligencia (§8.4). El 05–06/10/2026, con el mismo criterio, el resultado fue de 66/66 (IC 95 % [94,5 %; 100 %]); por event_id, 14/66 (21,2 %). Las dos métricas de exactitud registradas (umbral del 95 %) dieron 32/32 indicadores del catálogo derivables del log de Cowrie y 1.108/1.108 filas de ioc_sessions atribuidas a su propia sesión. Miden la fidelidad del extractor a sus reglas, no si un indicador es malicioso.

VII.4 P4: cobertura por reportes en cada ventana (resumen en el §6.9).
P4 — Reportes automatizados (al menos el 90 % de los ataques de interés cubiertos sin intervención manual). Se define «ataque de interés» como sesión con autenticación exitosa (N = 5 sesiones en el lote del 25/09/2026, 10 en la validación del 28/09/2026 y 41 en la del 30/09/2026), y se computan solo los reportes generados en modo trigger (§4.9.4).

Primer contraste (lote del 25/09/2026). Lo que esa validación verificó es la corrección del mecanismo. La lógica del workflow report-generator es correcta: su ejecución manual recorrió los cinco nodos hasta «Postgres: Guardar Reporte», terminó con estado success y produjo el reporte id = 2, cuya ventana de 24 horas contiene las cinco sesiones (cobertura por cualquier reporte: 5/5). El disparador programado también está correctamente definido en el workflow exportado: un nodo Schedule Trigger con la expresión cron 0 8 * * * (ejecución diaria a las 08:00 ART, 11:00 UTC; §4.9.4), y el workflow figuraba activo al cierre de la validación.

Lo que esa validación no verificó fue el disparo autónomo. Al momento de la ejecución manual, report-generator no estaba activo, y tras su activación no transcurrió un ciclo diario completo; en consecuencia, no se observó ninguna ejecución programada y ningún reporte fue generado en modo trigger. La cobertura automática verificada es 0/5 (0 %; IC 95 % [0 %; 43,4 %]).

Veredicto del primer contraste (lote del 25/09/2026): P4 no se confirma; queda verificada exclusivamente mediante ejecución manual (CLI). Se reporta de forma transparente como una limitación del entorno de pruebas y del procedimiento de validación, no como un defecto del mecanismo: el veredicto significa «no verificada», no «refutada». Su confirmación requería mantener el entorno en funcionamiento y registrar, con la consulta V2 del §6.3.5 y el historial de ejecuciones de n8n, reportes en modo trigger que cubran las sesiones; ese contraste se realizó el 28/09/2026 (§6.2.4) y se presenta a continuación.

Contraste en modo automático (validación del 28/09/2026). Se computan las 10 sesiones con autenticación exitosa (reconstruidas desde el log de Cowrie; §6.2.4) y solo los reportes generados en modo trigger. Durante la ventana, report-generator se ejecutó 4 veces (01:00, 03:00, 05:00 y 07:00 UTC; reportes id 1 a 4), todas en modo trigger y con estado success (4/4); una consulta sobre el historial de n8n confirma cero ejecuciones en modo distinto de trigger de report-generator e ioc-extractor dentro de la ventana. El reporte id 1 no cubre

ninguna sesión, porque la primera autenticación exitosa de la ventana ocurrió a las 01:45:24 UTC. Nueve de las 10 sesiones quedaron cubiertas por al menos uno de esos reportes (Tabla VII-4): 9/10 (90 %; IC 95 % [59,6 %; 98,2 %]), frente al umbral del 90 %. La única sesión sin cobertura (106fccee7392) se registró a las 07:45:04 UTC y no fue alcanzada por ningún disparo posterior antes de que terminara la ventana (08:30 UTC); no se excluye del cómputo para no modificar la regla a posteriori.

La Tabla VII-4 detalla qué reportes en modo trigger cubren cada una de las 10 sesiones de W1.

Tabla VII-4: Cobertura por reportes en modo trigger de las sesiones con autenticación exitosa (validación del 28/09/2026).

Sesión	Autenticación exitosa (UTC)	Reportes en modo trigger que la cubren (id)
4d51d10d620a	01:45:24	2, 3, 4
eeac3a0a6920	02:45:12	2, 3, 4
25b267072c6e	03:45:11	3, 4
288c84e44ce4	03:45:24	3, 4
b823986a217b	04:45:05	3, 4
95e63fb49317	05:45:06	4
d29deee8497d	05:45:11	4
91e5b5a84891	06:45:14	4
ccd16ec6808b	06:45:33	4
106fccee7392	07:45:04	ninguno (posterior al último reporte de la ventana)
Sesiones cubiertas	9 de 10 (90 %)	

Nota. Elaboración propia a partir de la consulta V2 del §6.3.5 cruzada con el historial de ejecuciones de n8n (docs/evidencia/ejecuciones_2026-09-28.csv); la hora de autenticación y los reportes que cubren cada sesión están en docs/evidencia/sesiones_2026-09-28.csv. Todos los reportes que cubren cada sesión se generaron en modo trigger.

Veredicto del contraste del 28/09/2026. P4 alcanza el umbral en esa ventana, sin margen: la proporción observada iguala el 90 % exigido y el intervalo es amplio (límite inferior 59,6 %) por N = 10; con el cierre previsto de 6 h, la cobertura habría sido de 5/9 (55,6 %; §6.2.4). Su alcance está acotado por dos condiciones: (a) se midió con el disparador de 2 horas (§4.9.4), no con el diario de producción, con el cual la cobertura de una sesión puede demorar hasta 24 horas; y (b) cada reporte abarca las 24 horas previas, por lo que la cobertura queda garantizada por construcción una vez generado cualquier reporte posterior. Lo que esa observación verifica, en rigor, es que el disparo programado se produce sin intervención manual, en un entorno de laboratorio con tráfico simulado, y no que se alcance la cobertura exigida; el veredicto se toma de la ventana de veredicto (§4.6), que se presenta a continuación.

Contraste en la validación del 30/09/2026 (ventana de veredicto). Se computan las 41 sesiones con autenticación exitosa y solo los reportes generados en modo trigger. El workflow report-generator se ejecutó 4 veces (01:00, 03:00, 05:00 y 07:00 UTC; reportes id 1 a 4), todas en modo trigger y con éxito, sin ejecuciones en otro modo. Treinta y cinco sesiones quedaron

cubiertas por al menos un reporte: 35/41 (85,4 %; IC 95 % [71,6 %; 93,1 %]), frente al umbral del 90 %. Las seis sesiones sin cobertura son, sin excepción, las últimas de la ventana (entre las 07:00:13 y las 08:00:13 UTC): el último reporte se generó a las 07:00:00 UTC y el siguiente disparo habría ocurrido a las 09:00 UTC, después del cierre. No se excluyen del cómputo, con el mismo criterio aplicado el 28/09/2026. Veredicto: P4 no se confirma en la ventana de veredicto. Con un reporte cada 2 horas que abarca las 24 horas previas, la cobertura queda determinada por la proporción de sesiones posteriores al último disparo, no por fallas de ejecución: las cuatro ejecuciones programadas se produjeron sin intervención manual. La Tabla VII-5 agrupa las sesiones por tramo horario de autenticación.

La Tabla VII-5 agrupa por tramo horario de autenticación las 41 sesiones de W2 y los reportes en modo trigger que las cubren.

Tabla VII-5: Cobertura por reportes en modo trigger de las sesiones con autenticación exitosa (validación del 30/09/2026).

Autenticación exitosa (UTC)	Sesiones	Reportes en modo trigger que las cubren (id)
00:00 – 00:45	4	1, 2, 3, 4
01:00 – 02:45	12	2, 3, 4
03:00 – 04:45	11	3, 4
05:00 – 06:45	8	4
07:00 – 08:00	6	ninguno (posteriores al último reporte de la ventana)
Sesiones cubiertas	35 de 41 (85,4 %)	

Nota. Elaboración propia a partir de docs/VERIFICACION_2026-09-30.md, §5, cruzado con docs/evidencia/ejecuciones_2026-09-30.csv; todos los reportes que cubren cada sesión se generaron en modo trigger.

Contraste C1, con reglas escritas de antemano (02/10/2026). Con la expresión diaria 50 11 * * *, report-generator se ejecutó una vez, a las 11:50:00 ART, en modo trigger y con éxito, sin ejecuciones manuales, y su reporte cubre las 15 sesiones con autenticación exitosa: 15/15 (100 %; IC 95 % [79,6 %; 100 %]). El registro previo advertía que el resultado se da por construcción: el reporte abarca las 24 horas previas y la ventana duró 67 minutos, de modo que un disparo posterior a la última sesión las cubre a todas. Lo que esta validación agrega es la observación de un disparo diario, que no se había observado en ninguna ventana: un flujo idéntico al de producción, salvo la hora, disparó en modo trigger sin intervención. No mide la frecuencia adecuada del disparador ni observa el disparo de las 08:00 ART de producción. P4 alcanza el umbral en esa validación; el veredicto del 30/09 no cambia. Los resultados de P3 y P4 del 02/10 no constituyen evidencia independiente: P3 se mide contra las reglas del propio extractor y P4 se cumple por construcción (reporte de 24 h y ventana de 67 min); por eso no se suman a los de la ventana de veredicto. El 05–06/10/2026 se observó por primera vez el disparo de producción (0 8 * * *, sin modificaciones): una sola ejecución, a las 08:00:00 ART del 06/10, en modo trigger y con éxito, cuyo reporte cubre las 66 sesiones (IC 95 % [94,5 %; 100 %]). Se cumplen las condiciones registradas, de nuevo por construcción (ventana de 14 h).

VII.5 Consultas SQL de las cifras del Capítulo VI (resumen en el §6.3.5).

El texto completo de las consultas (conteos, V1 a V3 y las de P3 y P4 de W2) se reproduce a continuación; sus salidas literales están en docs/evidencia/ y en docs/VERIFICACION_2026-09-28.md y docs/VERIFICACION_2026-09-30.md.

```
-- Conteos antes y después de cada lote
SELECT count(*) FROM events;
SELECT count(*) FROM iocs;
-- Sesiones persistidas (Tabla 6.3)
SELECT session, count(*), min(timestamp), max(timestamp)
FROM events GROUP BY session ORDER BY max(timestamp) DESC LIMIT 6;
-- Evento y sesión a los que apunta cada IoC (Tabla VII-1)
SELECT i.id, i.type, i.value, i.event_id, e.session, e.eventid
FROM iocs i LEFT JOIN events e ON e.id = i.event_id ORDER BY i.id;
-- IoCs atribuidos por sesión mediante event_id (P3, Tabla VII-1); S = las cinco sesiones de la Tabla 6.3
SELECT e.session, count(i.id) AS iocs_atribuidos
FROM events e LEFT JOIN iocs i ON i.event_id = e.id
WHERE e.session IN S GROUP BY e.session ORDER BY e.session;
-- Sesiones con al menos un IoC atribuido (k de 5)
SELECT count(*) FILTER (WHERE n > 0) AS sesiones_atribuidas, count(*) AS total
FROM (SELECT e.session, count(i.id) AS n FROM events e
      LEFT JOIN iocs i ON i.event_id = e.id WHERE e.session IN S GROUP BY e.session) s;
-- V1. Latencia de inserción por evento (base de la medición principal de P1, §6.3.2)
SELECT count(*), min(ms), avg(ms), max(ms), stddev(ms),
       percentile_cont(0.5) WITHIN GROUP (ORDER BY ms)
FROM (SELECT EXTRACT(EPOCH FROM (created_at - timestamp)) * 1000 AS ms
      FROM events WHERE session IN S) t;
-- V2. Reportes que contienen cada autenticación exitosa (P4), a cruzar con el modo de ejecución del
historial de n8n
SELECT e.session, r.id AS report_id, r.created_at
FROM events e JOIN reports r ON e.timestamp BETWEEN r.period_start AND r.period_end
WHERE e.eventid = 'cowrie.login.success' AND e.session IN S;
-- V3. Eventos pendientes de procesar por el extractor
SELECT count(*), min(id) FROM events WHERE processed = false;
-- Validación del 30/09/2026. P3, operacionalización del §4.9.3 (event_id contra iocs)
SELECT e.session, count(i.id) AS iocs
FROM events e LEFT JOIN iocs i ON i.event_id = e.id
WHERE e.session IN (SELECT session FROM events WHERE eventid = 'cowrie.login.success')
GROUP BY e.session ORDER BY e.session;
-- P3, medición exploratoria sobre ioc_sessions
SELECT e.session, (SELECT count(*) FROM ioc_sessions s WHERE s.session = e.session) AS apariciones
FROM (SELECT DISTINCT session FROM events WHERE eventid = 'cowrie.login.success') e ORDER BY e.session;
-- P4, reportes que cubren cada sesión con autenticación exitosa
SELECT e.session, min(e.timestamp) AS login, count(DISTINCT r.id) AS reportes
FROM events e LEFT JOIN reports r ON e.timestamp BETWEEN r.period_start AND r.period_end
WHERE e.eventid = 'cowrie.login.success' GROUP BY e.session ORDER BY login;
```

Anexo VIII: Medición de la línea de base manual

Nota de alcance. Este anexo reúne, trasladada del §6.12.1, la descripción completa de las dos mediciones de la línea de base manual sobre 20 eventos reales: la del 02/10/2026, con dos integrantes del equipo, la del 05/10/2026, con cinco participantes externos (datos en docs/evidencia/b1/), y la del 08/10/2026, con diez más y registro previo (B1c; docs/evidencia/b1c/).

Medición de la línea de base (02/10/2026). Para contrastar la estimación, dos integrantes del equipo procesaron a mano 20 eventos reales de la validación del 30/09/2026 —una muestra con semilla fija (20260930) con 5 eventos cowrie.login.failed, 5 cowrie.command.input, 4 cowrie.session.connect, 3 cowrie.login.success, 2 cowrie.client.version y 1 cowrie.session.closed— en una página de cronometraje que muestra el JSON crudo y mide dos pasos por separado: el paso 1, triaje y extracción de campos (fases 1 y 2 de la Tabla VIII-1), y el paso 2, clasificación de severidad y registro del indicador (fases 4 y 5), con el criterio de clasificación escrito en la página. La fase 3 no se midió, porque todas las direcciones son internas del laboratorio y no admiten enriquecimiento externo. El tiempo corre solo con el

evento a la vista, y antes hubo dos eventos de práctica que no se computan. Un primer intento se descartó como piloto: los dos participantes compartieron cuenta y código, sus registros se mezclaron, no trabajaron de forma independiente y dos campos se interpretaron mal por una redacción ambigua de la página; su mediana del paso 1 (30,3 s) se informa solo como referencia. El intento válido usó una página por participante, con el código fijo y ejemplos por campo (docs/evidencia/b1/).

Resultado. La mediana por evento del paso 1 —la mediana entre participantes de cada evento y, luego, la de los 20 eventos, como fijó el registro previo— es de 18,44 s (media 29,94 s); las medianas individuales fueron de 18,85 s y 13,54 s. La media es mayor porque unos pocos eventos muy lentos la elevan (máximo individual de 200 s); con ella, la latencia media de la ingesta equivale al 4,9 %, y con la mediana registrada, al 7,9 %. El paso 2 tuvo medianas de 12,0 s y 11,2 s, y la mediana por evento de ambos pasos fue de 34,46 s. En el paso 1 ninguno cometió errores: los 100 campos de cada participante coinciden con el log, y el tipo y la relevancia son correctos en los 20 eventos. En el paso 2, con el criterio literal, cada uno clasificó correctamente 12 de 20 eventos; casi todas las diferencias son de formato de la credencial

(usuario / contraseña en lugar de usuario:contraseña) y, sin contarlas, quedan 16 de 20 y 19 de 20 (entre ambos, un valor de IoC equivocado, dos tipos de IoC y dos severidades). La medición queda por encima de la estimación en las fases comparables (18,44 s frente a ≈10,5 s en los puntos medios de las fases 1 y 2, y 34,46 s frente a ≈19,5 s en las fases 1, 2, 4 y 5): la Tabla VIII-1 subestimaba el trabajo manual, y la comparación de P1 con la línea de base medida es más favorable al pipeline que la original (§6.9). Los participantes integran el equipo autor y la muestra es de dos personas y 20 eventos, por lo que la cifra no es una medición representativa (§7.4).

Medición complementaria con participantes externos (05/10/2026). Para atenuar el sesgo de coautoría, cinco participantes ajenos al equipo (E1 a E5: tres estudiantes de informática o sistemas, uno con cursos de seguridad y uno con otra formación) procesaron la misma muestra con páginas idénticas salvo el código. Usaron de forma consecutiva la cuenta y el equipo de uno de los autores, sin ver la pantalla de los demás; antes de su turno, ese autor les explicó verbalmente el flujo de trabajo de la página. La mediana por evento del paso 1 fue de 10,36 s (media 10,69 s), con medianas individuales que tienden a descender con el orden de participación; con los siete participantes, la mediana por evento es de 11,50 s. En el paso 1 no hubo errores en los 500 campos; el tipo de evento fue correcto en los 100 eventos y la relevancia, en 97. En el paso 2, con el criterio literal, clasificaron correctamente entre 10 y 12 de 20 eventos; sin contar el formato de la credencial, que los siete participantes escribieron como usuario / contraseña, entre 17 y 19 de 20. La línea de base de los externos es menor que

la del equipo, de modo que el conocimiento del sistema no acortó la medición registrada; la explicación previa del autor y el uso consecutivo de un mismo equipo pueden haber acortado la de los externos.

Intento de ampliación (B1b). Se intentó ampliar la línea de base con más participantes ajenos al equipo; la medición no se completó y no se usa en ninguna cifra de este trabajo. El registro previo y el cierre están en docs/PREREGISTRO_B1b.md (tags 06f y 06g; Anexo X).

Ampliación con registro previo (B1c; 08/10/2026). Con el registro previo publicado (docs/PREREGISTRO_B1c.md), diez personas ajenas al equipo (E6 a E15: siete estudiantes de informática o sistemas y tres con formación en seguridad) procesaron la misma muestra de 20 eventos en el equipo de un autor, por turnos y sin ver la pantalla de las demás. La mediana por evento del paso 1 fue de 11,01 s (media 11,32 s; medianas individuales de 9,92 a 12,91 s; 10,92 s con los quince externos) y la mediana total de los dos pasos, de 17,83 s. El paso 1 fue correcto en 196 de 200 eventos; el paso 2, en 113 con el criterio literal (79 errores de valor por escribir usuario / contraseña, como en B1) y en 190 leyendo esa credencial como usuario:contraseña (análisis posterior). Se declara que el botón de reinicio dejó vacíos el inicio y el perfil de tres archivos (E6, E7 y E10; el perfil lo informaron los autores), una pausa de unos 14 min de E15 (no se descartó) y una primera tanda previa a la publicación, descartada entera. Otro análisis posterior, no fijado en el registro previo, es el intervalo bootstrap del 95 % de la mediana, de 10,44 a 11,93 s (reducción de la latencia de W2 y de C2 del 84,7 al 88,5 %). P1 se confirma con esta referencia, sin cambiar ningún veredicto. Detalle: docs/RESULTADOS_B1c.md y docs/evidencia/b1c/resultados/.

Descomposición estimada del procesamiento manual. La Tabla VIII-1, trasladada del §6.12.1, presenta la estimación propia del equipo, por fase, del tiempo de procesamiento manual de un evento; las fases 1 y 2 son las que se midieron con las personas participantes.

Tabla VIII-1: Descomposición del tiempo de procesamiento manual por fase.

#	Fase	Rango estimado	Justificación
1	Recepción y triaje inicial — apertura del log, identificación del eventid, determinación de relevancia inmediata	3–5 s	Estimación del equipo basada en su propia práctica: los eventos Cowrie llegan con campos indexados (eventid, src_ip, timestamp) que permiten un triaje rápido. Fase Detección y Análisis (Cichonski et al., 2012, §3.2).
2	Extracción manual de campos — lectura del JSON crudo, identificación y anotación de src_ip, username, password, input, session, timestamp	5–8 s	Estimación del equipo: un evento Cowrie contiene entre 8 y 12 campos; el analista debe leer el JSON, localizar los campos de interés y copiarlos a una plantilla de análisis. Fase Detección y Análisis (Cichonski et al., 2012, §3.2).
3	Enriquecimiento con fuentes externas — consulta de IP en plataformas de threat intelligence, geolocalización, verificación de reputación	8–12 s	Estimación del equipo: fase más demorada; cada consulta HTTP externa toma 1–3 s y el analista cruza resultados de 2–3 pestañas. Mapea a la consulta de país del nodo Code de event-ingest y al paso de enrichment del ciclo SOC.
4	Clasificación y decisión de severidad — asignación de nivel de criticidad según el contexto enriquecido	3–5 s	Estimación del equipo: con los campos extraídos y el enriquecimiento disponible, el analista aplica criterios de clasificación. Fase Detección y Análisis (Cichonski et al., 2012, §3.2).

#	Fase	Rango estimado	Justificación
5	Documentación del hallazgo — registro del IoC, actualización del ticket, notas de contexto para seguimiento	4–6 s	Estimación del equipo: escritura de una entrada de hallazgo y copia de IoCs a formato estructurado. Mapea a la fase de documentación del ciclo SOC.
Total		23–36 s (≈30 s)	Verificación de la suma: puntos medios de cada rango = 4 + 6,5 + 10 + 4 + 5 = 29,5 s. Se reporta como ≈30 s por consistencia con las demás tablas; el redondeo subestima la ventaja del pipeline en 0,5 s por evento.

Nota. Esta descomposición y su total son una estimación propia del equipo; no provienen de cifras publicadas por NIST, SANS, IBM Security, Verizon ni ENISA. Los anclajes a NIST SP 800-61 Rev. 2 remiten a las fases del ciclo de manejo de incidentes, no al valor de tiempo, que es de elaboración propia.

Anexo IX: Modelo de madurez SIM3 y estándares de representación de IoCs

Nota de alcance. Este anexo reúne el detalle del modelo de madurez SIM3, la autoevaluación del prototipo y la comparación de estándares de IoCs, trasladados del Capítulo III porque el trabajo los usa solo como marco interpretativo: SIM3 mide la madurez de un equipo de respuesta a incidentes y ninguno de los estándares se implementa en el prototipo (§3.10, §3.11).

IX.1 SIM3 (Security Incident Management Maturity Model)

SIM3 es un modelo de madurez desarrollado por Don Stikvoort, con derechos de la Open CSIRT Foundation, S-CURE y PRESECURE (2008-2019), que TF-CSIRT y su modelo de confianza Trusted Introducer adoptaron en mayo de 2010 (Stikvoort, 2019); ENISA, FIRST y la red de CSIRT de la UE lo adoptaron en su versión v2 interim o están en proceso de hacerlo, aunque ninguno es su autor (Open CSIRT Foundation, s.f.). En su versión mkXVIIIc (SIM3 v1) evalúa 44 parámetros agrupados en cuatro cuadrantes, con una escala única de cinco niveles, de 0 a 4 (Stikvoort, 2019). Se usa esa versión por estar completa y estable; el 1 de enero de 2023 se publicó SIM3 v2 interim (Open CSIRT Foundation, 2023), compatible con la v1 salvo por un parámetro agregado y cambios de redacción. Esta tesis no evalúa los 44 parámetros; su detalle, complementario, está en docs/ANEXOS_EXTENDIDOS.md del repositorio.

Una autoevaluación preliminar de la arquitectura contra los cuatro cuadrantes de SIM3 (Organisation, Human, Tools, Processes) se presenta, por dimensión, en la Tabla IX-1 (IX.2), sobre la escala oficial de 0 a 4. Es una evaluación orientativa del equipo, no exhaustiva sobre los 44 parámetros, y no sustituye una autoevaluación formal con la plantilla oficial. SIM3 mide la madurez de un equipo de respuesta a incidentes (CSIRT) y aquí se lo toma como marco de referencia para los componentes técnicos de un prototipo: los niveles asignados son indicativos y la correspondencia con los parámetros es interpretativa (el nivel 3 exige una

formalización avalada por la dirección del CSIRT (sellada o publicada), que un proyecto académico no tiene en sentido estricto).

IX.2 Autoevaluación del proyecto

La Tabla IX-1 resume la madurez actual de la arquitectura propuesta según los cuadrantes de SIM3 y, para las capacidades que SIM3 no cubre, con una descripción cualitativa sin escala.

Tabla IX-1: Autoevaluación integrada de madurez de la arquitectura.

Dimensión	Modelo de referencia	Nivel estimado	Observación
Captura de eventos	SIM3-Tools	2 — Explícito, interno	Cowrie con registro estructurado en JSON; un único honeypot (Dionaea previsto en el diseño, no desplegado)
Procesamiento automatizado	SIM3-Processes	2 — Explícito, interno	Flujos n8n operativos con parseo y transformación; el enriquecimiento se limita al país (fila Enriquecimiento contextual)
Generación de IoCs	Descriptivo (sin escala)	Implementada, sin estándar	IoCs generados de forma automática en un formato propio; sin STIX
Enriquecimiento contextual	Descriptivo (sin escala)	Parcial	Geolocalización por país implementada, verificada en prueba y no ejercida con tráfico real; reputación IP prevista, no implementada
Diseminación de inteligencia	SIM3-Processes	2 — Explícito, interno	Reportes automatizados, pendiente integración MISP/TAXII
Documentación metodológica	SIM3-Organisation	2 — Explícito, interno	Metodología documentada en el presente trabajo; el nivel 3 exigiría un aval formal que un proyecto académico no tiene
Gobernanza y organización	SIM3-Organisation	1 — Implícito	Proyecto académico sin estructura organizacional sostenida
Recursos humanos	SIM3-Human	2 — Explícito, interno	Equipo capacitado, pero sin dedicación exclusiva a operaciones

Nota. Elaboración propia.

Esta autoevaluación permite identificar las brechas de madurez: las principales fortalezas residen en la captura y el procesamiento automatizado (nivel 2), mientras que las oportunidades de mejora se concentran en la diseminación externa y en la integración con el ecosistema de compartición, que el prototipo no implementa. Estas brechas constituyen líneas de trabajo futuro identificadas en el §9.4 y en el §8.4.

IX.3 Estándares de representación, transporte y enriquecimiento de IoCs

Tabla IX-2: Estándares de representación, transporte y enriquecimiento de IoCs.

Estándar	Qué define y cómo	Caso de uso en el proyecto	Fuente
STIX 2.1	Lenguaje de OASIS basado en JSON que modela el ecosistema completo de una amenaza mediante STIX Domain Objects (SDOs: Indicator, Observed Data, Attack Pattern, Threat Actor, Malware, Course of Action, entre otros) conectados por STIX Relationship Objects (SROs): Relationship tipa vínculos como indicates, targets, uses o mitigates, y Sighting registra cuándo y por quién fue observado un indicador. Los indicadores se expresan con el STIX Patterning Language (p. ej., [ipv4-addr:value = '203.0.113.42']), que soporta comparaciones, pertenencia a conjuntos, expresiones regulares y combinaciones AND/OR. Es extensible mediante objetos personalizados; sus limitaciones son la complejidad de implementación y el tamaño de los objetos JSON.	Estándar de salida de los IoCs, consumibles por plataformas externas (MISP, SIEM, firewalls) sin transformaciones adicionales; fundamento conceptual de la integración futura con MISP. Comparación con los demás estándares en el Anexo IX, Tabla IX-2; estructura de objetos descripta en OASIS (2021a).	OASIS (2021a)
TAXII 2.1	Protocolo de transporte de OASIS que define cómo se intercambian los objetos STIX (STIX define qué se dice; TAXII, cómo y entre quiénes). Distingue TAXII Server, que expone API Roots con colecciones, y TAXII Client, que las consume o publica mediante una API REST sobre HTTPS. Define dos servicios para soportar distintos modelos de intercambio: Collections (petición-respuesta: el productor aloja objetos que los consumidores solicitan) y Channels (publicación-suscripción: reservado en la versión 2.1 y aún no especificado).	No implementado en la fase inicial; vía natural de diseminación estandarizada hacia CSIRTs, MISP o SIEM una vez que los IoCs se representen como objetos STIX, sin adaptadores propietarios.	OASIS (2021b)
OpenIOC	Formato XML de Mandiant, publicado como estándar abierto en 2011 tras un período de uso interno (FireEye adquirió Mandiant a fines de 2013), uno de los primeros formatos abiertos para el intercambio de IoCs, usado sobre todo en el ámbito forense. Define un indicador como expresión booleana (AND/OR/NOT) sobre indicadores	No se usa como formato primario de salida; se documenta para asegurar interoperabilidad con herramientas forenses legadas. Comparación con STIX y TAXII en el Anexo IX, Tabla IX-2.	Kampanakis (2014)
	atómicos (hashes, direcciones IP, rutas de archivo, claves de registro).		
Taxonomías MISP	MISP es una plataforma open source para el intercambio, almacenamiento, correlación y enriquecimiento de inteligencia que publica taxonomías estandarizadas en forma de etiquetas de máquina (namespace:predicado="valor") para clasificar eventos e indicadores. Taxonomías aplicables: tlp (restricción de difusión: clear/white, green, amber, red), estimative-language (confianza del juicio analítico: low, moderate, high), admiralty-scale (confiabilidad de la fuente y credibilidad de la información) y rsit (tipo de incidente, p. ej. information-gathering:scanner, intrusion-attempts:brute-force, malicious-code:malware-distribution); el tipo de indicador lo expresa el propio tipo de atributo MISP (ip-src, sha256, etc.). P. ej., un hash capturado por Dionaea puede registrarse como atributo sha256 con las etiquetas tlp:green, rsit:malicious-code="malware-distribution" y estimative-language:confidence-in-analytic-judgment="high".	Vocabulario complementario de enriquecimiento; la integración con MISP es una propuesta no implementada en esta tesis.	MISP Project (s.f.); Wagner et al. (2016)

Nota. Elaboración propia a partir de las fuentes indicadas.

Anexo X: Trazabilidad del repositorio

Nota de alcance. Este anexo reúne la trazabilidad de las versiones del repositorio citadas en la tesis; los tags se abrevian en la tabla.

Versión de la portada. Código, workflows y evidencia de las ventanas L0 a C2 idénticos a los del tag Honeypot_Final_2026-10-06e (commit e858cc4); difiere solo en documentación, en scripts y archivos de análisis de B1c y de B4, en la evidencia de C3 (docs/evidencia/b4/), en el retiro de la landing page y en dos cambios de la demo (API/public/index.html).

Tabla X-1: Tags del repositorio citados en esta tesis.

Tag	Qué agrega (commit)	Validación a la que se asocia
Honeypot_Final_2026-09-28	fix del extractor, pin de imágenes (7882a30)	W1 (§6.2.4)
Honeypot_Final_2026-09-29 / 2026-09-30	Honeypot_Final_2026-09-29: código y evidencia idénticos a 2026-09-28c; solo cambia BITACORA.md (3e42c14). Honeypot_Final_2026-09-30: suma los cambios de código del 29/09 previos a la ventana (extractor que ignora ítems vacíos, eb3f585; geolocalización, 0a3f523; tabla ioc_sessions, 2e383b1) y la evidencia de W2 (b41f900)	09-29: W1 (entregado con la tesis v11, según el mensaje del tag). 09-30: tag de código de W2: el último commit que cambia código o workflows (2e383b1) es anterior al inicio de W2
Honeypot_Final_2026-10-02	evidencia del 30/09 (4f44f83)	base de C1 (código)
Honeypot_Final_2026-10-02-B2	registro previo de C1 y línea de base B1 (1378677)	evidencia de W1, W2 y C1; plantilla de credencial con "ssl": false
Honeypot_Final_2026-10-05	plantilla de credencial corregida (17d0a51)	—
Honeypot_Final_2026-10-05d	último tag con SQL concatenado (cd4e043)	—
Honeypot_Final_2026-10-05e	consultas parametrizadas, segmentación, health-monitor, prueba de carga (fa01b95)	C2
Honeypot_Final_2026-10-06	evidencia de C2 (docs/evidencia/b3/)	—
Honeypot_Final_2026-10-06b	forwarder: registro y envío en hilo aparte de Telegram (e3c9bad)	—
Honeypot_Final_2026-10-06c	correcciones, endurecimiento, cowrie-proxy, internal en las tres redes (486954c)	pruebas de docs/evidencia/correcciones/ (sin ventana propia)
Honeypot_Final_2026-10-06d	evaluación del reporte (c9af3c3)	§7.7.3
Honeypot_Final_2026-10-06e	congelado de README y BITACORA (e858cc4); código = 06c	congelado final del código y la evidencia (heredado por los tags 07 a 08g)
Honeypot_Final_2026-10-07	solo documentación respecto del 06e: corrección de BITACORA.md (be6d182) y registro previo y cierre de B1b (tags 06f, 701e39e, y 06g, 26408da)	hereda el código y la evidencia del 06e
Honeypot_Final_2026-10-08	documentación y retiro de la landing page y de dos elementos de la demo respecto del 07 (abf2485); agrega docs/ANEXOS_EXTENDIDOS.md	hereda el código y la evidencia del 06e
Honeypot_Final_2026-10-08b a 08e	B1c: registro previo (08b), resultados (08c), análisis posterior (08d) y verificación de cifras (08e)	hereda el código y la evidencia del 06e
Honeypot_Final_2026-10-08f y 08g	B4: registro previo con los scripts de la ventana (08f, 6aaf3ef) y evidencia y resultados (08g, 930acf6)	C3 (docs/evidencia/b4/); tag de la portada: hereda el código del 06e

Nota. Los tags se abrevian (05e es Honeypot_Final_2026-10-05e). Los mensajes de commit se tomaron del repositorio. Elaboración propia.

Validación R1. R1 (10–11/10/2026) agrega al código del tag de la portada solo attack-runner/replay_dataset.py, scripts/r1_* y las cuentas de la muestra en cowrie/userdb.txt; el resto del código, los workflows y docker-compose.yml son idénticos. Registro previo: tags Honeypot_R1_prereg y Honeypot_R1_prereg2; evidencia: Honeypot_R1_evidencia; réplica C: Honeypot_Rep_evidencia.

Referencias

Akamai Technologies. (2024). Digital fortresses under siege: Threats to modern application architectures (State of the Internet Report). https://www.akamai.com/resources/state-of-the-internet/securing-apps-report-2024

Almeshekah, M. H., & Spafford, E. H. (2016). Cyber security deception. En S. Jajodia, V. S. Subrahmanian, V. Swarup, & C. Wang (Eds.), Cyber deception: Building the scientific foundation (pp. 23–50). Springer. https://doi.org/10.1007/978-3-319-32699-3_2

Al-Mohannadi, H., Awan, I., Al Hamar, J., Cullen, A. J., Disso, J. P., & Armitage, L. (2018). Cyber threat intelligence from honeypot data using Elasticsearch. Proceedings of the 2018 IEEE 32nd International Conference on Advanced Information Networking and Applications (AINA) (pp. 900–906). IEEE. https://doi.org/10.1109/AINA.2018.00132

Amal, M., & Venkadesh, P. (2023). H-DOCTOR: Honeypot based firewall tuning for attack prevention. Measurement: Sensors, 25, 100664. https://doi.org/10.1016/j.measen.2022.100664

Bartwal, U., Mukhopadhyay, S., Negi, R., & Shukla, S. (2022). Security orchestration, automation, and response engine for deployment of behavioural honeypots. 2022 IEEE Conference on Dependable and Secure Computing (DSC). arXiv:2201.05326. https://arxiv.org/abs/2201.05326

Baser, M., Guven, E. Y., & Aydin, M. A. (2021). SSH and Telnet protocols attack analysis using honeypot technique: Analysis of SSH and Telnet honeypot. En *2021 6th International Conference on Computer Science and Engineering (UBMK)* (pp. 806–811). IEEE. https://doi.org/10.1109/UBMK52708.2021.9558948

Bejtlich, R. (2004). The Tao of network security monitoring: Beyond intrusion detection. Addison-Wesley.

Bianco, D. (2013, marzo). The Pyramid of Pain [Entrada de blog]. Enterprise Detection & Response. http://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html

Bromiley, M. (2016). Threat intelligence: What it is, and how to use it effectively [Informe técnico]. SANS Institute (patrocinado por NSFOCUS). https://nsfocusglobal.com/wp-content/uploads/2017/01/SANS_Whitepaper_Threat_Intelligence__What_It_Is__and_How_to_Use_It_Effectively.pdf

Casaclang, H., Ionescu, B., Kim, Y., & Jo, J.-Y. (2026). Self-hosted workflow automation for AI-based cybersecurity operation. Journal of The Colloquium for Information Systems Security Education, 13(1). https://doi.org/10.53735/cisse.v13i1.241

Chin, D. M., Isfaq, B., & Enoch, S. Y. (2025). A practical honeypot-based threat intelligence framework for cyber defence in the cloud. arXiv preprint arXiv:2512.05321. https://arxiv.org/abs/2512.05321

Cichonski, P., Millar, T., Grance, T., & Scarfone, K. (2012). Computer security incident handling guide (NIST Special Publication 800-61 Rev. 2). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-61r2

Cisco Talos. (2025). Cisco Talos 2024 year in review. https://blog.talosintelligence.com/2024yearinreview

Cowrie Project. (s.f.). Cowrie SSH/Telnet Honeypot Documentation. Recuperado el 29 de septiembre de 2026 de https://docs.cowrie.org/

Crowley, C., Filkins, B., & Pescatore, J. (2023). SANS 2023 SOC survey. SANS Institute. https://www.sans.org/white-papers/2023-sans-soc-survey

d'Ambrosio, N., Lista, C., Perrone, G., & Romano, S. P. (2025). SMASH: An SDN-MTD framework for efficient honeypot deployment and insider threat mitigation. Computer Networks, 269, Article 111327. https://doi.org/10.1016/j.comnet.2025.111327

Decreto 1558/2001. (2001). Reglamentación de la Ley 25.326 de Protección de los Datos Personales. Boletín Oficial de la República Argentina. http://servicios.infoleg.gob.ar/infolegInternet/anexos/70000-74999/70368/norma.htm

Deutsche Telekom Security GmbH. (2023). T-Pot: The all-in-one multi-honeypot platform. Recuperado el 29 de septiembre de 2026 de https://github.com/telekom-security/tpotce

Dionaea Project. (s.f.). Dionaea Documentation. Recuperado el 29 de septiembre de 2026 de https://github.com/DinoTools/dionaea

Franco, J., Aris, A., Canberk, B., & Uluagac, A. S. (2021). A survey of honeypots and honeynets for Internet of Things, Industrial Internet of Things, and cyber-physical systems. IEEE Communications Surveys & Tutorials, 23(4), 2351–2383. https://doi.org/10.1109/COMST.2021.3106669

Group-IB. (2024). Hi-tech crime trends 2023/2024. https://www.group-ib.com/resources/research-hub/hi-tech-crime-trends-2023-global/

Hernández Sampieri, R., Fernández Collado, C., & Baptista Lucio, P. (2014). Metodología de la investigación (6.ª ed.). McGraw-Hill Education.

Honeynet Project. (s.f.). About the Honeynet Project [Sitio web]. Recuperado el 2 de octubre de 2026 de https://www.honeynet.org/about/

Honeynet Project. (2009, 27 de mayo). Introducing Glastopf, a web application honeypot [Entrada de blog]. https://www.honeynet.org/2009/05/27/introducing-glastopf-a-web-application-honeypot/

hpfeeds Project. (2011). hpfeeds: Honeynet Project generic authenticated datafeed protocol [Software]. Recuperado el 29 de septiembre de 2026 de https://github.com/hpfeeds/hpfeeds

Ikuomenisan, G., & Morgan, Y. (2022). Meta-review of recent and landmark honeypot research and surveys. Journal of Information Security, 13(4), 181–209. https://doi.org/10.4236/jis.2022.134011

Ilg, N., Duplys, P., Sisejkovic, D., & Menth, M. (2023). A survey of contemporary open-source honeypots, frameworks, and tools. Journal of Network and Computer Applications, 220, 103737. https://doi.org/10.1016/j.jnca.2023.103737

Islam, C., Babar, M. A., & Nepal, S. (2019). A multi-vocal review of security orchestration. ACM Computing Surveys, 52(2), 1–45. https://doi.org/10.1145/3305268

Javadpour, A., Ja'fari, F., Taleb, T., Shojafar, M., & Benzaïd, C. (2024). A comprehensive survey on cyber deception techniques to improve honeypot performance. Computers & Security, 140, 103792. https://doi.org/10.1016/j.cose.2024.103792

Kampanakis, P. (2014). Security automation and threat information-sharing options. IEEE Security & Privacy, 12(5), 42–51. https://doi.org/10.1109/MSP.2014.99

Kindervag, J. (2010). Build security into your network's DNA: The Zero Trust network architecture [Informe]. Forrester Research.

Kinyua, J., & Awuah, L. (2021). AI/ML in security orchestration, automation and response: Future research directions. Intelligent Automation & Soft Computing, 28(2). https://doi.org/10.32604/iasc.2021.016240

Ley 25.326. (2000). Protección de los Datos Personales. Boletín Oficial de la República Argentina. http://servicios.infoleg.gob.ar/infolegInternet/anexos/60000-64999/64790/texact.htm

Lingenfelter, B., Vakilinia, I., & Sengupta, S. (2020). Analyzing variation among IoT botnets using medium interaction honeypots. En *2020 10th Annual Computing and Communication Workshop and Conference (CCWC)* (pp. 761–767). IEEE. https://doi.org/10.1109/CCWC47524.2020.9031234

MISP Project. (s.f.). MISP threat sharing platform documentation. Recuperado el 29 de septiembre de 2026 de https://www.misp-project.org/documentation/

MITRE Corporation. (2024). MITRE ATT&CK. Recuperado el 29 de septiembre de 2026 de https://attack.mitre.org

Morić, Z., Dakić, V., & Regvart, D. (2025). Advancing cybersecurity with honeypots and deception strategies. Informatics, 12(1), 14. https://doi.org/10.3390/informatics12010014

n8n GmbH. (s.f.). n8n Documentation. Recuperado el 29 de septiembre de 2026 de https://docs.n8n.io

Năstase, V.-I., Mihăilescu, M.-E., Weisz, S., Dagilis, L. V., Mihai, D., & Carabas, M. (2024). Cowrie SSH honeypot: Architecture, improvements and data visualization. En *2024 23rd RoEduNet Conference: Networking in Education and Research (RoEduNet)* (pp. 1–7). IEEE. https://doi.org/10.1109/RoEduNet64292.2024.10722609

Nawrocki, M., Wählisch, M., Schmidt, T. C., Keil, C., & Schönfelder, J. (2016). A survey on honeypot software and data analysis. arXiv preprint arXiv:1608.06249. https://doi.org/10.48550/arXiv.1608.06249

Neiva, C., Lawson, C., Bussa, T., & Sadowski, G. (2017). Innovation insight for security orchestration, automation and response. Gartner. https://www.gartner.com/en/documents/3834578

Nelson, A., Rekhi, S., Souppaya, M., & Scarfone, K. (2025). Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile (NIST Special Publication 800-61 Rev. 3). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-61r3

OASIS. (2021a). STIX Version 2.1 (OASIS Standard, 10 de junio de 2021). https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html

OASIS. (2021b). TAXII Version 2.1 (OASIS Standard, 10 de junio de 2021). https://docs.oasis-open.org/cti/taxii/v2.1/os/taxii-v2.1-os.html

Open CSIRT Foundation. (s.f.). SIM3: Security Incident Management Maturity Model. Recuperado el 29 de septiembre de 2026 de https://opencsirt.org/csirt-maturity/sim3-and-references/

Open CSIRT Foundation. (2023). SIM3 v2 interim: Security Incident Management Maturity Model. https://opencsirt.org/wp-content/uploads/2023/11/SIM3_v2_interim_standard.pdf

Parmaksız, H., & Turhan, Ö. (2026). Open-source automation in cyber incident response processes: A low-code-based continuous detection, verification, and response architecture. Beykent University Journal of Science and Engineering, 19(1), 32–54. https://doi.org/10.20854/bujse.1918683

Provos, N. (2004). A virtual honeypot framework. En Proceedings of the 13th USENIX Security Symposium (pp. 1–14). USENIX Association. https://www.usenix.org/legacy/publications/library/proceedings/sec04/tech/full_papers/provos/provos.pdf

Provos, N., & Holz, T. (2007). Virtual honeypots: From botnet tracking to intrusion detection. Addison-Wesley.

Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., & Lear, E. (1996). Address allocation for private Internets (RFC 1918; BCP 5). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc1918

Sanjeev, K., Janet, B., & Eswari, R. (2020). Automated cyber threat intelligence generation from honeypot data. En Inventive communication and computational technologies (Lecture Notes in Networks and Systems, Vol. 89, pp. 591–598). Springer. https://doi.org/10.1007/978-981-15-0146-3_56

SANS Internet Storm Center. (s.f.). Port 22 (tcp/udp) attack activity [Base de datos]. Recuperado el 2 de octubre de 2026 de https://isc.sans.edu/data/port.html?port=22

Shackleford, D. (2015). Who's using cyberthreat intelligence and how? A SANS survey. SANS Institute. https://cdn-cybersecurity.att.com/docs/SANS-Cyber-Threat-Intelligence-Survey-2015.pdf

Shodan. (s.f.). What is Shodan? [Documentación]. Recuperado el 2 de octubre de 2026 de https://help.shodan.io/the-basics/what-is-shodan

Shuffle. (s.f.). Shuffle: Security automation platform [Repositorio de software]. GitHub. Recuperado el 6 de octubre de 2026 de https://github.com/Shuffle/Shuffle

Sokol, P., Míšek, J., & Husák, M. (2017). Honeypots and honeynets: Issues of privacy. EURASIP Journal on Information Security, 2017(1), 4. https://doi.org/10.1186/s13635-017-0057-4

Sokol, P., Pekarčík, P., & Bajtoš, T. (2015). Data collection and data analysis in honeypots and honeynets. En Security and Protection of Information 2015 (Proceedings). University of Defence. https://web.archive.org/web/20180421062733/http://spi.unob.cz/papers/2015/2015-19.pdf

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. Proceedings of the 2010 IEEE Symposium on Security and Privacy (pp. 305–316). IEEE. https://doi.org/10.1109/SP.2010.25

SonicWall. (2024). 2024 SonicWall cyber threat report. SonicWall Inc. https://www.sonicwall.com/resources/white-papers/2024-sonicwall-cyber-threat-report

Spitzner, L. (2002). Honeypots: Tracking hackers. Addison-Wesley.

StackStorm. (s.f.). StackStorm: Event-driven automation platform [Repositorio de software]. GitHub. Recuperado el 6 de octubre de 2026 de https://github.com/StackStorm/st2

Stikvoort, D. (2019). SIM3: Security Incident Management Maturity Model (mkXVIIIc, versión del 1 de mayo de 2019). Open CSIRT Foundation. https://opencsirt.org/wp-content/uploads/2019/12/SIM3-mkXVIIIc.pdf

TheHive Project. (s.f.). Cortex: Observable analysis and active response engine [Repositorio de software]. GitHub. Recuperado el 6 de octubre de 2026 de https://github.com/TheHive-Project/Cortex

ThreatStream. (s.f.). Modern Honey Network: Sensor management platform [Repositorio de software]. GitHub. Recuperado el 29 de septiembre de 2026 de https://github.com/pwnlandia/mhn

Tounsi, W. (2019). What is cyber threat intelligence and how is it evolving? En W. Tounsi (Ed.), Cyber-vigilance and digital trust: Cyber security in the era of cloud computing and IoT (pp. 1–49). ISTE / Wiley. https://doi.org/10.1002/9781119618393.ch1

Tounsi, W., & Rais, H. (2018). A survey on technical threat intelligence in the age of sophisticated cyber attacks. Computers & Security, 72, 212–233. https://doi.org/10.1016/j.cose.2017.09.001

Vielberth, M., Böhm, F., Fichtinger, I., & Pernul, G. (2020). Security operations center: A systematic study and open challenges. IEEE Access, 8, 227756–227779. https://doi.org/10.1109/ACCESS.2020.3045514

Wagner, C., Dulaunoy, A., Wagener, G., & Iklody, A. (2016). MISP: The design and implementation of a collaborative threat intelligence sharing platform. Proceedings of the 2016 ACM on Workshop on Information Sharing and Collaborative Security (pp. 49–56). ACM. https://doi.org/10.1145/2994539.2994542

Wang, Z., Lv, S., Wang, H., You, J., Liu, S., Yuan, T., Lu, X., & Sun, L. (2025). Unveiling evolving threats: A data analysis for next-generation honeypot development. En 2025 44th International Symposium on Reliable Distributed Systems (SRDS) (pp. 207–217). IEEE. https://doi.org/10.1109/SRDS69199.2025.00032

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). Experimentation in software engineering. Springer. https://doi.org/10.1007/978-3-642-29044-2

Yin, R. K. (2018). Case study research and applications: Design and methods (6.ª ed.). SAGE Publications.